

Universidad del Istmo

Programacion de Computadoras · Unity 2D

CCPP

Derechito a la Iglesia

Documentacion Tecnica delCodigo

AUTORES

Jose David Navichoque Pineda

Jose Gonzalez

Mynor Barrios

Andre Gil

Videojuego Educativo · Unity 2D · C# · 2026

Universidad del Istmo

Documentacion generada automaticamente

Tabla de Contenidos

Tabla de Contenidos	2
Resumen del Proyecto	3
Estadisticas delCodigo Fuente	3
Arquitectura de Carpetas	3
Flujo General del Juego	3
Patrones de Diseno Utilizados	4
Colisiones	5
Configuraciones	9
Data	13
Enumeraciones	27
Estructuras	28
Interfaces	29
Managers	30
Misa	33
Misiones	39
Modelos	49

Resumen del Proyecto

CCPP — Derechito a la Iglesia es un videojuego educativo 2D desarrollado en **Unity** con **C#** como parte de un proyecto academico en la **Universidad del Istmo**. El juego ensena al jugador las posturas y gestos correctos durante una misa catolica, mediante una serie de minijuegos interactivos que evaluan el control del cuerpo (piernas, pecho, cabeza, manos, boca) en diferentes momentos de la liturgia.

La arquitectura del codigo sigue el patron **MVC (Model-View-Controller)** adaptado a Unity: las clases *Data* y *ScriptableObjects* representan el modelo, los *Controllers* implementan la logica de comportamiento y los sistemas de UI actuan como vista. Los *Managers* orquestan la comunicacion entre subsistemas mediante **eventos C# (event/delegate)**, manteniendo un bajo acoplamiento entre componentes.

Estadisticas del Codigo Fuente

59	5493	10
Archivos .cs	Lineas de codigo	Categorias

Arquitectura de Carpetas

Colisiones	3	Contiene scripts que detectan colisiones fisicas en el mundo del juego. Estos componentes se adjuntan a objetos de escen...
Configuraciones	6	ScriptableObjects de configuracion para cada minijuego. Definen parametros ajustables desde el Inspector de Unity como t...
Data	20	Clases de datos en tiempo de ejecucion (MonoBehaviours y ScriptableObjects) que almacenan el estado del jugador, la cama...
Enumeraciones	3	Enumeraciones de C# que definen conjuntos de valores con nombre. Facilitan la lectura del codigo y evitan el uso de ente...
Estructuras	2	Structs de C# que agrupan datos relacionados de forma ligera y eficiente. Se usan para representar resultados de accione...
Interfaces	1	Contratos de interfaz que definen el comportamiento minimo que deben implementar ciertas clases. Permiten el polimorfism...
Managers	2	Orquestadores centrales del juego. Escuchan eventos del sistema y delegan la logica al componente apropiado. El Minijueg...
Misa	3	Scripts especificos de la escena de Misa. Controlan el flujo narrativo de la ceremonia, la barra de vida/energia del jug...
Misiones	4	Implementaciones concretas del sistema de misiones. Cada clase extiende la base de mision y define las condiciones de in...
Modelos	15	Controladores MVC del juego. Implementan la logica de presentacion y comportamiento de cada entidad: jugador, camara, di...

Flujo General del Juego

El jugador inicia en la escena principal y avanza hacia la iglesia. Al colisionar con zonas de dialogo (*ColisionDialogo*) se activan conversaciones que bloquean el movimiento. Al llegar al altar, se activa el **GestorMisa** que orquesta las misiones (*Mision01*, *MisionMinijuego*, *MisionPosicion*, *MisionPosturaParado*). Cada mision puede requerir que el jugador complete un minijuego a traves del **MinijuegoManager**, el cual delega segun la parte del cuerpo seleccionada en el panel *CuerpoUIController*. Al completar todos los objetivos, *FinalJuego* registra el resultado y termina la partida.

Patrones de Diseno Utilizados

MVC Adaptado a Unity	El proyecto adapta el patron MVC a las restricciones del engine. Los *Data (MonoBehaviour) actuan como Modelo: almacenan el estado sin logica. Los *Controller leen los Data y manipulan la escena, cumpliendo el rol de Controlador. La Vista es Unity mismo: prefabs, Canvas y el Inspector. La adaptacion clave es que los Controllers no renderizan, solo escriben en componentes de Unity (Animator, Image, RectTransform) y Unity se encarga del render.
Observer (event / delegate de C#)	Desacopla emisores de receptores sin que se conozcan entre si. Implementaciones en el proyecto: <i>CuerpoUIController.OnParteSeleccionada</i> → <i>MinijuegoManager</i> reacciona y lanza el minijuego. <i>DialogoController.OnDialogoTerminado</i> → <i>ColisionDialogo</i> y <i>GestorMisa</i> reanudan su flujo. <i>PanelManoController.OnSaludoSeleccionado</i> / <i>OnSenalSeleccionada</i> → <i>MinijuegoManager</i> . <i>PanelBocaController.OnResponderSeleccionado</i> / <i>OnCantarSeleccionado</i> → <i>MinijuegoManager</i> . Patron registrado en <i>OnEnable()</i> y liberado en <i>OnDisable()</i> para evitar memory leaks.
Command (MisionBase + callbacks)	Cada mision encapsula una accion diferida. <i>MisionBase</i> define el contrato: <i>TipoEsperado</i> , <i>ValorEsperado</i> y el metodo abstracto <i>Actuar()</i> que se ejecuta al cumplirse. <i>MisionPosicion</i> lo extiende recibiendo dos delegates (<i>onExito</i> , <i>onFallo</i>) en su constructor, lo que permite que <i>GestorMisa</i> defina el comportamiento sin que la mision conozca el flujo. <i>MisionesGlobal</i> actua como <i>Invoker</i> : recibe <i>AccionResultado</i> , evalua y dispara <i>Actuar()</i> o <i>ActuarFallo()</i> .
Singleton (MisionesGlobal)	<i>MisionesGlobal</i> utiliza el patron Singleton con <i>DontDestroyOnLoad</i> para persistir entre cambios de escena. Su propiedad estatica <i>Instancia</i> provee acceso global. El <i>Awake()</i> protege contra duplicados destruyendo la instancia sobrante. Es el unico Singleton del proyecto, lo que mantiene acotado el estado global.

Colisiones

Contiene scripts que detectan colisiones físicas en el mundo del juego. Estos componentes se adjuntan a objetos de escena y reaccionan cuando el jugador entra en contacto con ellos, disparando diálogos, efectos o el fin de la partida.

Archivo	Lineas	Ruta relativa
ColisionDialogo.cs	57	<i>colisiones/ColisionDialogo.cs</i>
FinalJuego.cs	232	<i>colisiones/FinalJuego.cs</i>
TriggerDesactivarObjeto.cs	29	<i>colisiones/TriggerDesactivarObjeto.cs</i>

ColisionDialogo.cs

57 líneas

Scripts/colisiones/ColisionDialogo.cs

Descripcion	Componente de trigger que se adjunta a zonas de colision en la escena. Cuando el jugador entra en el area (OnTriggerEnter), bloquea su movimiento mediante JugadorData.puedeMoverse y lanza una conversacion a traves de DialogoController. Al terminar el dialogo (evento OnDialogoTerminado) restaura el movimiento. Dependencias: JugadorData, DialogoController, DialogoData.
--------------------	---

```
using UnityEngine;
/// <summary>
/// Al colisionar con el jugador lanza un dialogo, bloquea movimiento
/// y al terminar lo reactiva. Se puede repetir al regresar.
/// </summary>
public class ColisionDialogo : MonoBehaviour
{
    [Header("Dialogo")]
    public DialogoData dialogo;
    [Header("Jugador")]
    public JugadorData jugadorData;
    private DialogoController _dialogoController;
    private bool _disparado = false;
    private void Awake()
    {
        _dialogoController = FindFirstObjectByType<DialogoController>();
        jugadorData = FindFirstObjectByType<JugadorData>();
    }
    /// <summary>
    /// Al colisionar con el jugador inicia el dialogo y bloquea el movimiento.
    /// Se ignora si ya hay un dialogo activo en este trigger.
    /// </summary>
    private void OnCollisionEnter(Collision other)
    {
        if (_disparado) return;
        if (!other.gameObject.CompareTag("Player")) return;
        if (_dialogoController == null)
        {
            Debug.LogWarning("[ColisionDialogo] No se encontro DialogoController.");
            return;
        }
        _disparado = true;
        if (jugadorData != null) jugadorData.puedeMoverse = false;
        DialogoController.OnDialogoTerminado += AlTerminarDialogo;
        _dialogoController.Iniciar(dialogo);
    }
    /// <summary>
    /// Se llama al terminar el dialogo: desuscribe el handler,
    /// reactiva el movimiento del jugador y permite disparar el dialogo de nuevo.
    /// </summary>
    private void AlTerminarDialogo()
```

```

{
    DialogoController.OnDialogoTerminado -= AlTerminarDialogo;
    if (jugadorData != null) jugadorData.puedeMoverse = true;
    _disparado = false;
}
}

```

FinalJuego.cs

232 líneas

Scripts/collisiones/FinalJuego.cs

Descripcion	Gestiona la pantalla de fin de juego. Detecta la colision del jugador con la zona final, detiene el movimiento y muestra el resultado. Contiene la logica de puntuacion final y la transicion a la pantalla de creditos o reinicio. Dependencias: JugadorData, SceneManager de Unity.
--------------------	---

```

using UnityEngine;
using UnityEngine.UI;
using UnityEngine.Video;
using UnityEngine.SceneManagement;
using System.Collections;

/// <summary>
/// Al colisionar con el jugador dispara la cinematica final:
/// desactiva objetos → paneles automaticos con fade → video con fade de audio
/// → pantalla de gracias → escena de inicio.
/// </summary>
[RequireComponent(typeof(Collider))]
public class FinalJuego : MonoBehaviour
{
    [Header("Jugador")]
    public JugadorData jugadorData;
    [Header("Objetos a desactivar y activar")]
    public GameObject[] objetosADesactivar;
    public GameObject[] objetosAActivar;
    [Header("Fade")]
    [Tooltip("Image negra que cubre toda la pantalla para los fades")]
    public Image imagenNegra;
    [Tooltip("Velocidad del fade in/out")]
    public float velocidadFade = 2f;
    [Header("Paneles finales")]
    public GameObject[] paneles;
    [Tooltip("Segundos que se muestra cada panel ya visible")]
    public float tiempoPorPanel = 3f;
    [Header("Video")]
    public VideoPlayer videoPlayer;
    public AudioSource audioVideo;
    [Tooltip("Segundos antes del fin del video en que empieza el fade de audio")]
    public float tiempoFadeAntesDeFin = 3f;
    [Header("Pantalla final")]
    public GameObject panelGracias;
    public float tiempoPantallaGracias = 3f;
    [Header("Escena de inicio")]
    public string nombreEscenaInicio = "Inicio";
    private bool _disparado = false;
    #region Unity Callbacks
    private void Awake()
    {
        GetComponent<Collider>().isTrigger = true;
        if (paneles != null)
            foreach (GameObject p in paneles)
                if (p != null) p.SetActive(false);
        if (panelGracias != null) panelGracias.SetActive(false);
        // Imagen negra empieza transparente
        if (imagenNegra != null)
        {
            imagenNegra.gameObject.SetActive(true);
            SetAlphaNegra(0f);
        }
    }
}

```

```

}
private void OnTriggerEnter(Collider other)
{
    if (_disparado) return;
    if (!other.CompareTag("Player")) return;
    _disparado = true;
    StartCoroutine(SecuenciaFinal());
}
#endregion
#region Secuencia Final
/// <summary>
/// Orquesta la secuencia final del juego: bloquea al jugador, hace fade,
/// muestra paneles, reproduce el video y carga la escena de inicio.
/// </summary>
private IEnumerator SecuenciaFinal()
{
    // 1 - Bloquear jugador
    if (jugadorData != null) jugadorData.puedeMoverse = false;
    // 2 - Fade in negro inicial
    yield return StartCoroutine(FadeNegra(0f, 1f));
    // 3 - Desactivar / activar objetos
    if (objetosADesactivar != null)
    foreach (GameObject obj in objetosADesactivar)
    if (obj != null) obj.SetActive(false);
    if (objetosAActivar != null)
    foreach (GameObject obj in objetosAActivar)
    if (obj != null) obj.SetActive(true);
    // 4 - Paneles automaticos
    yield return StartCoroutine(MostrarPaneles());
    // 5 - Video
    yield return StartCoroutine(ReproducirVideo());
    // 6 - Fade in negro antes de gracias
    yield return StartCoroutine(FadeNegra(0f, 1f));
    // 7 - Pantalla de gracias
    if (panelGracias != null) panelGracias.SetActive(true);
    yield return StartCoroutine(FadeNegra(1f, 0f));
    yield return new WaitForSeconds(tiempoPantallaGracias);
    // 8 - Fade out final y cargar escena
    yield return StartCoroutine(FadeNegra(0f, 1f));
    SceneManager.LoadScene(nombreEscenaInicio);
}
/// <summary>
/// Muestra cada panel de la secuencia final con fade in/out y tiempo de espera entre ellos.
/// </summary>
private IEnumerator MostrarPaneles()
{
    if (paneles == null) yield break;
    foreach (GameObject panel in paneles)
    {
        if (panel == null) continue;
        // Fade out negro → mostrar panel
        panel.SetActive(true);
        yield return StartCoroutine(FadeNegra(1f, 0f));
        // Esperar tiempo visible
        yield return new WaitForSeconds(tiempoPorPanel);
        // Fade in negro → ocultar panel
        yield return StartCoroutine(FadeNegra(0f, 1f));
        panel.SetActive(false);
    }
}
/// <summary>
/// Reproduce el VideoPlayer con fade de audio al final del clip
/// para una transicion suave hacia la pantalla de gracias.
/// </summary>
private IEnumerator ReproducirVideo()
{

```

```

if (videoPlayer == null) yield break;
videoPlayer.gameObject.SetActive(true);
videoPlayer.Play();
yield return new WaitUntil(() => videoPlayer.isPrepared);
// Fade out negro para revelar video
yield return StartCoroutine(FadeNegra(1f, 0f));
float duracion = (float)videoPlayer.length;
float tiempoEsperaAntesFade = duracion - tiempoFadeAntesDeFin;
if (tiempoEsperaAntesFade > 0f)
yield return new WaitForSeconds(tiempoEsperaAntesFade);
// Fade de audio mientras termina el video
if (audioVideo != null)
{
float volumenInicial = audioVideo.volume;
float tiempoTranscurrido = 0f;
while (tiempoTranscurrido < tiempoFadeAntesDeFin)
{
tiempoTranscurrido += Time.deltaTime;
float t = Mathf.Clamp01(tiempoTranscurrido / tiempoFadeAntesDeFin);
audioVideo.volume = Mathf.Lerp(volumenInicial, 0f, t);
yield return null;
}
audioVideo.volume = 0f;
}
else
{
yield return new WaitUntil(() => !videoPlayer.isPlaying);
}
videoPlayer.Stop();
videoPlayer.gameObject.SetActive(false);
}
#endregion
#region Fade
/// <summary>
/// Anima el alpha de la imagen negra desde un valor hasta otro para crear transiciones.
/// </summary>
/// <param name="desde">Alpha inicial (0 = transparente, 1 = opaco).</param>
/// <param name="hasta">Alpha final deseado.</param>
private IEnumerator FadeNegra(float desde, float hasta)
{
if (imagenNegra == null) yield break;
float tiempoTranscurrido = 0f;
float duracion = Mathf.Abs(hasta - desde) / velocidadFade;
while (tiempoTranscurrido < duracion)
{
tiempoTranscurrido += Time.deltaTime;
float t = Mathf.Clamp01(tiempoTranscurrido / duracion);
SetAlphaNegra(Mathf.Lerp(desde, hasta, t));
yield return null;
}
SetAlphaNegra(hasta);
}
/// <summary>
/// Aplica directamente el valor de alpha a la imagen negra.
/// </summary>
/// <param name="alpha">Valor de alpha a asignar (0-1).</param>
private void SetAlphaNegra(float alpha)
{
if (imagenNegra == null) return;
Color c = imagenNegra.color;
c.a = alpha;
imagenNegra.color = c;
}
#endregion
}

```


TriggerDesactivarObjeto.cs

29 líneas

Scripts/collisiones/TriggerDesactivarObjeto.cs

Descripción	Trigger simple de un solo uso. Al entrar el jugador, desactiva un GameObject referenciado en el Inspector. Útil para elementos de escena que deben desaparecer tras ser descubiertos (pistas, obstáculos). Se desactiva a sí mismo tras dispararse para evitar efectos duplicados. Dependencias: ninguna externa.
--------------------	---

```

using UnityEngine;

/// <summary>
/// Al entrar el jugador al trigger desactiva un GameObject específico.
/// Solo se dispara una vez.
/// </summary>
[RequireComponent(typeof(Collider))]
public class TriggerDesactivarObjeto : MonoBehaviour
{
    [Tooltip("Objeto a desactivar al entrar el jugador")]
    public GameObject objetoADesactivar;
    private void Awake()
    {
        GetComponent<Collider>().isTrigger = true;
    }

    /// <summary>
    /// Al entrar el jugador desactiva el objeto configurado y desactiva este script para que no se repita.
    /// </summary>
    private void OnTriggerEnter(Collider other)
    {
        if (!other.CompareTag("Player")) return;
        if (objetoADesactivar == null) return;
        objetoADesactivar.SetActive(false);
        this.enabled = false;
    }
}

```

Configuraciones

ScriptableObjects de configuración para cada minijuego. Definen parámetros ajustables desde el Inspector de Unity como tiempos, puntajes, sprites y textos, separando datos de lógica.

Archivo	Líneas	Ruta relativa
ConfigCanto.cs	36	<i>Configuraciones/ConfigCanto.cs</i>
ConfigCerebro.cs	34	<i>Configuraciones/ConfigCerebro.cs</i>
ConfigPecho.cs	35	<i>Configuraciones/ConfigPecho.cs</i>
ConfigResponder.cs	47	<i>Configuraciones/ConfigResponder.cs</i>
ConfigSaludo.cs	28	<i>Configuraciones/ConfigSaludo.cs</i>
ConfigSenal.cs	29	<i>Configuraciones/ConfigSenal.cs</i>

ConfigCanto.cs

36 líneas

Scripts/Configuraciones/ConfigCanto.cs

Descripción	ScriptableObject-style configurator para el minijuego de canto. Al activarse (OnEnable) copia sus valores serializados hacia MinijuegoCantoData, separando la configuración de la lógica. Permite diseñar múltiples dificultades o variaciones del canto desde el Inspector sin tocar código. Dependencias: MinijuegoCantoData.
--------------------	---

```

using UnityEngine;

/// <summary>

```

```

/// Al activarse aplica configuracion especifica al minijuego de canto.
/// </summary>
public class ConfigCanto : MonoBehaviour
{
    [Header("Referencia")]
    public MinijuegoCantoData data;
    [Header("Configuracion")]
    public float velocidadMin = 150f;
    public float velocidadMax = 280f;
    public float posYInicial = 400f;
    public float intervaloNotas = 1.2f;
    public float margenAcierto = 60f;
    public int notasParaGanar = 8;
    public int fallosPermitidos = 3;
    /// <summary>
    /// Copia los valores serializados de este Config al MinijuegoCantoData de referencia.
    /// Se ejecuta cada vez que este GameObject se activa en escena.
    /// </summary>
    private void OnEnable()
    {
        if (data == null) { Debug.LogWarning("[ConfigCanto] data no asignado."); return; }
        data.velocidadMin = velocidadMin;
        data.velocidadMax = velocidadMax;
        data.posYInicial = posYInicial;
        data.intervaloNotas = intervaloNotas;
        data.margenAcierto = margenAcierto;
        data.notasParaGanar = notasParaGanar;
        data.fallosPermitidos = fallosPermitidos;
    }
}

```

ConfigCerebro.cs

34 líneas

Scripts/Configuraciones/ConfigCerebro.cs

Descripcion	Configurador del minijuego de cerebro. Transfiere parametros de velocidad de distractores, perdida/ganancia de alpha del cerebro y tiempo de supervivencia hacia MinijuegoCerebroData. Permite balancear la dificultad por escena. Dependencias: MinijuegoCerebroData.
--------------------	--

```

using UnityEngine;
/// <summary>
/// Al activarse aplica configuracion especifica al minijuego de cerebro.
/// </summary>
public class ConfigCerebro : MonoBehaviour
{
    [Header("Referencia")]
    public MinijuegoCerebroData data;
    [Header("Configuracion")]
    public float perdidaAlpha = 0.08f;
    public float gananciaAlpha = 0.3f;
    public float velocidadMin = 60f;
    public float velocidadMax = 160f;
    public float radioColision = 50f;
    public float tiempoSobrevivir = 15f;
    /// <summary>
    /// Copia los valores serializados de este Config al MinijuegoCerebroData de referencia.
    /// Se ejecuta cada vez que este GameObject se activa en escena.
    /// </summary>
    private void OnEnable()
    {
        if (data == null) { Debug.LogWarning("[ConfigCerebro] data no asignado."); return; }
        data.perdidaAlpha = perdidaAlpha;
        data.gananciaAlpha = gananciaAlpha;
        data.velocidadMin = velocidadMin;
        data.velocidadMax = velocidadMax;
        data.radioColision = radioColision;
        data.tiempoSobrevivir = tiempoSobrevivir;
    }
}

```

```

}
}

```

ConfigPecho.cs

35 líneas

Scripts/Configuraciones/ConfigPecho.cs

Descripcion	Configurador del minijuego de pecho (ritmo cardiaco). Define cuantas rondas debe completar el jugador, el rango de velocidad del circulo decreciente y el margen de acierto. Escribe directamente en MinijuegoPechoData al activarse. Dependencias: MinijuegoPechoData.
--------------------	---

```

using UnityEngine;

/// <summary>
/// Al activarse aplica configuracion especifica al minijuego de pecho.
/// Coloca este script en el mismo GameObject que activas para la mision.
/// </summary>
public class ConfigPecho : MonoBehaviour
{
    [Header("Referencia")]
    public MinijuegoPechoData data;
    [Header("Configuracion")]
    public int rondasTotales = 4;
    public float velocidadMin = 80f;
    public float velocidadMax = 180f;
    public float margenAcierto = 30f;
    public float tamanoInicial = 400f;
    public float tamanoCorazon = 100f;

    /// <summary>
    /// Copia los valores serializados de este Config al MinijuegoPechoData de referencia.
    /// Se ejecuta cada vez que este GameObject se activa en escena.
    /// </summary>
    private void OnEnable()
    {
        if (data == null) { Debug.LogWarning("[ConfigPecho] data no asignado."); return; }
        data.rondasTotales = rondasTotales;
        data.velocidadMin = velocidadMin;
        data.velocidadMax = velocidadMax;
        data.margenAcierto = margenAcierto;
        data.tamanoInicial = tamanoInicial;
        data.tamanoCorazon = tamanoCorazon;
    }
}

```

ConfigResponder.cs

47 líneas

Scripts/Configuraciones/ConfigResponder.cs

Descripcion	Configurador del minijuego de preguntas y respuestas. Ademas de parametros de tiempo y barra, define el contenido de los tres botones de respuesta, el indice de la respuesta correcta y el texto de ayuda. Permite tener distintas preguntas por escena de misa sin duplicar prefabs. Dependencias: MinijuegoResponderData.
--------------------	--

```

using UnityEngine;

/// <summary>
/// Al activarse aplica configuracion especifica al minijuego de responder.
/// </summary>
public class ConfigResponder : MonoBehaviour
{
    [Header("Referencia")]
    public MinijuegoResponderData data;
    [Header("Configuracion")]
    public float tiempoLimite = 10f;
    public float anchoBarraInicial = 600f;
    public float alphaAyudaPorEspacio = 0.15f;
    public float aceleracionBarraPorEspacio = 1.5f;
    public int indiceRespuestaCorrecta = 0;
    [Header("Contenido")]
}

```

```

public string textoBoton0 = "Opcion A";
public string textoBoton1 = "Opcion B";
public string textoBoton2 = "Opcion C";
public string textoAyudaContenido = "La respuesta es...";

/// <summary>
/// Copia los valores serializados de este Config al MinijuegoResponderData de referencia,
/// incluyendo el texto de los tres botones y la ayuda.
/// Se ejecuta cada vez que este GameObject se activa en escena.
/// </summary>
private void OnEnable()
{
    if (data == null) { Debug.LogWarning("[ConfigResponder] data no asignado."); return; }
    data.tiempoLimite = tiempoLimite;
    data.anchoBarraInicial = anchoBarraInicial;
    data.alphaAyudaPorEspacio = alphaAyudaPorEspacio;
    data.aceleracionBarraPorEspacio = aceleracionBarraPorEspacio;
    data.indiceRespuestaCorrecta = indiceRespuestaCorrecta;
    data.textoAyudaContenido = textoAyudaContenido;
    data.textoBotones = new string[3]
    {
        textoBoton0,
        textoBoton1,
        textoBoton2
    };
}

```

ConfigSaludo.cs

28 líneas

Scripts/Configuraciones/ConfigSaludo.cs

Descripcion	Configurador del minijuego de saludo de mano. Ajusta el tiempo limite, el tiempo de exito para mostrar la animacion de apreton y el ancho inicial de la barra de tiempo. Dependencias: MinijuegoSaludoData.
--------------------	---

```

using UnityEngine;

/// <summary>
/// Al activarse aplica configuracion especifica al minijuego de saludo.
/// </summary>
public class ConfigSaludo : MonoBehaviour
{
    [Header("Referencia")]
    public MinijuegoSaludoData data;
    [Header("Configuracion")]
    public float tiempoLimite = 3f;
    public float tiempoExito = 1.2f;
    public float anchoBarraInicial = 400f;

    /// <summary>
    /// Copia los valores serializados de este Config al MinijuegoSaludoData de referencia.
    /// Se ejecuta cada vez que este GameObject se activa en escena.
    /// </summary>
    private void OnEnable()
    {
        if (data == null) { Debug.LogWarning("[ConfigSaludo] data no asignado."); return; }
        data.tiempoLimite = tiempoLimite;
        data.tiempoExito = tiempoExito;
        data.anchoBarraInicial = anchoBarraInicial;
    }
}

```

ConfigSenal.cs

29 líneas

Scripts/Configuraciones/ConfigSenal.cs

Descripcion	Configurador del minijuego de senal de la cruz. Controla el tiempo limite, el ancho de la barra y el flag esCompleja que determina si se usan los botones simples (4 pasos) o los complejos (mas pasos). Dependencias: MinijuegoSenalData.
--------------------	--

```

using UnityEngine;

/// <summary>
/// Al activarse aplica configuracion especifica al minijuego de senal.
/// </summary>
public class ConfigSenal : MonoBehaviour
{
    [Header("Referencia")]
    public MinijuegoSenalData data;
    [Header("Configuracion")]
    public float tiempoLimite = 5f;
    public float anchoBarraInicial = 400f;
    public bool esCompleja = false;

    /// <summary>
    /// Copia los valores serializados de este Config al MinijuegoSenalData de referencia,
    /// incluyendo el modo simple o complejo.
    /// Se ejecuta cada vez que este GameObject se activa en escena.
    /// </summary>
    private void OnEnable()
    {
        if (data == null) { Debug.LogWarning("[ConfigSenal] data no asignado."); return; }
        data.tiempoLimite = tiempoLimite;
        data.anchoBarraInicial = anchoBarraInicial;
        data.esCompleja = esCompleja;
    }
}

```

Data

Clases de datos en tiempo de ejecucion (MonoBehaviours y ScriptableObjects) que almacenan el estado del jugador, la camara, los dialogos y cada minijuego. Actuan como la 'memoria' viva del juego.

Archivo	Lineas	Ruta relativa
CamaraData.cs	21	Data/Camara/CamaraData.cs
CuerpoUIData.cs	24	Data/Cuerpo/CuerpoUIData.cs
DialogoData.cs	13	Data/Dialogo/DialogoData.cs
DialogoUIData.cs	29	Data/Dialogo/DialogoUIData.cs
JugadorData.cs	40	Data/Jugador/JugadorData.cs
JugadorHabilidadesData.cs	48	Data/Jugador/JugadorHabilidadesData.cs
DistractorCerebro.cs	47	Data/Minijuego/Cabeza/DistractorCerebro.cs
MinijuegoCerebroData.cs	44	Data/Minijuego/Cabeza/MinijuegoCerebroData.cs
MinijuegoCantoData.cs	43	Data/Minijuego/Cantar/MinijuegoCantoData.cs
NotaCanto.cs	78	Data/Minijuego/Cantar/NotaCanto.cs
MinijuegoSaludoData.cs	37	Data/Minijuego/Mano/MinijuegoSaludoData.cs
MinijuegoSenalData.cs	37	Data/Minijuego/Mano/MinijuegoSenalData.cs
PanelManoData.cs	21	Data/Minijuego/Mano/PanelManoData.cs

CeldaCuadriculaData.cs	51	Data/Minijuego/Pierna/CeldaCuadriculaData.cs
MinijuegoPiernaData.cs	86	Data/Minijuego/Pierna/MinijuegoPiernaData.cs
MinijuegoResponderData.cs	59	Data/Minijuego/boca/MinijuegoResponderData.cs
PanelBocaData.cs	17	Data/Minijuego/boca/PanelBocaData.cs
MinijuegoPechoData.cs	46	Data/Minijuego/pecho/MinijuegoPechoData.cs
ParteCuerpoData.cs	19	Data/Partes del cuerpo/ParteCuerpoData.cs
TransicionZonaData.cs	47	Data/transicion/TransicionZonaData.cs

CamaraData.cs

21 líneas

Scripts/Data/Camara/CamaraData.cs

Descripcion	Contenedor de datos para la camara principal del juego. Expone el Transform del jugador objetivo, la velocidad de suavizado (Smooth Follow) y el flag seguirJugador que CamaraController consulta cada frame. Al desactivarse seguirJugador durante dialogos o minijuegos, la camara se inmoviliza. Dependencias: CamaraController (lee los datos), JugadorData (origen del transform).
--------------------	---

```

/// <summary>
/// Contenedor de datos para la camara principal.
/// CamaraController lee estas referencias cada frame en LateUpdate.
/// Desactivar seguirJugador inmoviliza la camara durante dialogos o minijuegos.
/// </summary>
using UnityEngine;
/// <summary>
/// Contenedor de datos de la camara principal del juego.
/// Expone el objetivo a seguir y la velocidad de suavizado.
/// Sin logica, solo referencias que CamaraController consume.
/// </summary>
public class CamaraData : MonoBehaviour
{
    /// <summary>Transform del jugador que la camara sigue horizontalmente.</summary>
    [Header("Objetivo")]
    public Transform jugador;
    /// <summary>Factor de suavizado Lerp aplicado cada frame en LateUpdate.</summary>
    [Header("Configuracion")]
    public float velocidadSmooth = 5f;
    /// <summary>Si es false la camara deja de seguir al jugador (util en cinematicas).</summary>
    [Header("Estado")]
    public bool seguirJugador = true;
}

```

CuerpoUIData.cs

24 líneas

Scripts/Data/Cuerpo/CuerpoUIData.cs

Descripcion	Contenedor de referencias para el panel de seleccion de partes del cuerpo. Guarda el panel raiz y el arreglo de ParteCuerpoData con sus botones asociados. Tambien define los colores de estado bloqueado/desbloqueado que CuerpoUIController aplica. Dependencias: ParteCuerpoData, CuerpoUIController.
--------------------	--

```

using UnityEngine;
/// <summary>
/// Datos de la UI del cuerpo.
/// Expone el panel y el array de partes con sus botones.

```

```

/// Sin logica, solo referencias.
/// </summary>
public class CuerpoUIData : MonoBehaviour
{
    [Header("Panel principal")]
    [Tooltip("Panel que contiene todos los botones de partes del cuerpo")]
    public GameObject panelCuerpo;
    [Header("Partes del cuerpo")]
    [Tooltip("Arrastrar aqui los GameObjects que tengan el componente ParteCuerpoData")]
    public ParteCuerpoData[] partes;
    [Header("Colores de estado")]
    [Tooltip("Color del boton cuando la parte esta desbloqueada")]
    public Color colorDesbloqueado = Color.white;
    [Tooltip("Color del boton cuando la parte esta bloqueada")]
    public Color colorBloqueado = Color.gray;
}

```

DialogoData.cs

13 líneas

Scripts/Data/Dialogo/DialogoData.cs

Descripcion	Clase serializable que representa una conversacion completa como un arreglo de DialogoLinea. Al ser [Serializable] puede configurarse inline en el Inspector de cualquier MonoBehaviour. DialogoController la consume linea a linea. Dependencias: DialogoLinea.
--------------------	--

```

using UnityEngine;
/// <summary>
/// Contenedor de una conversacion completa.
/// Clase serializable, se rellena inline en el Inspector.
/// Sin logica, solo datos.
/// </summary>
[System.Serializable]
public class DialogoData
{
    [Tooltip("Lista de lineas de esta conversacion en orden")]
    public DialogoLinea[] lineas;
}

```

DialogoUIData.cs

29 líneas

Scripts/Data/Dialogo/DialogoUIData.cs

Descripcion	Centraliza todas las referencias UI del panel de dialogo: el panel raiz, el TextMeshPro de texto, las imagenes de personaje izquierda/derecha y la tecla para avanzar. DialogoController lee estos datos en cada frame para saber que actualizar. Dependencias: DialogoController, TMPPro.
--------------------	--

```

using UnityEngine;
using UnityEngine.UI;
using TMPro;
/// <summary>
/// Referencias a los elementos UI del panel de dialogo.
/// MonoBehaviour, sin logica, solo datos.
/// </summary>
public class DialogoUIData : MonoBehaviour
{
    [Header("Panel")]
    [Tooltip("Panel raiz del dialogo, se activa y desactiva")]
    public GameObject panelDialogo;
    [Header("Texto")]
    [Tooltip("TextMeshPro donde se muestra el texto de cada linea")]
    public TextMeshProUGUI textoDialogo;
    [Header("Imagenes de personaje")]
    [Tooltip("Imagen del lado izquierdo")]
    public Image imagenIzquierda;
    [Tooltip("Imagen del lado derecho")]
    public Image imagenDerecha;
    [Header("Configuracion")]
}

```

```
[Tooltip("Tecla para avanzar al siguiente dialogo")]
public KeyCode teclaAvanzar = KeyCode.Space;
}
```

JugadorData.cs

40 líneas

Scripts/Data/Jugador/JugadorData.cs

Descripcion	Fuente de verdad sobre el estado del jugador. Centraliza velocidad maxima, aceleracion, desaceleracion, direccion de input, el flag puedeMoverse (respetado por colisiones y gestores de misa), referencias de audio y nombres de animacion. JugadorController lee y escribe estos valores cada frame. Dependencias: JugadorController, ColisionDialogo, GestorMisa.
--------------------	--

```
using UnityEngine;
/// <summary>
/// Contenedor de datos del jugador.
/// No contiene logica, solo expone las variables
/// que JugadorController necesita para operar.
/// </summary>
public class JugadorData : MonoBehaviour
{
    [Header("Movimiento")]
    [Tooltip("Velocidad maxima en unidades/segundo")]
    public float velocidadMaxima = 5f;
    [Tooltip("Que tan rapido alcanza la velocidad maxima (unidades/s²)")]
    public float aceleracion = 20f;
    [Tooltip("Que tan rapido frena al soltar el input (unidades/s²)")]
    public float desaceleracion = 30f;
    [Header("Estado (solo lectura)")]
    [Tooltip("Direccion de input detectada este frame")]
    public Vector3 direccionInput = Vector3.zero;
    [Header("Condicion para moverse")]
    [Tooltip("Impide movimiento en ciertas acciones o cuando se desee")]
    public bool puedeMoverse = true;

    [Header("Audio")]
    public AudioSource audioCaminata;
    public AudioClip clipCaminata;
    [Header("Animacion")]
    public Animator animador;
    public SpriteRenderer spriteRenderer;
    [Header("Nombres de animacion")]
    public string animCaminando = "Caminando";
    public string animQuieto = "Quieto";
}
```

JugadorHabilidadesData.cs

48 líneas

Scripts/Data/Jugador/JugadorHabilidadesData.cs

Descripcion	Registra que partes del cuerpo tiene desbloqueadas el jugador mediante bools. Expone el metodo EstaDesbloqueada(ParteCuerpo) para que CuerpoUIController decida que botones habilitar. Al inicio todos los bools son true (todas las partes disponibles). Dependencias: CuerpoUIController, ParteCuerpo (enum).
--------------------	---

```
using UnityEngine;
/// <summary>
/// Fuente de verdad sobre que partes del cuerpo tiene desbloqueadas el jugador.
/// Solo datos, sin logica.
/// Vive en el mismo GameObject que JugadorData y JugadorController.
/// </summary>
public class JugadorHabilidadesData : MonoBehaviour
{
    [Header("Partes desbloqueadas")]
    [Tooltip("El jugador puede usar sus piernas")]
    public bool tienePiernas = true;
    [Tooltip("El jugador puede usar su pecho")]
    public bool tienePecho = true;
}
```



```
[Tooltip("El jugador puede usar su cabeza")]
public bool tieneCabeza = true;
[Tooltip("El jugador puede usar su boca")]
public bool tieneBoca = true;
[Tooltip("El jugador puede usar su mano derecha")]
public bool tieneManoDerecha = true;
#region API Publica
    /// <summary>
    /// Devuelve si una parte especifica esta desbloqueada.
    /// Util para consultar por enum sin exponer los bools directamente.
    /// </summary>
    public bool EstaDesbloqueada(ParteCuerpo parte)
    {
        switch (parte)
        {
            case ParteCuerpo.Piernas: return tienePiernas;
            case ParteCuerpo.Pecho: return tienePecho;
            case ParteCuerpo.Cabeza: return tieneCabeza;
            case ParteCuerpo.Boca: return tieneBoca;
            case ParteCuerpo.ManoDerecha: return tieneManoDerecha;
            default:
                Debug.LogWarning($"[JugadorHabilidadesData] Parte no reconocida: {parte}");
                return false;
        }
    }
#endregion
}
```

DistractorCerebro.cs

47 líneas

Scripts/Data/Minijuego/Cabeza/DistractorCerebro.cs

Descripción

Datos y comportamiento de un distractor individual en el minijuego de cerebro. Cada distractor tiene su Image, Button y estado de posición/velocidad/activo. El método Resetear() lo reposita al origen con nueva velocidad aleatoria; AlClickear() lo recicla al ser tocado por el jugador. Dependencias: MinijuegoCerebroController, MinijuegoCerebroData.

```
using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// Datos y estado de un distractor individual del minijuego de cerebro.
/// MonoBehaviour, vive en cada GameObject distractor.
/// </summary>
public class DistractorCerebro : MonoBehaviour
{
    [Tooltip("Image del distractor")]
    public Image imagen;
    [Tooltip("Boton para que el jugador pueda hacer click")]
    public Button boton;
    [HideInInspector] public Vector2 posicionOriginal;
    [HideInInspector] public float velocidadActual;
    [HideInInspector] public bool activo = false;
    private void Awake()
    {
        // Forzamos a que el RectTransform se asiente si esta dentro de un LayoutGroup
        Canvas.ForceUpdateCanvases();
        posicionOriginal = GetComponent<RectTransform>().anchoredPosition;
    }
    /// <summary>
    /// Resetea el distractor a su posicion original con nueva velocidad.
    /// </summary>
    public void Resetear(float nuevaVelocidad)
    {
        GetComponent<RectTransform>().anchoredPosition = posicionOriginal;
        velocidadActual = nuevaVelocidad;
        activo = true;
        if (imagen != null)

```

```

imagen.gameObject.SetActive(true);
}
/// <summary>
/// Vuelve a posicion original al ser clickeado.
/// </summary>
public void AlClickear(float velocidadMin, float velocidadMax)
{
    Resetear(Random.Range(velocidadMin, velocidadMax));
}
}

```

MinijuegoCerebroData.cs

44 lineas

Scripts/Data/Minijuego/Cabeza/MinijuegoCerebroData.cs

Descripcion	Contiene todas las referencias UI y parametros del minijuego de cabeza (cerebro): el panel, la imagen del cerebro, los cuatro distractores, y valores de perdida/ganancia de alpha, velocidad y radio de colision. MinijuegoCerebroController escribe el estado (tiempoTranscurrido, minijuegoActivo). Dependencias: DistractorCerebro, MinijuegoCerebroController.
--------------------	---

```

using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// Datos del minijuego de cerebro.
/// Solo datos, sin logica.
/// </summary>
public class MinijuegoCerebroData : MonoBehaviour
{
    [Header("Panel")]
    public GameObject panelCerebro;
    [Header("Cerebro")]
    [Tooltip("Image del cerebro central")]
    public Image imagenCerebro;
    [Tooltip("RectTransform del cerebro para detectar colision")]
    public RectTransform rectCerebro;
    [Header("Distractores")]
    [Tooltip("Los 4 distractores, cada uno con su Image y RectTransform")]
    public DistractorCerebro[] distractores;
    [Header("Configuracion cerebro")]
    [Tooltip("Cuanto alpha pierde por segundo")]
    public float perdidaAlpha = 0.08f;
    [Tooltip("Cuanto alpha gana al presionar espacio")]
    public float gananciaAlpha = 0.3f;
    [Header("Configuracion distractores")]
    public float velocidadMin = 60f;
    public float velocidadMax = 160f;
    [Tooltip("Radio de colision con el cerebro en pixeles")]
    public float radioColision = 50f;
    [Header("Configuracion de victoria")]
    [Tooltip("Segundos que debe sobrevivir para ganar")]
    public float tiempoSobrevivir = 15f;
    [HideInInspector] public float tiempoTranscurrido = 0f;
    [HideInInspector] public bool minijuegoActivo = false;
}

```

MinijuegoCantoData.cs

43 lineas

Scripts/Data/Minijuego/Cantar/MinijuegoCantoData.cs

Descripcion	Almacena los pools de notas por carril (0-3), los botones de carril, y configuracion de velocidad, intervalo e intervalos de victoria. El estado (notasAcertadas, fallosActuales, timerNota) es escrito por MinijuegoCantoController. Dependencias: NotaCanto, MinijuegoCantoController.
--------------------	--

```

using UnityEngine;
using UnityEngine.UI;
/// <summary>

```

```

/// Datos del minijuego de canto.
/// Solo datos, sin logica.
/// </summary>
public class MinijuegoCantoData : MonoBehaviour
{
    [Header("Panel")]
    public GameObject panelCanto;
    [Header("Pools de notas por carril")]
    public NotaCanto[] notasCarril0;
    public NotaCanto[] notasCarril1;
    public NotaCanto[] notasCarril2;
    public NotaCanto[] notasCarril3;
    [Header("Botones de carril (en orden 0-3)")]
    public Button[] botones;
    [Tooltip("Margen Y en pixeles mundiales para considerar acierto")]
    public float margenAcierto = 60f;
    [Header("Configuracion de notas")]
    public float velocidadMin = 150f;
    public float velocidadMax = 280f;
    [Tooltip("Pixeles sobre el boton donde aparece la nota")]
    public float posYInicial = 400f;
    [Tooltip("Intervalo en segundos entre aparicion de notas")]
    public float intervaloNotas = 1.2f;
    [Header("Configuracion de victoria")]
    public int notasParaGanar = 8;
    public int fallosPermitidos = 3;
    [HideInInspector] public int notasAcertadas = 0;
    [HideInInspector] public int fallosActuales = 0;
    [HideInInspector] public float timerNota = 0f;
    [HideInInspector] public bool minijuegoActivo = false;
}

```

NotaCanto.cs

78 líneas

Scripts/Data/Minijuego/Cantar/NotaCanto.cs

Descripcion	Representa una nota musical individual del minijuego de canto. Vive en cada GameObject de nota dentro del pool. Activar() la posiciona y la pone en movimiento; Desactivar() la retorna al origen y oculta la imagen. MoverAbajo() la desplaza cada frame segun su velocidad. Dependencias: MinijuegoCantoController.
--------------------	---

```

using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// Datos y estado de una nota musical que cae en el minijuego de canto.
/// MonoBehaviour, vive en cada GameObject de nota.
/// </summary>
public class NotaCanto : MonoBehaviour
{
    [Tooltip("Image de la nota")]
    public Image imagen;
    [HideInInspector] public int carril;
    [HideInInspector] public float velocidad;
    [HideInInspector] public bool activa = false;
    [Tooltip("Posicion inicial de esta nota, asignar manualmente en el Inspector")]
    public Vector2 posicionInicial;
    private RectTransform _rect;
    private void Start()
    {
        _rect = GetComponent<RectTransform>();
    }

    /// <summary>
    /// Activa esta nota en el carril indicado con la velocidad dada,
    /// restaurando su posicion inicial y mostrando su imagen.
    /// </summary>
    /// <param name="carrilAsignado">Indice del carril (0-3) al que pertenece la nota.</param>
    /// <param name="velocidadAsignada">Pixeles por segundo a los que cae la nota.</param>

```

```

public void Activar(int carrilAsignado, float velocidadAsignada)
{
    if (_rect == null)
    _rect = GetComponent<RectTransform>();
    carril = carrilAsignado;
    velocidad = velocidadAsignada;
    activa = true;
    _rect.anchoredPosition = posicionInicial;
    if (imagen != null)
    imagen.gameObject.SetActive(true);
}

/// <summary>
/// Desactiva esta nota: oculta la imagen y restaura la posicion inicial.
/// </summary>
public void Desactivar()
{
    activa = false;
    if (_rect == null)
    _rect = GetComponent<RectTransform>();
    _rect.anchoredPosition = posicionInicial;
    if (imagen != null)
    imagen.gameObject.SetActive(false);
}

/// <summary>Posicion anclada actual del RectTransform de esta nota.</summary>
public Vector2 PosicionActual => GetComponent<RectTransform>().anchoredPosition;
/// <summary>
/// Desplaza la nota hacia abajo en el canvas segun su velocidad y el delta de tiempo.
/// </summary>
/// <param name="delta">Time.deltaTime del frame actual.</param>
public void MoverAbajo(float delta)
{
    if (_rect == null)
    _rect = GetComponent<RectTransform>();
    _rect.anchoredPosition += Vector2.down * velocidad * delta;
}
}

```

MinijuegoSaludoData.cs

37 líneas

Scripts/Data/Minijuego/Mano/MinijuegoSaludoData.cs

Descripcion	Datos del minijuego de saludo. Contiene referencias a los dos botones (arriba/abajo), imagenes de las manos, objeto de exito, barra de tiempo y configuracion de tiempos. El flag botonCorrectoEsArriba se asigna aleatoriamente al iniciar. Dependencias: MinijuegoSaludoController.
--------------------	---

```

using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// Datos del minijuego de saludo de mano.
/// Solo datos, sin logica.
/// </summary>
public class MinijuegoSaludoData : MonoBehaviour
{
    [Header("Panel")]
    public GameObject panelSaludo;
    [Header("Botones")]
    public Button botonArriba;
    public Button botonAbajo;
    [Tooltip("Imagen de la mano del otro personaje (va en el boton correcto aleatorio)")]
    public Image imagenManoOtro;
    [Tooltip("Imagen de tu mano (va en el boton incorrecto)")]
    public Image imagenManoJugador;
    [Header("Imagen de exito")]
    public GameObject imagenExito;
    [Header("Barra de tiempo")]
    [Tooltip("RectTransform de la barra que se acorta con el tiempo")]
}

```

```

public RectTransform barraTiempo;
[Tooltip("Ancho inicial de la barra en pixeles")]
public float anchoBarraInicial = 400f;
[Header("Configuracion")]
public float tiempoLimite = 3f;
public float tiempoExito = 1.2f;
[HideInInspector] public bool botonCorrectoEsArriba;
}

```

MinijuegoSenalData.cs

37 líneas

Scripts/Data/Minijuego/Mano/MinijuegoSenalData.cs

Descripcion	Datos del minijuego de senal de la cruz. Soporta modo simple (4 botones) y modo complejo (mas pasos) controlado por el flag esCompleja. Guarda el arreglo de botones para cada modo y el indice actual en la secuencia. Dependencias: MinijuegoSenalController, ConfigSenal.
--------------------	--

```

using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// Datos del minijuego de senal.
/// Soporta modo simple (4 botones) y modo complejo (mas botones).
/// Solo datos, sin logica.
/// </summary>
public class MinijuegoSenalData : MonoBehaviour
{
    [Header("Panel")]
    public GameObject panelSenal;
    [Header("Barra de tiempo")]
    public RectTransform barraTiempo;
    public float anchoBarraInicial = 400f;
    [Header("Modo Simple")]
    [Tooltip("Los 4 botones del modo simple")]
    public Button[] botones;
    [Tooltip("GameObjects del modo simple, se desactivan en modo complejo")]
    public GameObject[] objetosSimple;
    [Header("Modo Complejo")]
    [Tooltip("Si true usa los botones complejos en lugar de los simples")]
    public bool esCompleja = false;
    [Tooltip("Los botones del modo complejo (pueden ser mas de 4)")]
    public Button[] botonesComplejos;
    [Tooltip("GameObjects del modo complejo, se activan solo en modo complejo")]
    public GameObject[] objetosComplejos;
    [Header("Configuracion")]
    public float tiempoLimite = 5f;
    [HideInInspector] public int indiceActual = 0;
    [HideInInspector] public int[] ordenAleatorio = new int[4];
}

```

PanelManoData.cs

21 líneas

Scripts/Data/Minijuego/Mano/PanelManoData.cs

Descripcion	Datos del panel intermedio de la mano derecha. El jugador ve dos botones: Saludo o Senal. Al pulsar, PanelManoController dispara el evento correspondiente que MinijuegoManager escucha para lanzar el minijuego correcto. Dependencias: PanelManoController.
--------------------	---

```

using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// Datos del panel intermedio de la mano.
/// El jugador elige entre Saludo o Senal.
/// Solo datos, sin logica.
/// </summary>
public class PanelManoData : MonoBehaviour
{
    [Header("Panel")]

```

```
[Tooltip("Panel raiz, se activa al seleccionar ManoDerecha")]
public GameObject panelMano;
[Header("Botones")]
[Tooltip("Boton para lanzar el minijuego de saludo")]
public Button botonSaludo;
[Tooltip("Boton para lanzar el minijuego de senal")]
public Button botonSenal;
}
```

CeldaCuadrículaData.cs

51 líneas

Scripts/Data/Minijuego/Pierna/CeldaCuadrículaData.cs

Descripcion	Representa una celda de la cuadrícula 3×3 del minijuego de pierna. Registra su posición (columna, fila), la parte ocupante, el botón UI y la imagen que muestra. Limpiar() resetea la celda al estado vacío. EstaOcupada es una propiedad calculada. Dependencias: MinijuegoPiernaController.
-------------	---

```
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// Datos de una celda de la cuadrícula del minijuego de pierna.
/// MonoBehaviour, vive en cada uno de los 9 GameObjects de celda.
/// Sin lógica, solo estado y referencias.
/// </summary>
public class CeldaCuadrículaData : MonoBehaviour
{
    [Header("Posición en la grilla (0-2)")]
    [Tooltip("Columna de esta celda en la cuadrícula")]
    public int columna;
    [Tooltip("Fila de esta celda en la cuadrícula")]
    public int fila;
    [Header("Estado")]
    [Tooltip("Parte de la pierna colocada en esta celda. Null si esta vacía.")]
    public PartePierna? parteOcupante = null;
    [Header("Referencias UI")]
    [Tooltip("Botón de esta celda para recibir clicks")]
    public Button boton;
    [Tooltip("Imagen que muestra la parte colocada en esta celda")]
    public Image imagenParte;
    [Tooltip("Sprite por defecto de la celda cuando esta vacía")]
    public Sprite spriteDefault;
    #region API Publica
    /// <summary>Devuelve true si hay alguna parte de pierna colocada en esta celda.</summary>
    public bool EstaOcupada => parteOcupante.HasValue;

    /// <summary>
    /// Vacía la celda: elimina la parte ocupante y restaura el sprite por defecto.
    /// </summary>
    public void Limpiar()
    {
        parteOcupante = null;
        if (imagenParte != null)
        {
            imagenParte.sprite = spriteDefault;
            imagenParte.gameObject.SetActive(false);
        }
    }
    #endregion
}
```

MinijuegoPiernaData.cs

86 líneas

Scripts/Data/Minijuego/Pierna/MinijuegoPiernaData.cs

Descripcion	Datos del minijuego de ensamblaje de pierna. Referencia las 9 celdas, las imagenes del panel de orden (cadera→rodilla→pie), los sprites de cada parte y lineas de conexion visual. ObtenerSprite() y ObtenerImagenOrden() proveen acceso tipado. El orden aleatorio se genera al iniciar. Dependencias: CeldaCuadrículaData, MinijuegoPiernaController.
--------------------	---

```

using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// Datos del minijuego de pierna.
/// Sin logica, solo referencias y estado.
/// </summary>
public class MinijuegoPiernaData : MonoBehaviour
{
    [Header("Panel raiz")]
    [Tooltip("Panel completo del minijuego, se activa al iniciar")]
    public GameObject panelMinijuego;
    [Header("Cuadrícula")]
    [Tooltip("Las 9 celdas de la cuadrícula, asignar en orden de izquierda a derecha, arriba a abajo")]
    public CeldaCuadrículaData[] celdas;
    [Header("Imagenes de orden (panel lateral)")]
    [Tooltip("Imagen del slot 1 del panel de orden")]
    public Image imagenOrden1;
    [Tooltip("Imagen del slot 2 del panel de orden")]
    public Image imagenOrden2;
    [Tooltip("Imagen del slot 3 del panel de orden")]
    public Image imagenOrden3;
    [Header("Sprites de cada parte")]
    [Tooltip("Sprite visual de la cadera")]
    public Sprite spriteCadera;
    [Tooltip("Sprite visual de la rodilla")]
    public Sprite spriteRodilla;
    [Tooltip("Sprite visual del pie")]
    public Sprite spritePie;
    [Header("Conexiones")]
    [Tooltip("Imagen de linea entre cadera y rodilla")]
    public Image lineaCaderaRodilla;
    [Tooltip("Imagen de linea entre rodilla y pie")]
    public Image lineaRodillaPie;
    [Header("Configuración")]
    [Tooltip("Segundos de espera antes de reportar el resultado")]
    public float tiempoEspera = 1.2f;
    [Header("Estado (solo lectura)")]
    [Tooltip("Índice de la parte que toca colocar ahora (0=primera del orden)")]
    public int turnoActual = 0;
    [Tooltip("Orden aleatorio generado al iniciar")]
    public PartePierna[] ordenAleatorio = new PartePierna[3];
    #region Helpers
    /// <summary>
    /// Devuelve el sprite correspondiente a una parte.
    /// </summary>
    public Sprite ObtenerSprite(PartePierna parte)
    {
        switch (parte)
        {
            case PartePierna.Cadera: return spriteCadera;
            case PartePierna.Rodilla: return spriteRodilla;
            case PartePierna.Pie: return spritePie;
            default:
                Debug.LogWarning($"[MinijuegoPiernaData] Parte no reconocida: {parte}");
                return null;
        }
    }
    /// <summary>
    /// Devuelve la imagen del slot de orden segun indice (0,1,2).
    /// </summary>
    public Image ObtenerImagenOrden(int indice)

```

```

{
switch (indice)
{
case 0: return imagenOrden1;
case 1: return imagenOrden2;
case 2: return imagenOrden3;
default:
Debug.LogWarning($"[MinijuegoPiernaData] Indice de orden fuera de rango: {indice}");
return null;
}
}
#endregion
}

```

MinijuegoResponderData.cs

59 líneas

Scripts/Data/Minijuego/boca/MinijuegoResponderData.cs

Descripcion	Datos del minijuego de preguntas. Contiene la imagen de pregunta, barra de tiempo, tres botones con textos configurables, sistema de ayuda con alpha progresivo y el indice de la respuesta correcta. ConfigResponder escribe el contenido de la pregunta antes de iniciar. Dependencias: MinijuegoResponderController, ConfigResponder.
--------------------	--

```

using UnityEngine;
using UnityEngine.UI;
using TMPro;
/// <summary>
/// Datos del minijuego de responder.
/// Solo datos, sin logica.
/// </summary>
public class MinijuegoResponderData : MonoBehaviour
{
[Header("Panel")]
public GameObject panelResponder;
[Header("Pregunta")]
[Tooltip("Imagen superior con la pregunta o contexto visual")]
public Image imagenPregunta;
[Header("Barra de tiempo")]
[Tooltip("RectTransform de la barra que se acorta con el tiempo")]
public RectTransform barraTiempo;
[Tooltip("Ancho inicial de la barra en pixeles")]
public float anchoBarraInicial = 600f;
[Header("Botones de respuesta")]
public Button boton0;
public Button boton1;
public Button boton2;
public TextMeshProUGUI textoBoton0;
public TextMeshProUGUI textoBoton1;
public TextMeshProUGUI textoBoton2;
[Header("Ayuda")]
[Tooltip("Texto con la respuesta correcta, tapado por imagenAyuda")]
public TextMeshProUGUI textoAyuda;
[Tooltip("Imagen que tapa la respuesta, se transparenta con espacio")]
public Image imagenAyuda;
[Tooltip("Cuanto alpha pierde la imagen de ayuda por cada espacio")]
public float alphaAyudaPorEspacio = 0.15f;
[Tooltip("Cuanto se acelera la barra por cada espacio usado")]
public float aceleracionBarraPorEspacio = 1.5f;
[Header("Configuracion")]
[Tooltip("Tiempo total para responder en segundos")]
public float tiempoLimite = 10f;
[Header("Contenido")]
[Tooltip("Texto de los 3 botones")]
public string[] textoBotones = new string[3] { "Opcion A", "Opcion B", "Opcion C" };
[Tooltip("Indice del boton correcto: 0, 1 o 2. Menor a 0 o mayor a 2 = ninguno correcto")]
public int indiceRespuestaCorrecta = 0;
[Tooltip("Texto que revela la ayuda")]

```



```

public string textoAyudaContenido = "La respuesta es...";
[HideInInspector] public float tiempoRestante;
[HideInInspector] public float multiplicadorVelocidad = 1f;
[HideInInspector] public bool esperandoRespuesta = false;
}

```

PanelBocaData.cs

17 líneas

Scripts/Data/Minijuego/boca/PanelBocaData.cs

Descripcion	Datos del panel intermedio de la boca. Ofrece al jugador elegir entre Responder (preguntas) o Cantar (notas musicales). PanelBocaController gestiona los eventos de seleccion. Dependencias: PanelBocaController.
--------------------	---

```

using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// Datos del panel intermedio de la boca.
/// El jugador elige entre Responder o Cantar.
/// Solo datos, sin logica.
/// </summary>
public class PanelBocaData : MonoBehaviour
{
    [Header("Panel")]
    public GameObject panelBoca;
    [Header("Botones")]
    public Button botonResponder;
    public Button botonCantar;
}

```

MinijuegoPechoData.cs

46 líneas

Scripts/Data/Minijuego/pecho/MinijuegoPechoData.cs

Descripcion	Datos del minijuego de ritmo del pecho. El jugador debe presionar el boton del corazon cuando el circulo externo decreciente coincida con el corazon fijo. Configura rondas totales, tamanos y velocidades. El estado de ronda y tamaño actual es escrito por MinijuegoPechoController. Dependencias: MinijuegoPechoController, ConfigPecho.
--------------------	--

```

using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// Datos del minijuego de ritmo del pecho.
/// Solo datos, sin logica.
/// </summary>
public class MinijuegoPechoData : MonoBehaviour
{
    [Header("Panel")]
    public GameObject panelPecho;
    [Header("Visual")]
    [Tooltip("RectTransform del circulo externo que se achica")]
    public RectTransform circuloExterno;
    [Tooltip("RectTransform del corazon fijo en el centro")]
    public RectTransform corazon;
    [Tooltip("Boton del corazon que el jugador presiona")]
    public Button botonCorazon;
    [Header("Configuracion de rondas")]
    [Tooltip("Cuantas veces debe acertar para ganar")]
    public int rondasTotales = 4;
    [Tooltip("Tamaño inicial del circulo externo")]
    public float tamañoInicial = 400f;
    [Tooltip("Tamaño del corazon fijo (referencia para comparar)")]
    public float tamañoCorazon = 100f;
    [Tooltip("Margen de tolerancia para considerar acierto")]
    public float margenAcierto = 30f;
    [Tooltip("Velocidad minima de achicamiento")]
}

```

```

public float velocidadMin = 80f;
[Tooltip("Velocidad maxima de achicamiento")]
public float velocidadMax = 180f;
[HideInInspector] public int rondaActual = 0;
[HideInInspector] public float tamanoActual = 0f;
[HideInInspector] public float velocidadActual = 0f;
[HideInInspector] public bool esperandoClick = false;
}

```

ParteCuerpoData.cs

19 lineas

Scripts/Data/Partes del cuerpo/ParteCuerpoData.cs

Descripcion	Asocia un valor del enum ParteCuerpo con su boton UI e imagen. CuerpoUIController itera el arreglo de ParteCuerpoData para registrar listeners y aplicar colores segun el estado de JugadorHabilidadesData. Dependencias: CuerpoUIController, ParteCuerpo.
--------------------	--

```

using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// Datos de una parte del cuerpo vinculada a un boton de la UI.
/// MonoBehaviour para poder asignarse como componente en el Inspector.
/// Solo datos, sin logica.
/// </summary>
public class ParteCuerpoData : MonoBehaviour
{
[Tooltip("Que parte del cuerpo representa este componente")]
public ParteCuerpo parte;
[Tooltip("Boton de la UI asociado a esta parte")]
public Button boton;
[Tooltip("Imagen del boton para feedback visual de bloqueado/desbloqueado")]
public Image imagenBoton;
}

```

TransicionZonaData.cs

47 lineas

Scripts/Data/transicion/TransicionZonaData.cs

Descripcion	Datos de la transicion de zona con cuadro negro. Define el cuadro de fade, velocidad de transicion, zonas a activar/desactivar, puntos de destino para jugador y camara, y audio de transicion. El flag SeguirJugadorSiguiente indica si la camara debe retomar el follow tras la transicion. Dependencias: TransicionZonaController.
--------------------	---

```

using UnityEngine;
/// <summary>
/// Datos de la transicion de zona.
/// Sin logica, solo referencias y configuracion.
/// </summary>
public class TransicionZonaData : MonoBehaviour
{
[Header("Cuadro negro")]
[Tooltip("Transform del cuadro negro hijo de la camara")]
public Transform cuadroNegro;
[Tooltip("Velocidad a la que sube y baja el cuadro negro")]
public float velocidadTransicion = 2f;
[Header("Zonas")]
[Tooltip("GameObject de la nueva zona a activar")]
public GameObject zonaEntrada;
[Tooltip("GameObject de la zona anterior a desactivar")]
public GameObject zonaSalida;
[Header("Puntos de destino")]
[Tooltip("Punto donde se teletransporta el jugador")]
public Transform puntoJugador;
[Tooltip("Punto donde se teletransporta la camara")]
public Transform puntoCamara;
[Header("Referencias")]

```

```
[Tooltip("Transform de la camara principal")]
public Transform camara;

[Header("Audio")]
[Tooltip("AudioSource que reproduce el sonido de transicion")]
public AudioSource audioTransicion;
[Tooltip("Nuevo AudioClip de caminata para la zona destino")]
public AudioClip nuevaCaminata;

[Header("Seguir Al Jugador Luego")]
public bool SeguirJugadorSiguiente;
}
```

Enumeraciones

Enumeraciones de C# que definen conjuntos de valores con nombre. Facilitan la lectura del código y evitan el uso de enteros mágicos al representar partes del cuerpo, tipos de acción, etc.

Archivo	Líneas	Ruta relativa
TipoAccion.cs	12	Enums/Acciones Mentales/TipoAccion.cs
PartePierna.cs	9	Enums/Minijuego/Pierna/PartePierna.cs
ParteCuerpo.cs	12	Enums/Partes del cuerpo/ParteCuerpo.cs

TipoAccion.cs

12 líneas

Scripts/Enums/Acciones Mentales/TipoAccion.cs

Descripción	Enumeración de los cinco tipos de acción que puede devolver un minijuego: Postura, Mano, Pecho, Cabeza, Boca. MisionBase usa TipoAccion junto con ValorEsperado para evaluar si el resultado reportado coincide con la misión activa. Extender este enum al agregar nuevos minijuegos.
--------------------	--

```
/// <summary>
/// Tipos de accion que un minijuego puede devolver.
/// Agregar nuevos tipos aqui segun se expanda el juego.
/// </summary>
public enum TipoAccion
{
    Postura,
    Mano,
    Pecho,
    Cabeza,
    Boca
}
```

PartePierna.cs

9 líneas

Scripts/Enums/Minijuego/Pierna/PartePierna.cs

Descripción	Define las tres partes ensamblables de la pierna: Cadera, Rodilla, Pie. El minijuego de pierna genera un orden aleatorio con estos valores y CeldaCuadrículaData los usa como identificador de que parte ocupa cada celda.
--------------------	--

```
/// <summary>
/// Partes que componen la pierna en el minijuego.
/// </summary>
public enum PartePierna
{
    Cadera,
    Rodilla,
    Pie
}
```

ParteCuerpo.cs

12 líneas

Scripts/Enums/Partes del cuerpo/ParteCuerpo.cs

Descripción	Enumeración de las partes del cuerpo disponibles: Piernas, Pecho, Cabeza, Boca, ManoDerecha. CuerpoUIController dispara OnParteSeleccionada con este valor, que MinijuegoManager interpreta para lanzar el minijuego correcto.
--------------------	--

```

/// <summary>
/// Partes del cuerpo disponibles en el juego.
/// Agregar nuevas partes aquí según se expanda el juego.
/// </summary>
public enum ParteCuerpo
{
    Piernas,
    Pecho,
    Cabeza,
    Boca,
    ManoDerecha
}

```

Estructuras

Structs de C# que agrupan datos relacionados de forma ligera y eficiente. Se usan para representar resultados de acciones y líneas de diálogo sin la sobrecarga de un MonoBehaviour.

Archivo	Líneas	Ruta relativa
AccionResultado.cs	22	<i>Structuras/Acciones Mentales/AccionResultado.cs</i>
DialogoLinea.cs	19	<i>Structuras/Dialogo/DialogoLinea.cs</i>

AccionResultado.cs

22 líneas

Scripts/Structuras/Acciones Mentales/AccionResultado.cs

Descripción	Struct ligero que encapsula el resultado de un minijuego: un TipoAccion y un entero valor. MisionesGlobal.ReportarResultado() recibe este struct y lo compara con la misión activa. Al ser struct evita allocations en el heap y se pasa por valor. Dependencias: TipoAccion.
--------------------	---

```

/// <summary>
/// Resultado que devuelve cualquier minijuego al terminar.
/// Solo datos, sin lógica.
/// </summary>
public struct AccionResultado
{
    /// <summary>Tipo de acción que reporta el minijuego (Postura, Mano, Pecho, Cabeza, Boca).</summary>
    public TipoAccion tipo;
    /// <summary>Código de resultado devuelto por el minijuego; cada uno define sus propios valores.</summary>
    public int valor;

    /// <summary>
    /// Crea un nuevo resultado con el tipo y valor indicados.
    /// </summary>
    /// <param name="tipo">Tipo de acción del minijuego que terminó.</param>
    /// <param name="valor">Código de resultado (p. ej. 1 = éxito, 0 = fallo).</param>
    public AccionResultado(TipoAccion tipo, int valor)
    {
        this.tipo = tipo;
        this.valor = valor;
    }
}

```

DialogoLinea.cs

19 líneas

Scripts/Structuras/Dialogo/DialogoLinea.cs

Descripción	Struct serializable que representa una línea individual de diálogo: el texto, el sprite del personaje izquierdo y el sprite del personaje derecho. DialogoData contiene un arreglo de DialogoLinea; DialogoController los itera para actualizar la UI. Dependencias: DialogoData, DialogoController.
--------------------	--

```
using UnityEngine;
/// <summary>
/// Datos de una línea de diálogo.
/// Struct puro, sin lógica.
/// </summary>
[System.Serializable]
public struct DialogoLinea
{
    [Tooltip("Texto de esta línea de diálogo")]
    [TextArea]
    public string texto;
    [Tooltip("Imagen del personaje que habla")]
    public Sprite imagen;
    [Tooltip("Si es true la imagen aparece a la izquierda, si es false a la derecha")]
    public bool esIzquierda;
}
```

Interfaces

Contratos de interfaz que definen el comportamiento mínimo que deben implementar ciertas clases. Permiten el polimorfismo y la extensión del sistema de misiones sin romper el código existente.

Archivo	Líneas	Ruta relativa
MisionBase.cs	47	Interfaces/Mision/MisionBase.cs

MisionBase.cs

47 líneas

Scripts/Interfaces/Mision/MisionBase.cs

Descripción	Clase base abstracta para todas las misiones del juego. No es MonoBehaviour, se instancia como objeto puro y se pasa a MisionesGlobal. Define TipoEsperado y ValorEsperado, el método EvaluarResultado() que compara el AccionResultado recibido, y el método abstracto Actuar() que cada misión concreta implementa. Dependencias: TipoAccion, AccionResultado.
--------------------	--

```
/// <summary>
/// Clase base para cualquier misión del juego.
/// Cada misión concreta hereda de aquí, declara que resultado espera
/// e implementa su propia lógica en Actuar().
/// No es MonoBehaviour: se instancia y se pasa como objeto.
/// </summary>
public abstract class MisionBase
{
    /// <summary>Tipo de acción que debe devolver el minijuego para que esta misión se cumpla.</summary>
    public TipoAccion TipoEsperado { get; protected set; }
    /// <summary>Valor entero esperado del resultado; cada minijuego define su propio código de éxito.</summary>
    public int ValorEsperado { get; protected set; }
    #region Constructor
    /// <summary>
    /// Inicializa la misión con el tipo de acción y el valor de éxito esperados.
    /// </summary>
    /// <param name="tipoEsperado">Tipo de acción que debe devolver el minijuego.</param>
    /// <param name="valorEsperado">Valor de éxito definido por el minijuego concreto.</param>
    protected MisionBase(TipoAccion tipoEsperado, int valorEsperado)
    {
```

```

    TipoEsperado = tipoEsperado;
    ValorEsperado = valorEsperado;
}
#endregion
#region API Publica
    /// <summary>
    /// Evalua si el resultado dado cumple esta mision.
    /// </summary>
    public bool EvaluarResultado(AccionResultado resultado)
    {
        return resultado.tipo == TipoEsperado && resultado.valor == ValorEsperado;
    }

    /// <summary>
    /// Logica que se ejecuta cuando la mision se cumple.
    /// Cada mision concreta define que hace aqui:
    /// abrir puerta, avanzar dialogo, disparar evento, etc.
    /// </summary>
    public abstract void Actuar();
#endregion
}

```

Managers

Orquestadores centrales del juego. Escuchan eventos del sistema y delegan la logica al componente apropiado. El MinijuegoManager coordina todos los minijuegos; el MisionesGlobal gestiona el progreso de misiones.

Archivo	Lineas	Ruta relativa
MinijuegoManager.cs	125	Managers/Minijuegos/MinijuegoManager.cs
MisionesGlobal.cs	114	Managers/Misiones/MisionesGlobal.cs

MinijuegoManager.cs

125 lineas

Scripts/Managers/Minijuegos/MinijuegoManager.cs

Descripcion	Orquestador central de minijuegos. Se suscribe a CuerpoUIController.OnParteSeleccionada y a los eventos de sub-seleccion de PanelManoController y PanelBocaController. Segun la parte elegida, llama a Iniciar() en el controlador de minijuego correspondiente. Mantiene bajo acoplamiento: solo conoce las referencias, no la logica interna. Dependencias: todos los controladores de minijuego, CuerpoUIController.
--------------------	---

```

using UnityEngine;

/// <summary>
/// Escucha el evento de seleccion de parte del cuerpo
/// y dispara el minijuego o panel correspondiente.
/// </summary>
public class MinijuegoManager : MonoBehaviour
{
    [Header("Minijuegos")]
    public MinijuegoPiernaController minijuegoPierna;
    public MinijuegoPechoController minijuegoPecho;
    public MinijuegoCerebroController minijuegoCerebro;

    [Header("Mano")]
    public PanelManoController panelMano;
    public MinijuegoSaludoController minijuegoSaludo;
    public MinijuegoSenalController minijuegoSenal;

    [Header("Boca")]
    public PanelBocaController panelBoca;
    public MinijuegoResponderController minijuegoResponder;
    public MinijuegoCantoController minijuegoCanto;

    #region Unity Callbacks
    private void OnEnable()
    {

```

```

{
    CuerpoUIController.OnParteSeleccionada += AlSeleccionarParte;
    PanelManoController.OnSaludoSeleccionado += AlSeleccionarSaludo;
    PanelManoController.OnSenalSeleccionada += AlSeleccionarSenal;
    PanelBocaController.OnResponderSeleccionado += AlSeleccionarResponder;
    PanelBocaController.OnCantarSeleccionado += AlSeleccionarCantar;
}

private void OnDisable()
{
    CuerpoUIController.OnParteSeleccionada -= AlSeleccionarParte;
    PanelManoController.OnSaludoSeleccionado -= AlSeleccionarSaludo;
    PanelManoController.OnSenalSeleccionada -= AlSeleccionarSenal;
    PanelBocaController.OnResponderSeleccionado -= AlSeleccionarResponder;
    PanelBocaController.OnCantarSeleccionado -= AlSeleccionarCantar;
}

#endregion
#region Logica
    /// <summary>
    /// Recibe la parte del cuerpo seleccionada por el jugador y lanza el minijuego o panel correspondiente.
    /// </summary>
    /// <param name="parte">Parte del cuerpo que el jugador selecciono en la UI.</param>
    private void AlSeleccionarParte(ParteCuerpo parte)
    {
        switch (parte)
        {
            case ParteCuerpo.Piernas:
                if (minijuegoPierna == null) { Advertir(parte); return; }
                minijuegoPierna.Iniciar();
                break;
            case ParteCuerpo.Pecho:
                if (minijuegoPecho == null) { Advertir(parte); return; }
                minijuegoPecho.Iniciar();
                break;
            case ParteCuerpo.Cabeza:
                if (minijuegoCerebro == null) { Advertir(parte); return; }
                minijuegoCerebro.Iniciar();
                break;
            case ParteCuerpo.ManoDerecha:
                if (panelMano == null) { Advertir(parte); return; }
                panelMano.AbrirPanel();
                break;
            case ParteCuerpo.Boca:
                if (panelBoca == null) { Advertir(parte); return; }
                panelBoca.AbrirPanel();
                break;
            default:
                Debug.LogWarning($"[MinijuegoManager] No hay minijuego implementado para: {parte}");
                break;
        }
    }

    /// <summary>Inicia el minijuego de saludo de mano cuando el jugador elige esa opcion en el
    panel.</summary>
    private void AlSeleccionarSaludo()
    {
        if (minijuegoSaludo == null) { Debug.LogWarning("[MinijuegoManager] minijuegoSaludo no asignado.");
        return; ...
        minijuegoSaludo.Iniciar();
    }

    /// <summary>Inicia el minijuego de senal cuando el jugador elige esa opcion en el panel de
    mano.</summary>
    private void AlSeleccionarSenal()
    {
        if (minijuegoSenal == null) { Debug.LogWarning("[MinijuegoManager] minijuegoSenal no asignado."); return; }
        minijuegoSenal.Iniciar();
    }

    /// <summary>Inicia el minijuego de responder preguntas cuando el jugador lo elige en el panel de
    boca.</summar ...

```

```

private void AlSeleccionarResponder()
{
    if (minijuegoResponder == null) { Debug.LogWarning("[MinijuegoManager] minijuegoResponder no asignado.");
    r ...
    minijuegoResponder.Iniciar();
}

/// <summary>Inicia el minijuego de canto cuando el jugador lo elige en el panel de boca.</summary>
private void AlSeleccionarCantar()
{
    if (minijuegoCanto == null) { Debug.LogWarning("[MinijuegoManager] minijuegoCanto no asignado."); return; }
    minijuegoCanto.Iniciar();
}

/// <summary>
/// Emite un warning en consola indicando que el minijuego de la parte indicada no esta asignado.
/// </summary>
/// <param name="parte">Parte del cuerpo cuyo minijuego falta en el Inspector.</param>
private void Advertir(ParteCuerpo parte)
{
    Debug.LogWarning($"[MinijuegoManager] El minijuego de {parte} no esta asignado en el Inspector.");
}

#endregion
}

```

MisionesGlobal.cs

114 líneas

Scripts/Managers/Misiones/MisionesGlobal.cs

Descripcion	Singleton persistente (DontDestroyOnLoad) que mantiene una mision activa a la vez. AsignarMision() registra la nueva mision; ReportarResultado() la evalua y, si acierta, llama Actuar(); si falla, llama ActuarFallo() en MisionPosicion. CancelarMision() limpia sin ejecutar. Previene duplicados de singleton. Dependencias: MisionBase, AccionResultado, MisionPosicion.
--------------------	---

```

using UnityEngine;

/// <summary>
/// Manager global de misiones. Gestiona una sola mision activa a la vez.
/// Recibe el resultado del minijuego, evalua la mision activa y la limpia al cumplirse.
/// Protegido contra referencias nulas en todos los puntos de entrada.
/// </summary>
public class MisionesGlobal : MonoBehaviour
{
    public static MisionesGlobal Instancia { get; private set; }
    private MisionBase _misionActiva;

    /// <summary>True si hay una mision pendiente de resolver en este momento.</summary>
    public bool TieneMisionActiva => _misionActiva != null;

    #region Unity Callbacks
    private void Awake()
    {
        ConfigurarSingleton();
    }

    #endregion

    #region Inicializacion
    /// <summary>
    /// Aplica el patron Singleton: si ya existe una instancia destruye este duplicado,
    /// de lo contrario se registra como instancia global y persiste entre escenas.
    /// </summary>
    private void ConfigurarSingleton()
    {
        if (Instancia != null && Instancia != this)
        {
            Debug.LogWarning("[MisionesGlobal] Ya existe una instancia. Destruyendo duplicado.");
            Destroy(gameObject);
            return;
        }

        Instancia = this;
        DontDestroyOnLoad(gameObject);
    }

    #endregion
}

```



```

#region API Publica
    /// <summary>
    /// Asigna una nueva mision activa.
    /// </summary>
    public void AsignarMision(MisionBase nuevaMision)
    {
        if (nuevaMision == null)
        {
            Debug.LogWarning("[MisionesGlobal] Se intento asignar una mision nula. Ignorado.");
            return;
        }
        if (_misionActiva != null)
            Debug.LogWarning("[MisionesGlobal] Se reemplazo una mision activa antes de que se completara.");
        _misionActiva = nuevaMision;
    }

    /// <summary>
    /// Llamado por el minijuego al terminar.
    /// Si coincide llama Actuar(), si no llama ActuarFallo() si la mision lo soporta.
    /// En ambos casos limpia la mision activa.
    /// </summary>
    public void ReportarResultado(AccionResultado resultado)
    {
        if (_misionActiva == null)
        {
            Debug.Log("[MisionesGlobal] Resultado reportado sin mision activa. Ignorado.");
            return;
        }
        MisionBase misionResuelta = _misionActiva;
        _misionActiva = null;
        if (misionResuelta.EvaluarResultado(resultado))
        {
            misionResuelta.Actuar();
        }
        else
        {
            Debug.Log($"[MisionesGlobal] Fallo. Esperaba tipo={misionResuelta.TipoEsperado} " +
                $"valor={misionResuelta.ValorEsperado}. " +
                $"Recibido tipo={resultado.tipo} valor={resultado.valor}.");
            // Si la mision soporta fallo lo notifica
            if (misionResuelta is MisionPosicion misionPosicion)
                misionPosicion.ActuarFallo();
        }
    }

    /// <summary>
    /// Cancela la mision activa sin ejecutarla.
    /// </summary>
    public void CancelarMision()
    {
        if (_misionActiva == null)
        {
            Debug.Log("[MisionesGlobal] No hay mision activa que cancelar.");
            return;
        }
        Debug.Log("[MisionesGlobal] Mision cancelada.");
        _misionActiva = null;
    }
#endregion
}

```

Misa

Scripts específicos de la escena de Misa. Controlan el flujo narrativo de la ceremonia, la barra de vida/energía del jugador y las transiciones entre etapas de la escena.

Archivo	Lineas	Ruta relativa
EscenaMisa.cs	44	Misa/EscenaMisa.cs
GestorMisa.cs	349	Misa/GestorMisa.cs
VidaUI.cs	42	Misa/VidaUI.cs

EscenaMisa.cs

44 líneas

Scripts/Misa/EscenaMisa.cs

Descripcion	Clase serializable que describe una escena individual dentro de la secuencia de misa: nombre, si muestra titulo, dialogos de inicio/exito/fallo, tipo de accion y valor esperado para la mision, musica opcional, animacion del sacerdote y su punto de destino, objetos a activar/desactivar, y visuales del jugador. GestorMisa itera un arreglo de EscenaMisa configurado en el Inspector. Dependencias: GestorMisa, DialogoData.
--------------------	--

```

using UnityEngine;

/// <summary>
/// Datos de una escena individual de la misa.
/// Serializable para configurar inline en el Inspector del GestorMisa.
/// </summary>
[System.Serializable]
public class EscenaMisa
{
    [Header("Identificacion")]
    public string nombreEscena;
    [Tooltip("Si es true muestra el ttulo grande antes del dialogo inicial")]
    public bool mostrarTitulo;
    public string textoTitulo;
    [Header("Dialogos")]
    public DialogoData dialogoInicial;
    public DialogoData dialogoExito;
    public DialogoData dialogoFallo;
    [Header("Mision")]
    public TipoAccion tipoAccion;
    public int valorEsperado;
    [Header("Msica")]
    [Tooltip("Si no es null cambia la msica al iniciar esta escena")]
    public AudioClip musicaOpcional;
    [Header("Sacerdote")]
    public string animacionSacerdote;
    [Tooltip("Si no es null mueve al sacerdote a este punto")]
    public Transform puntoDestino;
    public float velocidadSacerdote = 2f;
    [Header("Objetos")]
    [Tooltip("Se activan al iniciar la escena")]
    public GameObject[] activar;
    [Tooltip("Se desactivan al iniciar la escena")]
    public GameObject[] desactivar;
    [Header("Sprites del jugador en escena")]
    public GameObject[] visualesJugador;
}

```

GestorMisa.cs

349 líneas

Scripts/Misa/GestorMisa.cs

Descripcion	Controlador principal de la ceremonia de misa. Al entrar el jugador al area trigger y presionar espacio, inicia la coroutine IniciarFlujo que ejecuta cada EscenaMisa en secuencia. Por cada escena: configura objetos, mueve al sacerdote, muestra titulo, reproduce dialogo inicial, espera resultado de mision, y segun exito/fallo avanza o reinicia. Al completar todas las escenas activa el objeto de comunion (ganar). Dependencias: EscenaMisa, DialogoController, MisionesGlobal, MisionPosicion, VidaUI.
--------------------	---

```

using UnityEngine;

```

```

using UnityEngine.UI;
using TMPPro;
using System.Collections;

/// <summary>
/// Gestiona el flujo completo de la misa.
/// Trigger → space → secuencia de escenas → vidas → ganar/perder.
/// </summary>
public class GestorMisa : MonoBehaviour
{
    [Header("Referencias")]
    public Transform cuadroNegro;
    public float velocidadCuadro = 3f;
    public AudioSource audioSource;
    public Animator animadorSacerdote;
    public Transform sacerdote;

    [Header("Jugador")]
    public JugadorData jugadorData;
    public GameObject visualJugadorReal;
    public Transform puntoResetJugador;
    public Transform transformJugador;

    [Header("UI")]
    public VidaUI vidaUI;
    public GameObject panelTitulo;
    public TextMeshProUGUI textoTitulo;
    public GameObject pensamientoJugador;

    [Header("Indicador space")]
    public GameObject indicadorSpace;

    [Header("Al ganar")]
    [Tooltip("GameObject del padre con la comunion, se activa al ganar")]
    public GameObject objetoGanar;

    [Header("Escenas")]
    public EscenaMisa[] escenas;
    private int _escenaActual = 0;
    private bool _jugadorDentro = false;
    private bool _enFlujo = false;
    private DialogoController _dialogoController;

    #region Unity Callbacks
    private void Awake()
    {
        _dialogoController = FindFirstObjectByType<DialogoController>();
    }

    private void Start()
    {
        if (indicadorSpace != null) indicadorSpace.SetActive(false);
        if (panelTitulo != null) panelTitulo.SetActive(false);
        if (objetoGanar != null) objetoGanar.SetActive(false);
        if (vidaUI != null) vidaUI.gameObject.SetActive(false);
    }

    private void Update()
    {
        if (!_jugadorDentro || !_enFlujo) return;
        if (Input.GetKeyDown(KeyCode.Space))
            StartCoroutine(IniciarFlujo());
    }

    private void OnTriggerEnter(Collider other)
    {
        if (!other.CompareTag("Player")) return;
        _jugadorDentro = true;
        if (indicadorSpace != null) indicadorSpace.SetActive(true);
        if (vidaUI != null) vidaUI.gameObject.SetActive(true);
    }

    private void OnTriggerExit(Collider other)
    {
        if (!other.CompareTag("Player")) return;
        _jugadorDentro = false;
        if (indicadorSpace != null) indicadorSpace.SetActive(false);
        if (vidaUI != null) vidaUI.gameObject.SetActive(false);
    }

```

```

}
#endregion
#region Flujo Principal
  /// <summary>
  /// Arranca el flujo completo de la misa: bloquea movimiento, oculta al jugador
  /// y ejecuta la primera escena.
  /// </summary>
  private IEnumerator IniciarFlujo()
  {
    _enFlujo = true;
    if (indicadorSpace != null) indicadorSpace.SetActive(false);
    if (visualJugadorReal != null) visualJugadorReal.SetActive(false);
    if (jugadorData != null) jugadorData.puedeMoverse = false;
    yield return StartCoroutine(EjecutarEscena(_escenaActual));
  }

  /// <summary>
  /// Ejecuta la escena de la misa en el indice dado: configura objetos, musica,
  /// animacion del sacerdote, titulo y dialogo inicial antes de esperar la mision.
  /// </summary>
  /// <param name="indice">Indice de la escena dentro del array de escenas.</param>
  private IEnumerator EjecutarEscena(int indice)
  {
    if (indice >= escenas.Length)
    {
      yield return StartCoroutine(Ganar());
      yield break;
    }
    EscenaMisa escena = escenas[indice];
    ConfigurarObjetos(escena);
    if (escena.musicaOpcional != null && audioSource != null)
    {
      audioSource.clip = escena.musicaOpcional;
      audioSource.Play();
    }
    if (!string.IsNullOrEmpty(escena.animacionSacerdote) && animadorSacerdote != null)
      animadorSacerdote.Play(escena.animacionSacerdote);
    if (escena.puntoDestino != null && sacerdote != null)
      yield return StartCoroutine(MoverSacerdote(escena));
    if (escena.mostrarTitulo)
      yield return StartCoroutine(MostrarTitulo(escena.textoTitulo));
    yield return StartCoroutine(EsperarDialogo(escena.dialogoInicial));
    yield return StartCoroutine(EsperarMision(escena));
  }

  /// <summary>
  /// Muestra los visuales del jugador, crea y asigna la mision de postura
  /// y espera hasta recibir el resultado antes de continuar al acierto o fallo.
  /// </summary>
  /// <param name="escena">Escena actual cuya mision se va a ejecutar.</param>
  private IEnumerator EsperarMision(EscenaMisa escena)
  {
    if (escena.visualesJugador != null)
      foreach (GameObject v in escena.visualesJugador)
      {
        if (v != null) v.SetActive(true);
        if (pensamientoJugador != null)
          pensamientoJugador.SetActive(true);
        bool resultado = false;
        bool recibido = false;
        MisionPosicion mision = new MisionPosicion(
          escena.tipoAccion,
          escena.valorEsperado,
          () => { resultado = true; recibido = true; },
          () => { resultado = false; recibido = true; }
        );
        if (MisionesGlobal.Instancia != null)
          MisionesGlobal.Instancia.AsignarMision(mision);
        yield return new WaitUntil(() => recibido);
      }

```

```

    if (pensamientoJugador != null)
    pensamientoJugador.SetActive(false);
    if (escena.visualesJugador != null)
    foreach (GameObject v in escena.visualesJugador)
    if (v != null) v.SetActive(false);
    if (resultado)
    yield return StartCoroutine(AlAcertar(escena));
    else
    yield return StartCoroutine(AlFallar(escena));
}

/// <summary>
/// Muestra el dialogo de exito y avanza a la siguiente escena de la misa.
/// </summary>
/// <param name="escena">Escena que el jugador acabo de completar correctamente.</param>
private IEnumerator AlAcertar(EscenaMisa escena)
{
    yield return StartCoroutine(EsperarDialogo(escena.dialogoExito));
    _escenaActual++;
    yield return StartCoroutine(EjecutarEscena(_escenaActual));
}

/// <summary>
/// Muestra el dialogo de fallo, baja la cortina negra, reposiciona al jugador
/// y reintenta la escena actual.
/// </summary>
/// <param name="escena">Escena en la que el jugador fallo.</param>
private IEnumerator AlFallar(EscenaMisa escena)
{
    yield return StartCoroutine(EsperarDialogo(escena.dialogoFallo));
    // Bajar cortina, resetear jugador, subir cortina y reintentar
    yield return StartCoroutine(MoverCuadro(0f));
    if (transformJugador != null && puntoResetJugador != null)
    transformJugador.position = puntoResetJugador.position;
    yield return new WaitForSeconds(0.3f);
    yield return StartCoroutine(MoverCuadro(2f));
    yield return StartCoroutine(EjecutarEscena(_escenaActual));
}

#endregion
#region Ganar / Reiniciar
/// <summary>
/// Activa el objeto de comunion, oculta la barra de vidas,
/// devuelve el control al jugador y termina el flujo de la misa.
/// </summary>
private IEnumerator Ganar()
{
    if (objetoGanar != null) objetoGanar.SetActive(true);
    if (vidaUI != null) vidaUI.gameObject.SetActive(false);
    if (jugadorData != null) jugadorData.puedeMoverse = true;
    if (visualJugadorReal != null) visualJugadorReal.SetActive(true);
    _enFlujo = false;
    yield break;
}

/// <summary>
/// Reinicia el flujo completo de la misa: resetea vidas y escenas,
/// reposiciona al jugador y vuelve a ejecutar desde la escena 0.
/// </summary>
private IEnumerator Reiniciar()
{
    yield return StartCoroutine(MoverCuadro(0f));
    _escenaActual = 0;
    vidaUI?.Resetear();
    if (transformJugador != null && puntoResetJugador != null)
    transformJugador.position = puntoResetJugador.position;
    yield return new WaitForSeconds(0.5f);
    yield return StartCoroutine(MoverCuadro(2f));
    // Volver a ejecutar desde la escena 0 directamente
    yield return StartCoroutine(EjecutarEscena(_escenaActual));
}

```

```

}
#endregion
#region Helpers
    /// <summary>
    /// Inicia un dialogo y espera hasta que el DialogoController dispare OnDialogoTerminado.
    /// No hace nada si el dialogo es null.
    /// </summary>
    /// <param name="dialogo">Datos del dialogo a reproducir.</param>
    private IEnumerator EsperarDialogo(DialogoData dialogo)
    {
        if (dialogo == null) yield break;
        bool terminado = false;
        System.Action handler = () => terminado = true;
        DialogoController.OnDialogoTerminado += handler;
        _dialogoController.Iniciar(dialogo);
        yield return new WaitUntil(() => terminado);
        DialogoController.OnDialogoTerminado -= handler;
    }

    /// <summary>
    /// Muestra el panel de titulo con el texto indicado durante 2.5 segundos y luego lo oculta.
    /// </summary>
    /// <param name="texto">Texto del titulo de la escena a mostrar.</param>
    private IEnumerator MostrarTitulo(string texto)
    {
        if (panelTitulo == null) yield break;
        if (textoTitulo != null) textoTitulo.text = texto;
        panelTitulo.SetActive(true);
        yield return new WaitForSeconds(2.5f);
        panelTitulo.SetActive(false);
    }

    /// <summary>
    /// Activa e inactiva los GameObjects configurados en la escena de la misa.
    /// </summary>
    /// <param name="escena">Escena cuya lista de objetos se va a procesar.</param>
    private void ConfigurarObjetos(EscenaMisa escena)
    {
        if (escena.activar != null)
        foreach (GameObject obj in escena.activar)
        if (obj != null) obj.SetActive(true);
        if (escena.desactivar != null)
        foreach (GameObject obj in escena.desactivar)
        if (obj != null) obj.SetActive(false);
    }

    /// <summary>
    /// Mueve al sacerdote hacia el punto de destino de la escena a velocidad constante.
    /// </summary>
    /// <param name="escena">Escena que contiene el punto destino y la velocidad del sacerdote.</param>
    private IEnumerator MoverSacerdote(EscenaMisa escena)
    {
        while (Vector3.Distance(sacerdote.position, escena.puntoDestino.position) > 0.05f)
        {
            sacerdote.position = Vector3.MoveTowards(
                sacerdote.position,
                escena.puntoDestino.position,
                escena.velocidadSacerdote * Time.deltaTime
            );
            yield return null;
        }
        sacerdote.position = escena.puntoDestino.position;
    }

    /// <summary>
    /// Mueve el cuadro negro suavemente hasta la posicion Y indicada para crear transiciones cinematicas.
    /// Y = 0 lo baja (pantalla negra), Y = 2 lo sube (revela la escena).
    /// </summary>
    /// <param name="yDestino">Posicion Y local de destino del cuadro negro.</param>
    private IEnumerator MoverCuadro(float yDestino)
    {

```

```

if (cuadroNegro == null) yield break;
Vector3 posInicial = cuadroNegro.localPosition;
Vector3 posDestino = new Vector3(posInicial.x, yDestino, posInicial.z);
float distancia = Mathf.Abs(posInicial.y - yDestino);
if (distancia < 0.001f) yield break;
float tiempoTotal = distancia / velocidadCuadro;
float tiempoTranscurrido = 0f;
while (tiempoTranscurrido < tiempoTotal)
{
    tiempoTranscurrido += Time.deltaTime;
    float t = Mathf.Clamp01(tiempoTranscurrido / tiempoTotal);
    cuadroNegro.localPosition = Vector3.Lerp(posInicial, posDestino, t);
    yield return null;
}
cuadroNegro.localPosition = posDestino;
}
#endregion
}

```

VidaUI.cs

42 líneas

Scripts/Misa/VidaUI.cs

Descripcion	Muestra y gestiona los corazones de vida del jugador durante la misa. Resetear() activa los N corazones al inicio o al reiniciar. PerderVida() desactiva el ultimo corazon activo. SinVidas es una propiedad que GestorMisa puede consultar para decidir si reiniciar completamente. Dependencias: GestorMisa.
--------------------	--

```

using UnityEngine;

/// <summary>
/// Muestra los 3 corazones de vida del jugador.
/// Se desactiva uno por cada fallo.
/// </summary>
public class VidaUI : MonoBehaviour
{
    [Tooltip("Los 3 corazones en orden, el ultimo es el primero en perderse")]
    public GameObject[] corazones;
    private int _vidasActuales;
    private void Start()
    {
        Resetear();
    }

    /// <summary>
    /// Restaura todos los corazones a su estado visible y reinicia el contador de vidas.
    /// </summary>
    public void Resetear()
    {
        _vidasActuales = corazones.Length;
        foreach (GameObject c in corazones)
        {
            if (c != null) c.SetActive(true);
        }

        /// <summary>
        /// Descuenta una vida y desactiva el corazon correspondiente.
        /// No hace nada si ya no quedan vidas.
        /// </summary>
        public void PerderVida()
        {
            if (_vidasActuales <= 0) return;
            _vidasActuales--;
            corazones[_vidasActuales].SetActive(false);
        }

        /// <summary>True cuando el jugador ha perdido todas sus vidas.</summary>
        public bool SinVidas => _vidasActuales <= 0;
    }
}

```

Misiones

Implementaciones concretas del sistema de misiones. Cada clase extiende la base de mision y define las condiciones de inicio, completado y fallo de una tarea especifica dentro del juego.

Archivo	Líneas	Ruta relativa
Mision01.cs	149	Misiones/Mision01.cs
MisionMinijuego.cs	271	Misiones/MisionMinijuego.cs
MisionPosicion.cs	43	Misiones/MisionPosicion.cs
MisionPosturaParado. cs	261	Misiones/MisionPosturaParado.cs

Mision01.cs

149 líneas

Scripts/Misiones/Mision01.cs

Descripcion

Primera mision del juego: introduce al jugador al sistema de misiones. Hereda de MisionBase y define las condiciones de completado para la accion inicial (tipicamente postura parado). Su metodo Actuar() desencadena el siguiente paso narrativo, como activar una zona o iniciar un dialogo. Dependencias: MisionBase, MisionesGlobal.

```
using UnityEngine;
/// <summary>
/// Primera mision del juego.
/// Flujo: dialogo inicial → suscribe mision + activa PENSAMIENTOJUGADOR
/// → si falla: dialogo fallo → vuelve a suscribir + activar
/// → si acierta: dialogo exito → resto de logica a cargo del disenador
/// </summary>
public class Mision01 : MonoBehaviour
{
    [Header("Dialogos")]
    public DialogoData dialogoInicial;
    public DialogoData dialogoFallo;
    public DialogoData dialogoExito;
    [Header("Referencias")]
    public GameObject pensamientoJugador;
    [Header("Logicas de activacion")]
    public JugadorController controladorJugador;
    public AudioSource jugadorAudioSource;
    public GameObject[] VisualesJugador;
    public GameObject[] VisualesJugadorDesactivadas;
    public GameObject BurbujaEspaciadora;
    private DialogoController _dialogoController;
    #region Unity Callbacks
    private void Start()
    {
        _dialogoController = FindFirstObjectByType<DialogoController>();
        if (_dialogoController == null)
        {
            Debug.LogError("[Mision01] No se encontro DialogoController en escena.");
            return;
        }
        DesactivarPensamiento();
        IniciarDialogoInicial();
    }
    #endregion
    #region Flujo de Mision
    /// <summary>Suscribe el handler de fin de dialogo e inicia el dialogo de introduccion.</summary>
    private void IniciarDialogoInicial()
    {
        DialogoController.OnDialogoTerminado += AlTerminarDialogoInicial;
        _dialogoController.Iniciar(dialogoInicial);
    }
}
```



```

    /// <summary>Se llama al terminar el dialogo inicial; desuscribe el handler y activa la mision.</summary>
    private void AlTerminarDialogoInicial()
    {
        DialogoController.OnDialogoTerminado -= AlTerminarDialogoInicial;
        SuscribirseMision();
    }

    /// <summary>
    /// Crea y asigna una MisionPosicion de tipo Postura=1 (parado),
    /// activa el pensamiento del jugador y muestra la burbuja de espacio.
    /// </summary>
    private void SuscribirseMision()
    {
        MisionPosicion mision = new MisionPosicion(
            TipoAccion.Postura, 1,
            AlAcertar, AlFallar
        );
        if (MisionesGlobal.Instancia == null)
        {
            Debug.LogError("[Mision01] MisionesGlobal no encontrado.");
            return;
        }
        MisionesGlobal.Instancia.AsignarMision(mision);
        ActivarPensamiento();
        if (BurbujaEspaciadora != null)
            BurbujaEspaciadora.SetActive(true);
    }

    /// <summary>Se llama cuando el minijuego reporta un resultado incorrecto; inicia el dialogo de
    fallo.</summary> ...
    private void AlFallar()
    {
        DesactivarPensamiento();
        DialogoController.OnDialogoTerminado += AlTerminarDialogoFallo;
        _dialogoController.Iniciar(dialogoFallo);
    }

    /// <summary>Se llama al terminar el dialogo de fallo; vuelve a suscribir la mision para
    reintentar.</summary>
    private void AlTerminarDialogoFallo()
    {
        DialogoController.OnDialogoTerminado -= AlTerminarDialogoFallo;
        SuscribirseMision();
    }

    /// <summary>Se llama cuando el minijuego reporta el resultado correcto; inicia el dialogo de
    exito.</summary>
    private void AlAcertar()
    {
        DesactivarPensamiento();
        DialogoController.OnDialogoTerminado += AlTerminarDialogoExito;
        _dialogoController.Iniciar(dialogoExito);
    }

    /// <summary>
    /// Se llama al terminar el dialogo de exito; reactiva al jugador,
    /// muestra sus visuales finales y habilita el AudioSource de caminata.
    /// </summary>
    private void AlTerminarDialogoExito()
    {
        DialogoController.OnDialogoTerminado -= AlTerminarDialogoExito;
        if (controladorJugador != null)
            controladorJugador.enabled = true;
        foreach (var item in VisualesJugador)
            if (item != null) item.SetActive(true);
        foreach (var item in VisualesJugadorDesactivadas)
            if (item != null) item.SetActive(false);
        if (jugadorAudioSource != null)
            jugadorAudioSource.enabled = true;
    }
}
#endregion
#region Helpers

```

```

    /// <summary>Activa el panel de pensamiento del jugador para indicarle que debe elegir una
    postura.</summary>
    private void ActivarPensamiento()
    {
        if (pensamientoJugador != null)
            pensamientoJugador.SetActive(true);
    }

    /// <summary>Desactiva el panel de pensamiento del jugador.</summary>
    private void DesactivarPensamiento()
    {
        if (pensamientoJugador != null)
            pensamientoJugador.SetActive(false);
    }
#endregion
}

```

MisionMinijuego.cs

271 líneas

Scripts/Misiones/MisionMinijuego.cs

Descripción	Misión que requiere completar un minijuego específico. Hereda de MisiónBase, abre el panel CuerpoUI para que el jugador seleccione la parte correcta del cuerpo y espera el resultado a través de MisionesGlobal. Implementa Actuar() para avanzar el flujo narrativo tras el éxito. Dependencias: MisiónBase, CuerpoUIController, MisionesGlobal.
--------------------	--

```

using UnityEngine;
using System.Collections;
using System.Collections.Generic;

/// <summary>
/// Misión genérica reutilizable.
/// Configura TipoAcción y respuestaEsperada desde el Inspector.
/// </summary>
public class MisionMinijuego : MonoBehaviour
{
    [Header("Referencias")]
    public GameObject indicadorSpace;
    public GameObject pensamientoJugador;
    public Transform cuadroNegro;

    [Header("Habilidad Nueva (opcional)")]
    public JugadorHabilidadesData habilidadesJugador;
    public ParteCuerpo habilidadADesbloquear;
    public GameObject uiNuevaHabilidad;

    [Header("Misión")]
    public TipoAcción tipoAcción;
    public int respuestaEsperada;

    [Header("Dialogos")]
    public DialogoData dialogoInicial;
    public DialogoData dialogoExito;
    public DialogoData dialogoFallo;

    [Header("Cinematica - Sprites")]
    public List<SpriteRenderer> spritesDesactivar;
    public List<SpriteRenderer> spritesActivar;

    [Header("Cinematica - Tiempos")]
    public float velocidadCuadro = 2f;
    public float tiempoAntesDeSwap = 0.3f;
    public float tiempoEntreSwapSubida = 0.5f;
    public float tiempoAntesSegundoBajada = 0.8f;
    public float tiempoEntreSwapFinalSubida = 0.5f;

    [Header("Objetos Finales")]
    public List<GameObject> objetosADesactivar;
    public List<GameObject> objetosAActivar;

    private bool _jugadorDentro = false;
    private bool _misiónActiva = false;
    private JugadorData _jugadorData;
    private DialogoController _dialogoController;

    #region Unity Callbacks
    private void Awake()
    {

```

```

{
_dialogoController = FindFirstObjectByType<DialogoController>();
if (indicadorSpace != null) indicadorSpace.SetActive(false);
}

private void Update()
{
if (!_jugadorDentro || _misionActiva) return;
if (Input.GetKeyDown(KeyCode.Space))
StartCoroutine(IniciarMisionSiguienteFrame());
}

private void OnTriggerEnter(Collider other)
{
if (!other.CompareTag("Player")) return;
_jugadorData = other.GetComponent<JugadorData>();
_jugadorDentro = true;
if (indicadorSpace != null) indicadorSpace.SetActive(true);
}

private void OnTriggerExit(Collider other)
{
if (!other.CompareTag("Player")) return;
_jugadorDentro = false;
if (indicadorSpace != null) indicadorSpace.SetActive(false);
}

#endregion
#region Flujo
/// <summary>
/// Espera un frame para evitar que el Space que activa la mision sea consumido
/// por el DialogoController, luego bloquea al jugador e inicia el dialogo.
/// </summary>
private IEnumerator IniciarMisionSiguienteFrame()
{
_misionActiva = true;
if (indicadorSpace != null) indicadorSpace.SetActive(false);
if (_jugadorData != null) _jugadorData.puedeMoverse = false;
yield return null;
DialogoController.OnDialogoTerminado += AlTerminarDialogoInicial;
_dialogoController.Iniciar(dialogoInicial);
}

/// <summary>
/// Se llama al terminar el dialogo inicial; desbloquea la habilidad si corresponde
/// y registra la mision en MisionesGlobal.
/// </summary>
private void AlTerminarDialogoInicial()
{
DialogoController.OnDialogoTerminado -= AlTerminarDialogoInicial;
if (habilidadesJugador != null)
{
DesbloquearHabilidad();
if (uiNuevaHabilidad != null) uiNuevaHabilidad.SetActive(true);
}
SuscribirMision();
}

/// <summary>
/// Activa el flag de habilidad correspondiente en JugadorHabilidadesData
/// segun la parte del cuerpo configurada en el Inspector.
/// </summary>
private void DesbloquearHabilidad()
{
switch (habilidadADesbloquear)
{
case ParteCuerpo.Piernas: habilidadesJugador.tienePiernas = true; break;
case ParteCuerpo.Pecho: habilidadesJugador.tienePecho = true; break;
case ParteCuerpo.Cabeza: habilidadesJugador.tieneCabeza = true; break;
case ParteCuerpo.Boca: habilidadesJugador.tieneBoca = true; break;
case ParteCuerpo.ManoDerecha: habilidadesJugador.tieneManoDerecha = true; break;
}
}
}

```

```

    /// <summary>
    /// Crea y asigna en MisionesGlobal una MisionPosicion con el tipo y valor esperados,
    /// luego activa el pensamiento del jugador.
    /// </summary>
    private void SuscribirseMision()
    {
        MisionPosicion mision = new MisionPosicion(
            tipoAccion, respuestaEsperada,
            AlAcertar, AlFallar
        );
        if (MisionesGlobal.Instancia != null)
            MisionesGlobal.Instancia.AsignarMision(mision);
        else
            Debug.LogError("[MisionMinijuego] MisionesGlobal no encontrado.");
        if (pensamientoJugador != null)
            pensamientoJugador.SetActive(true);
    }

    /// <summary>Se llama cuando el minijuego devuelve un resultado incorrecto; muestra el dialogo de
    fallo.</summary>
    private void AlFallar()
    {
        if (pensamientoJugador != null) pensamientoJugador.SetActive(false);
        DialogoController.OnDialogoTerminado += AlTerminarDialogoFallo;
        _dialogoController.Iniciar(dialogoFallo);
    }

    /// <summary>Se llama al terminar el dialogo de fallo; reactiva el movimiento y permite
    reintentar.</summary>
    private void AlTerminarDialogoFallo()
    {
        DialogoController.OnDialogoTerminado -= AlTerminarDialogoFallo;
        if (_jugadorData != null) _jugadorData.puedeMoverse = true;
        StartCoroutine(ReactivarSiguienteFrame());
    }

    /// <summary>
    /// Espera dos frames antes de desmarcar la mision como activa
    /// para evitar que el jugador la reactive el mismo frame que termino el dialogo.
    /// </summary>
    private IEnumerator ReactivarSiguienteFrame()
    {
        yield return null;
        yield return null;
        _misionActiva = false;
    }

    /// <summary>Se llama cuando el minijuego devuelve el resultado correcto; muestra el dialogo de
    exito.</summary>
    private void AlAcertar()
    {
        if (pensamientoJugador != null) pensamientoJugador.SetActive(false);
        DialogoController.OnDialogoTerminado += AlTerminarDialogoExito;
        _dialogoController.Iniciar(dialogoExito);
    }

    /// <summary>Se llama al terminar el dialogo de exito; lanza la cinematica de transicion.</summary>
    private void AlTerminarDialogoExito()
    {
        DialogoController.OnDialogoTerminado -= AlTerminarDialogoExito;
        StartCoroutine(Cinematica());
    }
}

#endregion
#region Cinematica
    /// <summary>
    /// Ejecuta la cinematica de transicion: baja cortina, hace swap de sprites,
    /// sube cortina, activa/desactiva objetos finales y reactiva al jugador.
    /// </summary>
    private IEnumerator Cinematica()
    {
        yield return StartCoroutine(MoverCuadro(0f));
        yield return new WaitForSeconds(tiempoAntesDeSwap);
    }
}

```

```

SwapSprites(true);
yield return new WaitForSeconds(tiempoEntreSwapYSubida);
yield return StartCoroutine(MoverCuadro(2f));
yield return new WaitForSeconds(tiempoAntesSegundoBajada);
yield return StartCoroutine(MoverCuadro(0f));
yield return new WaitForSeconds(tiempoAntesDeSwap);
SwapSprites(false);
yield return new WaitForSeconds(tiempoEntreSwapFinalYSubida);
yield return StartCoroutine(MoverCuadro(2f));

if (objetosADesactivar != null)
foreach (GameObject obj in objetosADesactivar)
if (obj != null) obj.SetActive(false);
if (objetosAActivar != null)
foreach (GameObject obj in objetosAActivar)
if (obj != null) obj.SetActive(true);
if (_jugadorData != null) _jugadorData.puedeMoverse = true;
this.enabled = false;
}

/// <summary>
/// Intercambia la visibilidad de los grupos de sprites para simular el cambio de atuendo del jugador.
/// </summary>
/// <param name="primerSwap">Si es true aplica el primer intercambio; si es false lo invierte.</param>
private void SwapSprites(bool primerSwap)
{
foreach (SpriteRenderer sr in spritesDesactivar)
if (sr != null) sr.enabled = !primerSwap;
foreach (SpriteRenderer sr in spritesActivar)
if (sr != null) sr.enabled = primerSwap;
}

/// <summary>
/// Mueve el cuadro negro suavemente hasta la posicion Y indicada.
/// Y = 0 cubre la pantalla, Y = 2 la descubre.
/// </summary>
/// <param name="yDestino">Posicion Y local de destino del cuadro negro.</param>
private IEnumerator MoverCuadro(float yDestino)
{
if (cuadroNegro == null) yield break;
Vector3 posInicial = cuadroNegro.localPosition;
Vector3 posDestino = new Vector3(posInicial.x, yDestino, posInicial.z);
float distancia = Mathf.Abs(posInicial.y - yDestino);
if (distancia < 0.001f) yield break;
float tiempoTotal = distancia / velocidadCuadro;
float tiempoTranscurrido = 0f;
while (tiempoTranscurrido < tiempoTotal)
{
tiempoTranscurrido += Time.deltaTime;
float t = Mathf.Clamp01(tiempoTranscurrido / tiempoTotal);
cuadroNegro.localPosition = Vector3.Lerp(posInicial, posDestino, t);
yield return null;
}
cuadroNegro.localPosition = posDestino;
}
#endregion
}

```

MisionPosicion.cs

43 líneas

Scripts/Misiones/MisionPosicion.cs

Descripcion

Mision de postura: evalua si el resultado recibido coincide con el tipo y valor esperados. Recibe dos callbacks en el constructor: onExito y onFallo. Actuar() invoca onExito; ActuarFallo() (llamado por MisionesGlobal si el resultado no coincide) invoca onFallo. GestorMisa la usa para cada escena. Dependencias: MisionBase, MisionesGlobal.

```

using UnityEngine;
using System;

/// <summary>

```

```

/// Mision que espera una postura especifica.
/// Acepta callbacks de acierto y fallo para que
/// cada mision concreta defina que hacer en cada caso.
/// </summary>
public class MisionPosicion : MisionBase
{
    private readonly Action _onAcierto;
    private readonly Action _onFallo;
    /// <summary>
    /// Crea una mision de postura con callbacks para acierto y fallo.
    /// </summary>
    /// <param name="tipo">Tipo de accion esperada (Postura, Mano, etc.).</param>
    /// <param name="valor">Codigo de postura correcta (1=parado, 2=sentado, 3=arrodillado).</param>
    /// <param name="onAcierto">Accion que se ejecuta cuando el resultado coincide.</param>
    /// <param name="onFallo">Accion que se ejecuta cuando el resultado no coincide.</param>
    public MisionPosicion(TipoAccion tipo, int valor, Action onAcierto, Action onFallo)
    : base(tipo, valor)
    {
        _onAcierto = onAcierto;
        _onFallo = onFallo;
    }
    /// <summary>
    /// Se llama cuando el resultado coincide con lo esperado.
    /// </summary>
    public override void Actuar()
    {
        _onAcierto?.Invoke();
    }
    /// <summary>
    /// Se llama cuando el resultado NO coincide con lo esperado.
    /// </summary>
    public void ActuarFallo()
    {
        _onFallo?.Invoke();
    }
}

```

MisionPosturaParado.cs

261 líneas

Scripts/Misiones/MisionPosturaParado.cs

Descripcion	Mision compleja de postura en posicion parado. Abre el panel CuerpoUI, espera que el jugador interactue con multiples partes del cuerpo en secuencia y valida que todas esten en la postura correcta. Al completarse desbloquea la siguiente fase del juego. Dependencias: MisionBase, CuerpoUIController, JugadorHabilidadesData, MisionesGlobal.
--------------------	--

```

using UnityEngine;
using System.Collections;
using System.Collections.Generic;
/// <summary>
/// Mision de postura parado.
/// Trigger detecta al jugador, muestra indicador de space,
/// lanza dialogos, suscribe mision y maneja cinematica de exito/fallo.
/// </summary>
public class MisionPosturaParado : MonoBehaviour
{
    [Header("Referencias")]
    [Tooltip("Sprite hijo que indica al jugador que presione space")]
    public GameObject indicadorSpace;
    [Tooltip("PENSAMIENTOJUGADOR – se activa al terminar el dialogo inicial")]
    public GameObject pensamientoJugador;
    [Tooltip("Cuadro negro de la camara para la cinematica")]
    public Transform cuadroNegro;
    [Tooltip("Pared u obstaculo que se desactiva al completar la mision")]
    public GameObject pared;
    [Header("Dialogos")]
    public DialogoData dialogoInicial;
}

```

```

public DialogoData dialogoExito;
public DialogoData dialogoFallo;
[Header("Cinematica - Sprites")]
[Tooltip("Sprites que se DESACTIVAN en el primer swap")]
public List<SpriteRenderer> spritesDesactivar;
[Tooltip("Sprites que se ACTIVAN en el primer swap")]
public List<SpriteRenderer> spritesActivar;
[Header("Cinematica - Tiempos")]
public float velocidadCuadro = 2f;
public float tiempoAntesDeSwap = 0.3f;
public float tiempoEntreSwapYSubida = 0.5f;
public float tiempoAntesSegundoBajada = 0.8f;
public float tiempoEntreSwapFinalYSubida = 0.5f;
private bool _jugadorDentro = false;
private bool _misionActiva = false;
private JugadorData _jugadorData;
private DialogoController _dialogoController;
#region Unity Callbacks
private void Awake()
{
    _dialogoController = FindFirstObjectByType<DialogoController>();
    if (indicadorSpace != null)
    indicadorSpace.SetActive(false);
}
private void Update()
{
    if (!_jugadorDentro || !_misionActiva) return;
    if (Input.GetKeyDown(KeyCode.Space))
    StartCoroutine(IniciarMisionSiguienteFrame());
}
private void OnTriggerEnter(Collider other)
{
    if (!other.CompareTag("Player")) return;
    _jugadorData = other.GetComponent<JugadorData>();
    _jugadorDentro = true;
    if (indicadorSpace != null)
    indicadorSpace.SetActive(true);
}
private void OnTriggerExit(Collider other)
{
    if (!other.CompareTag("Player")) return;
    _jugadorDentro = false;
    if (indicadorSpace != null)
    indicadorSpace.SetActive(false);
}
#endregion
#region Flujo de Mision
/// <summary>
/// Espera un frame antes de abrir el dialogo para evitar que el mismo
/// Space que inicia la mision lo consuma el DialogoController.
/// </summary>
private IEnumerator IniciarMisionSiguienteFrame()
{
    _misionActiva = true;
    if (indicadorSpace != null)
    indicadorSpace.SetActive(false);
    if (_jugadorData != null)
    _jugadorData.puedeMoverse = false;
    yield return null; // ← espera el siguiente frame
    DialogoController.OnDialogoTerminado += AlTerminarDialogoInicial;
    _dialogoController.Iniciar(dialogoInicial);
}
/// <summary>Se llama al terminar el dialogo inicial; registra la mision de postura parado.</summary>
private void AlTerminarDialogoInicial()
{
    DialogoController.OnDialogoTerminado -= AlTerminarDialogoInicial;
}

```

```

SuscribirMision();
}

/// <summary>
/// Crea y asigna en MisionesGlobal una MisionPosicion que espera Postura=1 (parado).
/// Activa el pensamiento del jugador una vez registrada la mision.
/// </summary>
private void SuscribirMision()
{
MisionPosicion mision = new MisionPosicion(
TipoAccion.Postura,
1, // parado
AlAcertar,
AlFallar
);
if (MisionesGlobal.Instancia != null)
MisionesGlobal.Instancia.AsignarMision(mision);
else
Debug.LogError("[MisionPosturaParado] MisionesGlobal no encontrado.");
// Se activa DESPUES de que termina el dialogo inicial
if (pensamientoJugador != null)
pensamientoJugador.SetActive(true);
}

/// <summary>Se llama cuando la postura elegida no es la correcta; muestra el dialogo de fallo.</summary>
private void AlFallar()
{
if (pensamientoJugador != null)
pensamientoJugador.SetActive(false);
DialogoController.OnDialogoTerminado += AlTerminarDialogoFallo;
_dialogoController.Iniciar(dialogoFallo);
}

/// <summary>Se llama al terminar el dialogo de fallo; reactiva el movimiento para reintentar.</summary>
private void AlTerminarDialogoFallo()
{
DialogoController.OnDialogoTerminado -= AlTerminarDialogoFallo;
if (_jugadorData != null)
_jugadorData.puedeMoverse = true;
_misionActiva = false;
}

/// <summary>Se llama cuando la postura elegida es correcta; muestra el dialogo de exito.</summary>
private void AlAcertar()
{
if (pensamientoJugador != null)
pensamientoJugador.SetActive(false);
DialogoController.OnDialogoTerminado += AlTerminarDialogoExito;
_dialogoController.Iniciar(dialogoExito);
}

/// <summary>Se llama al terminar el dialogo de exito; lanza la cinematica de cambio de sprites.</summary>
private void AlTerminarDialogoExito()
{
DialogoController.OnDialogoTerminado -= AlTerminarDialogoExito;
StartCoroutine(Cinematica());
}
#endregion
#region Cinematica
private IEnumerator Cinematica()
{
// 1 – Bajar cuadro negro
yield return StartCoroutine(MoverCuadro(0f));
yield return new WaitForSeconds(tiempoAntesDeSwap);
// 2 – Primer swap de sprites
SwapSprites(true);
yield return new WaitForSeconds(tiempoEntreSwapYSubida);
// 3 – Subir cuadro negro
yield return StartCoroutine(MoverCuadro(2f));
yield return new WaitForSeconds(tiempoAntesSegundoBajada);
// 4 – Bajar cuadro negro de nuevo

```



```

yield return StartCoroutine(MoverCuadro(0f));
yield return new WaitForSeconds(tiempoAntesDeSwap);
// 5 - Segundo swap (invertido)
SwapSprites(false);
yield return new WaitForSeconds(tiempoEntreSwapFinalYSubida);
// 6 - Subir cuadro negro
yield return StartCoroutine(MoverCuadro(2f));
// 7 - Desactivar pared
if (pared != null)
pared.SetActive(false);
// 8 - Devolver movimiento
if (_jugadorData != null)
_jugadorData.puedeMoverse = true;
// 9 - Desactivar este script
this.enabled = false;
}

/// <summary>
/// primer swap: desactiva A y activa B.
/// segundo swap: activa A y desactiva B.
/// </summary>
private void SwapSprites(bool primerSwap)
{
foreach (SpriteRenderer sr in spritesDesactivar)
if (sr != null) sr.enabled = !primerSwap;
foreach (SpriteRenderer sr in spritesActivar)
if (sr != null) sr.enabled = primerSwap;
}

private IEnumerator MoverCuadro(float yDestino)
{
if (cuadroNegro == null) yield break;
Vector3 posInicial = cuadroNegro.localPosition;
Vector3 posDestino = new Vector3(posInicial.x, yDestino, posInicial.z);
float distancia = Mathf.Abs(posInicial.y - yDestino);
if (distancia < 0.001f) yield break;
float tiempoTotal = distancia / velocidadCuadro;
float tiempoTranscurrido = 0f;
while (tiempoTranscurrido < tiempoTotal)
{
tiempoTranscurrido += Time.deltaTime;
float t = Mathf.Clamp01(tiempoTranscurrido / tiempoTotal);
cuadroNegro.localPosition = Vector3.Lerp(posInicial, posDestino, t);
yield return null;
}
cuadroNegro.localPosition = posDestino;
}
#endregion
}

```

Modelos

Controladores MVC del juego. Implementan la lógica de presentación y comportamiento de cada entidad: jugador, cámara, diálogos, UI del cuerpo y todos los minijuegos. Son el núcleo de la jugabilidad.

Archivo	Líneas	Ruta relativa
CamaraController.cs	29	Modelos/Camara/CamaraController.cs
CuerpoUIController.cs	145	Modelos/Cuerpo/CuerpoUIController.cs
DialogoController.cs	154	Modelos/Dialogo/DialogoController.cs
JugadorController.cs	138	Modelos/Jugador/JugadorController.cs

TriggerCuerpoUI.cs	53	Modelos/Jugador/TriggerCuerpoUI.cs
MinijuegoCerebroController.cs	247	Modelos/Minijuego/Cabeza/MinijuegoCerebroController.cs
MinijuegoCantoController.cs	348	Modelos/Minijuego/Cantar/MinijuegoCantoController.cs
MinijuegoSaludoController.cs	187	Modelos/Minijuego/Mano/MinijuegoSaludoController.cs
MinijuegoSenalController.cs	273	Modelos/Minijuego/Mano/MinijuegoSenalController.cs
PanelManoController.cs	100	Modelos/Minijuego/Mano/PanelManoController.cs
MinijuegoPechoController.cs	188	Modelos/Minijuego/Pecho/MinijuegoPechoController.cs
MinijuegoPiernaController.cs	394	Modelos/Minijuego/Pierna/MinijuegoPiernaController.cs
MinijuegoResponderController.cs	220	Modelos/Minijuego/boca/MinijuegoResponderController.cs
PanelBocaController.cs	60	Modelos/Minijuego/boca/PanelBocaController.cs
TransicionZonaController.cs	163	Modelos/transicion/TransicionZonaController.cs

CamaraController.cs

29 líneas

Scripts/Modelos/Camara/CamaraController.cs

Descripcion	Controlador de seguimiento suave de camara. En LateUpdate() interpola la posición de la camara hacia el jugador usando Lerp con velocidadSmooth. Solo opera cuando CamaraData.seguirJugador es true, lo que permite inmovilizarla durante diálogos. Dependencias: CamaraData.
--------------------	---

```

using UnityEngine;

/// <summary>
/// Controlador de la camara principal. Lee los datos de CamaraData
/// y mueve la camara suavemente en LateUpdate para que siempre siga
/// al jugador por el eje X sin interferir con Y ni Z.
/// </summary>
[RequireComponent(typeof(CamaraData))]
public class CamaraController : MonoBehaviour
{
    private CamaraData _data;

    private void Awake()
    {
        _data = GetComponent<CamaraData>();
    }

    /// <summary>
    /// Mueve la camara hacia la posición X del jugador con interpolación suave.
    /// Se ejecuta en LateUpdate para que el jugador ya haya actualizado su posición.
    /// </summary>
    private void LateUpdate()
    {
        if (!_data.seguirJugador || _data.jugador == null) return;
        float nuevoX = Mathf.Lerp(transform.position.x, _data.jugador.position.x, _data.velocidadSmooth * Time.deltaTime);
        transform.position = new Vector3(nuevoX, transform.position.y, transform.position.z);
    }
}

```

```

}
}

```

CuerpoUIController.cs

145 líneas

Scripts/Modelos/Cuerpo/CuerpoUIController.cs

Descripción	Controlador del panel de selección de partes del cuerpo. Itera ParteCuerpoData[] para registrar listeners en cada botón. Al pulsar, verifica JugadorHabilidadesData y dispara el evento estático OnParteSeleccionada(ParteCuerpo) que MinijuegoManager escucha. Aplica colores de estado bloqueado/desbloqueado. Dependencias: CuerpoUIData, JugadorHabilidadesData, MinijuegoManager.
--------------------	--

```

using UnityEngine;

/// <summary>
/// Controlador de la UI del cuerpo.
/// Abre el panel al llamarse desde el trigger del jugador,
/// activa o desactiva botones según las habilidades desbloqueadas,
/// y notifica que parte fue seleccionada.
/// </summary>
[RequireComponent(typeof(CuerpoUIData))]
public class CuerpoUIController : MonoBehaviour
{
    private CuerpoUIData _data;
    private JugadorHabilidadesData _habilidades;

    /// <summary>
    /// Se dispara cuando el jugador selecciona una parte del cuerpo.
    /// El minijuego correspondiente se suscribe a este evento.
    /// </summary>
    public static event System.Action<ParteCuerpo> OnParteSeleccionada;

    #region Unity Callbacks
    private void Awake()
    {
        ObtenerReferencias();
    }

    private void Start()
    {
        ValidarReferencias();
        CerrarPanel();
    }
    #endregion

    #region Inicialización
    private void ObtenerReferencias()
    {
        _data = GetComponent<CuerpoUIData>();
        _habilidades = FindFirstObjectByType<JugadorHabilidadesData>();
    }

    private void ValidarReferencias()
    {
        if (_data.panelCuerpo == null)
            Debug.LogError("[CuerpoUIController] panelCuerpo no asignado en CuerpoUIData.");
        if (_habilidades == null)
            Debug.LogError("[CuerpoUIController] No se encontró JugadorHabilidadesData en escena.");
        if (_data.partes == null || _data.partes.Length == 0)
            Debug.LogError("[CuerpoUIController] El array de partes está vacío en CuerpoUIData.");
    }
    #endregion

    #region API Pública
    /// <summary>
    /// Abre el panel y configura los botones según las habilidades del jugador.
    /// Llamado por el trigger del jugador al recibir click.
    /// </summary>
    public void AbrirPanel()
    {
        if (_data.panelCuerpo == null || _habilidades == null)
        {
            Debug.LogWarning("[CuerpoUIController] No se puede abrir el panel: referencias faltantes.");
        }
    }

```

```
return;
}
ConfigurarBotones();
_data.panelCuerpo.SetActive(true);
}
/// <summary>
/// Cierra el panel y limpia los listeners de los botones.
/// </summary>
public void CerrarPanel()
{
    if (_data.panelCuerpo == null)
        return;
    _data.panelCuerpo.SetActive(false);
    LimpiarBotones();
}
#endregion
#region Botones
/// <summary>
/// Recorre el array de partes, activa o desactiva cada boton
/// segun el estado de habilidades del jugador, y asigna listeners.
/// </summary>
private void ConfigurarBotones()
{
    if (_data.partes == null)
        return;
    foreach (ParteCuerpoData entrada in _data.partes)
    {
        if (entrada == null || entrada.boton == null)
        {
            Debug.LogWarning("[CuerpoUIController] Entrada de parte o boton nulo. Revisar Inspector.");
            continue;
        }
        bool desbloqueada = _habilidades.EstaDesbloqueada(entrada.parte);
        entrada.boton.interactable = desbloqueada;
        if (entrada.imagenBoton != null)
            entrada.imagenBoton.color = desbloqueada ? _data.colorDesbloqueado : _data.colorBloqueado;
        entrada.boton.onClick.RemoveAllListeners();
        if (desbloqueada)
        {
            ParteCuerpo parteCapturada = entrada.parte;
            entrada.boton.onClick.AddListener(() => AlSeleccionarParte(parteCapturada));
        }
    }
}
private void LimpiarBotones()
{
    if (_data.partes == null)
        return;
    foreach (ParteCuerpoData entrada in _data.partes)
    {
        if (entrada != null && entrada.boton != null)
            entrada.boton.onClick.RemoveAllListeners();
    }
}
private void AlSeleccionarParte(ParteCuerpo parte)
{
    CerrarPanel();
    OnParteSeleccionada?.Invoke(parte);
}
#endregion
}
```

Descripcion	Controlador del sistema de dialogos. Iniciar(DialogoData) abre el panel y muestra la primera linea. Cada vez que el jugador presiona la tecla de avance (definida en DialogoUIData) pasa a la siguiente DialogoLinea. Al terminar cierra el panel y dispara el evento estatico OnDialogoTerminado. Dependencias: DialogoUIData, DialogoData, DialogoLinea.
--------------------	--

```

using UnityEngine;

/// <summary>
/// Gestiona la reproduccion de dialogos linea por linea.
/// Completamente independiente del resto del juego.
/// Recibe un DialogoData, lo ejecuta y dispara OnDialogoTerminado al acabar.
/// </summary>
[RequireComponent(typeof(DialogoUIData))]
public class DialogoController : MonoBehaviour
{
    private DialogoUIData _data;
    private DialogoData _dialogoActual;
    private int _indiceActual;
    private bool _dialogoActivo;

    /// <summary>
    /// Se dispara cuando el dialogo actual termina completamente.
    /// Cualquier sistema que inicie un dialogo puede suscribirse aqui.
    /// </summary>
    public static event System.Action OnDialogoTerminado;

    #region Unity Callbacks
    private void Awake()
    {
        _data = GetComponent<DialogoUIData>();
    }
    private void Update()
    {
        if (!_dialogoActivo) return;
        if (Input.GetKeyDown(_data.teclaAvanzar))
            Avanzar();
    }
    #endregion
    #region API Publica
    /// <summary>
    /// Inicia un dialogo dado un DialogoData.
    /// Si hay un dialogo activo lo interrumpe y comienza el nuevo.
    /// </summary>
    public void Iniciar(DialogoData dialogo)
    {
        if (dialogo == null)
        {
            Debug.LogWarning("[DialogoController] Se intento iniciar un dialogo nulo.");
            return;
        }
        if (dialogo.lineas == null || dialogo.lineas.Length == 0)
        {
            Debug.LogWarning("[DialogoController] El dialogo no tiene lineas.");
            return;
        }
        _dialogoActual = dialogo;
        _indiceActual = 0;
        _dialogoActivo = true;
        _data.panelDialogo.SetActive(true);
        MostrarLineaActual();
    }
    #endregion
    #region Logica
    /// <summary>
    /// Avanza a la siguiente linea o termina el dialogo.
    /// </summary>
    private void Avanzar()
    {
        _indiceActual++;
    }

```

```
if (_indiceActual >= _dialogoActual.lineas.Length)
{
    Terminar();
    return;
}
MostrarLineaActual();
}

/// <summary>
/// Muestra la línea actual en la UI.
/// </summary>
private void MostrarLineaActual()
{
    if (!ValidarReferenciasUI()) return;
    DialogoLinea linea = _dialogoActual.lineas[_indiceActual];
    // Texto
    _data.textoDialogo.text = linea.texto;
    // Imágenes – se muestra solo la del lado activo
    if (linea.esIzquierda)
    {
        MostrarImagen(_data.imagenIzquierda, linea.imagen);
        OcultarImagen(_data.imagenDerecha);
    }
    else
    {
        MostrarImagen(_data.imagenDerecha, linea.imagen);
        OcultarImagen(_data.imagenIzquierda);
    }
}

private void MostrarImagen(UnityEngine.UI.Image imagen, Sprite sprite)
{
    if (imagen == null) return;
    imagen.sprite = sprite;
    imagen.gameObject.SetActive(sprite != null);
}

private void OcultarImagen(UnityEngine.UI.Image imagen)
{
    if (imagen == null) return;
    imagen.gameObject.SetActive(false);
}

/// <summary>
/// Cierra el panel y dispara el evento de fin.
/// </summary>
private void Terminar()
{
    _dialogoActivo = false;
    _dialogoActual = null;
    CerrarPanel();
    OnDialogoTerminado?.Invoke();
}

private void CerrarPanel()
{
    if (_data != null && _data.panelDialogo != null)
    _data.panelDialogo.SetActive(false);
}

private bool ValidarReferenciasUI()
{
    if (_data.textoDialogo == null)
    {
        Debug.LogError("[DialogoController] textoDialogo no asignado en DialogoUIData.");
        return false;
    }
    return true;
}

#endregion
}
```

JugadorController.cs

138 lineas

Scripts/Modelos/Jugador/JugadorController.cs

Descripcion	Controlador principal del jugador. En Update() lee el input WASD/flechas, construye direccionInput y lo aplica a la velocidad del Rigidbody usando aceleracion/desaceleracion de JugadorData. Actualiza animaciones del Animator y el SpriteRenderer (flip). Respeta puedeMoveirse. Gestiona audio de caminata. Dependencias: JugadorData.
--------------------	--

```

using UnityEngine;
/// <summary>
/// Controlador de movimiento del jugador.
/// Lee input en Update y aplica fisica en FixedUpdate.
/// Requiere: Rigidbody, JugadorData en el mismo GameObject.
/// </summary>
[RequireComponent(typeof(Rigidbody))]
[RequireComponent(typeof(JugadorData))]
public class JugadorController : MonoBehaviour
{
    private JugadorData _data;
    private Rigidbody _rb;
    #region Unity Callbacks
    private void Awake()
    {
        ObtenerReferencias();
        ConfigurarRigidbody();
    }
    private void Update()
    {
        LeerInput();
    }
    private void FixedUpdate()
    {
        if (!_data.puedeMoveirse)
            return;
        AplicarMovimiento();
    }
    #endregion
    #region Inicializacion
    private void ObtenerReferencias()
    {
        _data = GetComponent<JugadorData>();
        _rb = GetComponent<Rigidbody>();
    }
    /// <summary>
    /// Congela rotacion para que la fisica no tumbe al jugador.
    /// </summary>
    private void ConfigurarRigidbody()
    {
        _rb.freezeRotation = true;
    }
    #endregion
    #region Input
    /// <summary>
    /// Detecta WASD y almacena la direccion normalizada en el plano XZ.
    /// Llamado en Update – no afecta la fisica directamente.
    /// </summary>
    private void LeerInput()
    {
        float h = Input.GetAxisRaw("Horizontal");
        float v = Input.GetAxisRaw("Vertical");
        _data.direccionInput = new Vector3(h, 0f, v).normalized;
    }
    #endregion
    #region Fisica
    /// <summary>
    /// Mueve al jugador con aceleracion y desaceleracion suaves,

```

```

    /// respetando la velocidad maxima sin importar los FPS.
    ///
    /// Por que NO usamos AddForce aqui:
    /// AddForce acumula indefinidamente; para limitar la velocidad
    /// hay que cancelar/ajustar la fuerza cada frame, lo que es
    /// equivalente (y mas claro) a manejar la velocidad directamente.
    ///
    /// Por que es frame-rate independent:
    /// Todo el calculo vive en FixedUpdate, que Unity llama a intervalos
    /// fijos (Time.fixedDeltaTime). MoveTowards avanza la misma cantidad
    /// de unidades por segundo sin importar cuantos frames renders haya.
    /// </summary>
private void AplicarMovimiento()
{
    // Velocidad actual solo en el plano XZ (ignoramos Y para no
    // interferir con la gravedad ni con saltos futuros)
    Vector3 velActualXZ = new Vector3(_rb.linearVelocity.x, 0f, _rb.linearVelocity.z);
    Vector3 velObjetivo;
    float tasa;
    if (_data.direccionInput != Vector3.zero)
    {
        // Hay input → acelerar hacia la direccion deseada
        velObjetivo = _data.direccionInput * _data.velocidadMaxima;
        tasa = _data.aceleracion * Time.fixedDeltaTime;
    }
    else
    {
        // Sin input → frenar hasta cero
        velObjetivo = Vector3.zero;
        tasa = _data.desaceleracion * Time.fixedDeltaTime;
    }

    // MoveTowards nunca supera el target ni la tasa: sin overshooting
    Vector3 nuevaVelXZ = Vector3.MoveTowards(velActualXZ, velObjetivo, tasa);
    // Reconponemos con la Y original para respetar gravedad / saltos
    _rb.linearVelocity = new Vector3(nuevaVelXZ.x, _rb.linearVelocity.y, nuevaVelXZ.z);

    bool moviendose = _data.direccionInput != Vector3.zero;
    // Animacion
    _data.animador.Play(moviendose ? _data.animCaminando : _data.animQuieto);
    // Audio
    if (moviendose)
    {
        if (!_data.audioCaminata.isPlaying)
        _data.audioCaminata.Play();
    }
    else
    {
        _data.audioCaminata.Stop();
    }

    // Flip X
    if (_data.direccionInput.x != 0)
    _data.spriteRenderer.flipX = _data.direccionInput.x < 0;
    }
    #endregion
}

```

TriggerCuerpoUI.cs

53 lineas

Scripts/Modelos/Jugador/TriggerCuerpoUI.cs

Descripcion	Trigger que abre el panel CuerpoUI cuando el jugador entra en la zona. Normalmente se coloca en la zona del altar o de mision activa. Al salir, cierra el panel. Delega en CuerpoUIController. Dependencias: CuerpoUIController, JugadorData.
--------------------	---

```

using UnityEngine;

/// <summary>
/// Vive en la Sphere hija del jugador.
/// Al presionar Espacio, avisa a CuerpoUIController que abra el panel.

```



```

/// </summary>
public class TriggerCuerpoUI : MonoBehaviour
{
    private CuerpoUIController _uiController;
    #region Unity Callbacks
    private void Awake()
    {
        ObtenerReferencias();
    }
    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            AbrirUI();
        }
    }
    #endregion
    #region Inicializacion
    private void ObtenerReferencias()
    {
        _uiController = FindFirstObjectByType<CuerpoUIController>();
        if (_uiController == null)
        {
            Debug.LogError("[TriggerCuerpoUI] No se encontro CuerpoUIController en escena.");
        }
    }
    #endregion
    #region Logica
    private void AbrirUI()
    {
        if (_uiController == null)
        {
            Debug.LogWarning("[TriggerCuerpoUI] No hay CuerpoUIController. No se puede abrir el panel.");
            return;
        }
        _uiController.AbrirPanel();
        gameObject.SetActive(false);
    }
    #endregion
}

```

MinijuegoCerebroController.cs

247 líneas

Scripts/Modelos/Minijuego/Cabeza/MinijuegoCerebroController.cs

Descripción	Controlador del minijuego de cabeza (cerebro). Iniciar() activa el panel y asigna listeners de click a los distractores. En Update() mueve los distractores hacia el cerebro, detecta colisiones por distancia y reduce el alpha del cerebro. El jugador presiona espacio para recuperar alpha. Al ganar/perder reporta el resultado via MisionesGlobal. Dependencias: MinijuegoCerebroData, DistractorCerebro.
--------------------	---

```

using UnityEngine;
using System.Collections;

/// <summary>
/// Controlador del minijuego de cerebro.
/// El jugador mantiene el alpha del cerebro con espacio
/// y hace click en los distractores para alejarlos.
/// Sobrevivir N segundos → valor 1. Fallo → valor 0.
/// </summary>
[RequireComponent(typeof(MinijuegoCerebroData))]
public class MinijuegoCerebroController : MonoBehaviour
{
    private MinijuegoCerebroData _data;
    #region Unity Callbacks
    private void Awake()
    {
        _data = GetComponent<MinijuegoCerebroData>();
    }
    private void Update()
    {
        if (!_data.minijuegoActivo) return;
    }
}

```

```

ActualizarTiempo();
ActualizarCerebro();
ActualizarDisstractores();
}
#endregion
#region API Publica
    /// <summary>
    /// Inicializa y abre el minijuego de concentracion del cerebro.
    /// Resetea el estado, configura los distractores y activa el panel.
    /// Llamado desde MinijuegoManager cuando el jugador selecciona la parte Cabeza.
    /// </summary>
    public void Iniciar()
    {
        if (!ValidarReferencias()) return;
        LimpiarEstado();
        ConfigurarDisstractores();
        _data.panelCerebro.SetActive(true);
        _data.minijuegoActivo = true;
        // Alpha inicial del cerebro al maximo
        SetAlphaCerebro(1f);
    }
#endregion
#region Update Logic
    /// <summary>
    /// Incrementa el temporizador y comprueba si el jugador ha sobrevivido el tiempo necesario.
    /// </summary>
    private void ActualizarTiempo()
    {
        _data.tiempoTranscurrido += Time.deltaTime;
        if (_data.tiempoTranscurrido >= _data.tiempoSobrevivir)
        {
            _data.minijuegoActivo = false;
            Debug.Log($"[MinijuegoCerebro] Exito - sobrevivio {_data.tiempoSobrevivir}s");
            StartCoroutine(Terminar(true));
        }
    }

    /// <summary>
    /// Reduce el alpha del cerebro cada frame y lo incrementa si el jugador presiona espacio.
    /// Detecta fallo cuando el alpha llega a cero.
    /// </summary>
    private void ActualizarCerebro()
    {
        // Perder alpha constantemente
        float alphaActual = _data.imagenCerebro.color.a;
        alphaActual -= _data.perdidaAlpha * Time.deltaTime;
        // Espacio rellena alpha
        if (Input.GetKeyDown(KeyCode.Space))
        {
            alphaActual += _data.gananciaAlpha;
            Debug.Log($"[MinijuegoCerebro] Espacio presionado - alpha={Mathf.Clamp01(alphaActual):F2}");
        }
        alphaActual = Mathf.Clamp01(alphaActual);
        SetAlphaCerebro(alphaActual);
        // Si el alpha llega a 0 → fallo
        if (alphaActual <= 0f)
        {
            _data.minijuegoActivo = false;
            Debug.Log($"[MinijuegoCerebro] Fallo - cerebro desvanecido");
            StartCoroutine(Terminar(false));
        }
    }

    /// <summary>
    /// Mueve cada distractor activo hacia el cerebro y detecta colision por distancia.
    /// </summary>
    private void ActualizarDisstractores()
    {
        if (_data.distractores == null) return;

```

```

Vector2 posCerebro = _data.rectCerebro.anchoredPosition;
foreach (DistractorCerebro distractor in _data.distractores)
{
    if (distractor == null || !distractor.activo) continue;
    RectTransform rect = distractor.GetComponent<RectTransform>();
    // Mover hacia el cerebro
    Vector2 direccion = (posCerebro - rect.anchoredPosition).normalized;
    rect.anchoredPosition += direccion * distractor.velocidadActual * Time.deltaTime;
    // Detectar colision por distancia
    float distancia = Vector2.Distance(rect.anchoredPosition, posCerebro);
    if (distancia <= _data.radioColision)
    {
        _data.minijuegoActivo = false;
        Debug.Log($"[MinijuegoCerebro] Fallo - distractor toco el cerebro");
        StartCoroutine(Terminar(false));
        return;
    }
}
}

#endregion
#region Configuracion
/// <summary>
/// Resetea y configura cada distractor con una velocidad aleatoria
/// y les asigna el listener de click para que reboten al ser presionados.
/// </summary>
private void ConfigurarDisrtractores()
{
    if (_data.distractores == null) return;
    foreach (DistractorCerebro distractor in _data.distractores)
    {
        if (distractor == null) continue;
        // SEGURO ADICIONAL: Si por alguna razon la posicion es cero,
        // intentar capturarla de nuevo si sabemos que no deberia ser el centro.
        if (distractor.posicionOriginal == Vector2.zero)
        {
            distractor.posicionOriginal = distractor.GetComponent<RectTransform>().anchoredPosition;
        }
        float velocidad = Random.Range(_data.velocidadMin, _data.velocidadMax);
        distractor.Resetear(velocidad);
        // Configurar boton
        distractor.boton.onClick.RemoveAllListeners();
        DistractorCerebro capturado = distractor;
        distractor.boton.onClick.AddListener(() =>
        capturado.ALClickear(_data.velocidadMin, _data.velocidadMax));
    }
}

#endregion
#region Terminar
/// <summary>
/// Cierra el panel y reporta el resultado a MisionesGlobal con TipoAccion.Cabeza.
/// </summary>
/// <param name="acerto">True si el jugador sobrevivio el tiempo; false si fallo.</param>
private IEnumerator Terminar(bool acerto)
{
    // Desactivar botones de distractores
    if (_data.distractores != null)
    foreach (DistractorCerebro d in _data.distractores)
    if (d != null && d.boton != null)
    d.boton.onClick.RemoveAllListeners();
    yield return new WaitForSeconds(0.5f);
    _data.panelCerebro.SetActive(false);
    int valor = acerto ? 1 : 0;
    Debug.Log($"[MinijuegoCerebro] Resultado → TipoAccion.Cabeza valor={valor}");
    AccionResultado resultado = new AccionResultado(TipoAccion.Cabeza, valor);
    if (MisionesGlobal.Instancia != null)
    MisionesGlobal.Instancia.ReportarResultado(resultado);
}

```

```

else
Debug.LogWarning("[MinijuegoCerebroController] MisionesGlobal no encontrado.");
}
#endregion
#region Helpers
/// <summary>Resetea el temporizador y la bandera de minijuego activo al estado inicial.</summary>
private void LimpiarEstado()
{
_data.tiempoTranscurrido = 0f;
_data.minijuegoActivo = false;
}
/// <summary>
/// Aplica directamente el valor de alpha a la imagen del cerebro.
/// </summary>
/// <param name="alpha">Valor de alpha a asignar (0 = invisible, 1 = opaco).</param>
private void SetAlphaCerebro(float alpha)
{
Color c = _data.imagenCerebro.color;
c.a = alpha;
_data.imagenCerebro.color = c;
}
/// <summary>Verifica que el panel, la imagen del cerebro, el RectTransform y los 4 distractores esten
asignado ...
/// <returns>True si todas las referencias son validas.</returns>
private bool ValidarReferencias()
{
if (_data.panelCerebro == null)
{
Debug.LogError("[MinijuegoCerebroController] panelCerebro no asignado.");
return false;
}
if (_data.imagenCerebro == null)
{
Debug.LogError("[MinijuegoCerebroController] imagenCerebro no asignada.");
return false;
}
if (_data.rectCerebro == null)
{
Debug.LogError("[MinijuegoCerebroController] rectCerebro no asignado.");
return false;
}
if (_data.distractores == null || _data.distractores.Length != 4)
{
Debug.LogError("[MinijuegoCerebroController] Se necesitan exactamente 4 distractores.");
return false;
}
return true;
}
#endregion
}

```

MinijuegoCantoController.cs

348 líneas

Scripts/Modelos/Minijuego/Cantar/MinijuegoCantoController.cs

Descripcion	Controlador del minijuego de canto. Gestiona el pool de notas por carril: en cada intervalo activa una nota aleatoria y la mueve hacia abajo. Al pulsar un boton de carril evalua si alguna nota activa esta dentro del margen de acierto. Cuenta aciertos y fallos; al alcanzar los limites reporta victoria o derrota. Dependencias: MinijuegoCantoData, NotaCanto.
--------------------	---

```

using UnityEngine;
using UnityEngine.UI;
using System.Collections;

/// <summary>
/// Controlador del minijuego de canto.
/// 4 pools independientes por carril. Deteccion por Y mundial.
/// Exito → valor 2. Fallo → valor 0.

```

```

/// </summary>
[RequireComponent(typeof(MinijuegoCantoData))]
public class MinijuegoCantoController : MonoBehaviour
{
    private MinijuegoCantoData _data;
    private Coroutine[] _flashCoroutines = new Coroutine[4];
    #region Unity Callbacks
    private void Awake()
    {
        _data = GetComponent<MinijuegoCantoData>();
    }
    private void Update()
    {
        if (!_data.minijuegoActivo) return;
        MoverNotas();
        ActualizarTimerNota();
    }
    #endregion
    #region API Publica
    /// <summary>
    /// Inicializa y abre el minijuego de canto.
    /// Resetea el estado, desactiva notas previas, configura botones y activa el panel.
    /// Llamado desde MinijuegoManager cuando el jugador elige Cantar en el panel de boca.
    /// </summary>
    public void Iniciar()
    {
        if (!ValidarReferencias()) return;
        LimpiarEstado();
        DesactivarTodasLasNotas();
        ResetearColoresBotones();
        ConfigurarBotones();
        _data.panelCanto.SetActive(true);
        _data.minijuegoActivo = true;
    }
    #endregion
    #region Notas
    /// <summary>
    /// Desplaza todas las notas activas hacia abajo y detecta cuando una nota pasa el margen del boton sin
    ser pre ...
    /// </summary>
    private void MoverNotas()
    {
        for (int carril = 0; carril < 4; carril++)
        {
            NotaCanto[] pool = ObtenerPool(carril);
            if (pool == null) continue;
            float botonY = _data.botones[carril].GetComponent<RectTransform>().position.y;
            foreach (NotaCanto nota in pool)
            {
                if (nota == null || !nota.activa) continue;
                nota.MoverAbajo(Time.deltaTime);
                float notaY = nota.GetComponent<RectTransform>().position.y;
                if (notaY < botonY - _data.margenAcierto)
                {
                    nota.Desactivar();
                    DispararFlashRojo(carril);
                    RegistrarFallo($"nota carril {carril} paso sin presionar");
                }
            }
        }
    }
    /// <summary>Reduce el temporizador y lanza una nueva nota cuando llega a cero, reiniciando el
    intervalo.</summ ...
    private void ActualizarTimerNota()
    {
        _data.timerNota -= Time.deltaTime;
        if (_data.timerNota <= 0f)

```

```

{
    LanzarNota();
    _data.timerNota = _data.intervaloNotas;
}
}

/// <summary>
/// Elige un carril aleatorio, busca una nota inactiva en su pool y la activa con velocidad aleatoria.
/// </summary>
private void LanzarNota()
{
    int carril = Random.Range(0, 4);
    NotaCanto[] pool = ObtenerPool(carril);
    if (pool == null) return;
    foreach (NotaCanto nota in pool)
    {
        if (nota == null || nota.activa) continue;
        float velocidad = Random.Range(_data.velocidadMin, _data.velocidadMax);
        float botonY = _data.botones[carril].GetComponent<RectTransform>().position.y;
        RectTransform rectNota = nota.GetComponent<RectTransform>();
        Vector3 pos = rectNota.position;
        pos.y = botonY + _data.posYInicial;
        rectNota.position = pos;
        nota.Activar(carril, velocidad);
        Debug.Log($"[MinijuegoCanto] Nota lanzada carril={carril} velocidad={velocidad:F1}");
        return;
    }
}

/// <summary>Desactiva todas las notas de todos los carriles para dejar el minijuego en estado
limpio.</summary> ...
private void DesactivarTodasLasNotas()
{
    for (int i = 0; i < 4; i++)
    {
        NotaCanto[] pool = ObtenerPool(i);
        if (pool == null) continue;
        foreach (NotaCanto nota in pool)
        {
            if (nota != null) nota.Desactivar();
        }
    }
}

/// <summary>
/// Devuelve el array de notas del carril especificado.
/// </summary>
/// <param name="carril">Indice del carril (0-3).</param>
/// <returns>Array de NotaCanto del carril o null si el indice es invalido.</returns>
private NotaCanto[] ObtenerPool(int carril)
{
    switch (carril)
    {
        case 0: return _data.notasCarril0;
        case 1: return _data.notasCarril1;
        case 2: return _data.notasCarril2;
        case 3: return _data.notasCarril3;
        default:
            Debug.LogWarning($"[MinijuegoCanto] Carril invalido: {carril}");
            return null;
    }
}

#endregion
#region Botones

/// <summary>Asigna los listeners de click a cada boton de carril segun su indice.</summary>
private void ConfigurarBotones()
{
    LimpiarBotones();
    for (int i = 0; i < _data.botones.Length; i++)
    {
        if (_data.botones[i] == null) continue;
        int carrilCapturado = i;

```

```

_data.botones[i].onClick.AddListener(() => AlPresionarBoton(carrilCapturado));
}
}

/// <summary>
/// Evalua si hay una nota en la zona de acierto del carril presionado.
/// Registra acierto o fallo segun la posicion de la nota.
/// </summary>
/// <param name="carril">Indice del carril cuyo boton fue presionado.</param>
private void AlPresionarBoton(int carril)
{
    if (!_data.minijuegoActivo) return;
    NotaCanto[] pool = ObtenerPool(carril);
    if (pool == null) return;
    float botonY = _data.botones[carril].GetComponent<RectTransform>().position.y;
    foreach (NotaCanto nota in pool)
    {
        if (nota == null || !nota.activa) continue;
        float notaY = nota.GetComponent<RectTransform>().position.y;
        float diferencia = Mathf.Abs(notaY - botonY);
        Debug.Log($"[MinijuegoCanto] Boton {carril} - notaY={notaY:F1} botonY={botonY:F1} dif={diferencia:F1} m ...
        if (diferencia <= _data.margenAcierto)
        {
            nota.Desactivar();
            _data.notasAcertadas++;
            Debug.Log($"[MinijuegoCanto] Acierto carril={carril} ({_data.notasAcertadas}/{_data.notasParaGanar} ...
            if (_data.notasAcertadas >= _data.notasParaGanar)
            {
                _data.minijuegoActivo = false;
                StartCoroutine(Terminar(true));
            }
            return;
        }
    }

    DispararFlashRojo(carril);
    RegistrarFallo($"boton {carril} sin nota en zona");
}

/// <summary>Elimina todos los listeners de los botones de carril.</summary>
private void LimpiarBotones()
{
    if (_data.botones == null) return;
    foreach (var b in _data.botones)
    {
        if (b != null) b.onClick.RemoveAllListeners();
    }
}

/// <summary>Restaura el color blanco de todos los botones de carril al iniciar o terminar el
minijuego.</summa ...
private void ResetearColoresBotones()
{
    for (int i = 0; i < 4; i++)
    {
        if (_data.botones != null && _data.botones.Length > i && _data.botones[i] != null)
        {
            Image img = _data.botones[i].GetComponent<Image>();
            if (img != null) img.color = Color.white;
        }
    }
}

#endregion
#region Flash Rojo

/// <summary>
/// Cancela el flash anterior del carril (si existe) y lanza uno nuevo para indicar error.
/// </summary>
/// <param name="carril">Indice del carril donde ocurrio el fallo.</param>
private void DispararFlashRojo(int carril)
{
    // Cancelar corrutina anterior del mismo carril si existe
    if (_flashCoroutines[carril] != null)
        StopCoroutine(_flashCoroutines[carril]);
}

```

```

_flashCoroutines[carril] = StartCoroutine(FlashRojo(carril));
}
/// <summary>
/// Pone el boton del carril en rojo durante 0.3 segundos y luego lo devuelve a blanco.
/// </summary>
/// <param name="carril">Indice del carril a iluminar en rojo.</param>
private IEnumerator FlashRojo(int carril)
{
if (_data.botones[carril] == null) yield break;
Image img = _data.botones[carril].GetComponent<Image>();
if (img == null) yield break;
img.color = Color.red;
yield return new WaitForSeconds(0.3f);
img.color = Color.white;
_flashCoroutines[carril] = null;
}
#endregion
#region Fallo y Fin
/// <summary>
/// Incrementa el contador de fallos y termina el minijuego si se supero el limite permitido.
/// </summary>
/// <param name="motivo">Descripcion del fallo para el log de depuracion.</param>
private void RegistrarFallo(string motivo)
{
if (!_data.minijuegoActivo) return;
_data.fallosActuales++;
Debug.Log($"[MinijuegoCanto] Fallo {_data.fallosActuales}/{_data.fallosPermitidos} - {motivo}");
if (_data.fallosActuales >= _data.fallosPermitidos)
{
_data.minijuegoActivo = false;
StartCoroutine(Terminar(false));
}
}
/// <summary>
/// Limpia el estado del minijuego, cierra el panel y reporta el resultado a MisionesGlobal con
TipoAccion.Boca ...
/// </summary>
/// <param name="acerto">True si el jugador acierto las notas necesarias; false si supero los
fallos.</param>
private IEnumerator Terminar(bool acerto)
{
LimpiarBotones();
DesactivarTodasLasNotas();
ResetearColoresBotones();
yield return new WaitForSeconds(0.4f);
_data.panelCanto.SetActive(false);
int valor = acerto ? 2 : 0;
Debug.Log($"[MinijuegoCanto] Resultado → TipoAccion.Boca valor={valor}");
AccionResultado resultado = new AccionResultado(TipoAccion.Boca, valor);
if (MisionesGlobal.Instancia != null)
MisionesGlobal.Instancia.ReportarResultado(resultado);
else
Debug.LogWarning("[MinijuegoCantoController] MisionesGlobal no encontrado.");
}
#endregion
#region Helpers
/// <summary>Resetea contadores de notas, fallos, timer y la lista de corrutinas de flash.</summary>
private void LimpiarEstado()
{
_data.notasAcertadas = 0;
_data.fallosActuales = 0;
_data.timerNota = _data.intervaloNotas;
_data.minijuegoActivo = false;
for (int i = 0; i < _flashCoroutines.Length; i++)
_flashCoroutines[i] = null;
}
/// <summary>Verifica que el panel y los 4 botones de carril esten asignados en el Inspector.</summary>

```



```

    /// <returns>True si todas las referencias son validas.</returns>
    private bool ValidarReferencias()
    {
        if (_data.panelCanto == null)
        {
            Debug.LogError("[MinijuegoCantoController] panelCanto no asignado.");
            return false;
        }
        if (_data.botones == null || _data.botones.Length != 4)
        {
            Debug.LogError("[MinijuegoCantoController] Se necesitan exactamente 4 botones.");
            return false;
        }
        return true;
    }
#endregion
}

```

MinijuegoSaludoController.cs

187 líneas

Scripts/Modelos/Minijuego/Mano/MinijuegoSaludoController.cs

Descripción	Controlador del minijuego de saludo de mano. Al iniciar elige aleatoriamente si la mano del otro personaje aparece arriba o abajo, coloca las imágenes en los botones correspondientes y empieza la cuenta regresiva de la barra. Si el jugador presiona el botón correcto dentro del tiempo muestra el éxito. Dependencias: MinijuegoSaludoData.
--------------------	---

```

using UnityEngine;
using System.Collections;

/// <summary>
/// Controlador del minijuego de saludo de mano.
/// Dos botones, uno correcto aleatorio, tiempo limitado con barra visual.
/// Exito → valor 1. Fallo → valor 0.
/// </summary>
[RequireComponent(typeof(MinijuegoSaludoData))]
public class MinijuegoSaludoController : MonoBehaviour
{
    private MinijuegoSaludoData _data;
    private bool _esperandoRespuesta = false;
    private float _tiempoRestante;
#region Unity Callbacks
    private void Awake()
    {
        _data = GetComponent<MinijuegoSaludoData>();
    }

    private void Update()
    {
        if (!_esperandoRespuesta) return;
        _tiempoRestante -= Time.deltaTime;
        ActualizarBarra();
        if (_tiempoRestante <= 0f)
        {
            _esperandoRespuesta = false;
            StartCoroutine(Terminar(false));
        }
    }
#endregion
#region API Publica
    /// <summary>
    /// Inicializa y abre el minijuego de saludo de mano.
    /// Elige aleatoriamente el botón correcto, configura listeners y activa el panel.
    /// Llamado desde MinijuegoManager cuando el jugador elige Saludo en el panel de mano.
    /// </summary>
    public void Iniciar()
    {
        if (!ValidarReferencias()) return;
        LimpiarEstado();
    }

```

```

AsignarManoAleatoria();
ConfigurarBotones();
ResetearBarra();
if (_data.imagenExito != null)
_data.imagenExito.SetActive(false);
_tiempoRestante = _data.tiempoLimite;
_esperandoRespuesta = true;
_data.panelSaludo.SetActive(true);
}
#endregion
#region Barra
/// <summary>Actualiza el ancho de la barra de tiempo proporcional al tiempo restante.</summary>
private void ActualizarBarra()
{
if (_data.barraTiempo == null) return;
float t = Mathf.Clamp01(_tiempoRestante / _data.tiempoLimite);
_data.barraTiempo.sizeDelta = new Vector2(_data.anchoBarraInicial * t, _data.barraTiempo.sizeDelta.y);
}

/// <summary>Restaura la barra de tiempo a su ancho inicial al comenzar el minijuego.</summary>
private void ResetearBarra()
{
if (_data.barraTiempo == null) return;
_data.barraTiempo.sizeDelta = new Vector2(_data.anchoBarraInicial, _data.barraTiempo.sizeDelta.y);
}
#endregion
#region Logica
/// <summary>
/// Elige aleatoriamente cual boton (arriba o abajo) es el correcto y
/// reasigna las imagenes de manos para que la del otro personaje este en el boton correcto.
/// </summary>
private void AsignarManoAleatoria()
{
_data.botonCorrectoEsArriba = Random.value > 0.5f;
Transform padreCorrecto = _data.botonCorrectoEsArriba ? _data.botonArriba.transform :
_data.botonAbajo.trans ...
Transform padreIncorrecto = _data.botonCorrectoEsArriba ? _data.botonAbajo.transform :
_data.botonArriba.tr ...
if (_data.imagenManoOtro != null)
_data.imagenManoOtro.transform.SetParent(padreCorrecto, false);
if (_data.imagenManoJugador != null)
_data.imagenManoJugador.transform.SetParent(padreIncorrecto, false);
}

/// <summary>Asigna los listeners a los botones de arriba y abajo para detectar la eleccion del
jugador.</summa ...
private void ConfigurarBotones()
{
LimpiarBotones();
if (_data.botonArriba != null) _data.botonArriba.onClick.AddListener(() => AlPresionar(true));
if (_data.botonAbajo != null) _data.botonAbajo.onClick.AddListener(() => AlPresionar(false));
}

/// <summary>
/// Compara el boton presionado con el boton correcto aleatorio y lanza la rutina de terminar.
/// </summary>
/// <param name="esArriba">True si el jugador presiono el boton de arriba.</param>
private void AlPresionar(bool esArriba)
{
if (!_esperandoRespuesta) return;
_esperandoRespuesta = false;
StartCoroutine(Terminar(esArriba == _data.botonCorrectoEsArriba));
}

/// <summary>
/// Muestra la imagen de exito si acierta, cierra el panel y reporta el resultado a MisionesGlobal con
TipoAcci ...
/// </summary>
/// <param name="acerto">True si el jugador presiono el boton correcto.</param>
private IEnumerator Terminar(bool acerto)
{

```

```

LimpiarBotones();
if (acerto)
{
    _data.botonArriba.gameObject.SetActive(false);
    _data.botonAbajo.gameObject.SetActive(false);
    if (_data.imagenExito != null) _data.imagenExito.SetActive(true);
    yield return new WaitForSeconds(_data.tiempoExito);
}
_data.panelSaludo.SetActive(false);
ReactivarBotones();
int valor = acerto ? 1 : 0;
Debug.Log($"[MinijuegoSaludo] {(acerto ? "Exito" : "Fallo")} → TipoAccion.Mano valor={valor}");
AccionResultado resultado = new AccionResultado(TipoAccion.Mano, valor);
if (MisionesGlobal.Instancia != null)
MisionesGlobal.Instancia.ReportarResultado(resultado);
else
Debug.LogWarning("[MinijuegoSaludoController] MisionesGlobal no encontrado.");
}
#endregion
#region Helpers
    /// <summary>Resetea la bandera de espera y reactiva ambos botones antes de iniciar.</summary>
    private void LimpiarEstado()
    {
        _esperandoRespuesta = false;
        ReactivarBotones();
    }

    /// <summary>Elimina todos los listeners de los botones arriba y abajo.</summary>
    private void LimpiarBotones()
    {
        if (_data.botonArriba != null) _data.botonArriba.onClick.RemoveAllListeners();
        if (_data.botonAbajo != null) _data.botonAbajo.onClick.RemoveAllListeners();
    }

    /// <summary>Reactiva la visibilidad de ambos botones para el siguiente intento.</summary>
    private void ReactivarBotones()
    {
        if (_data.botonArriba != null) _data.botonArriba.gameObject.SetActive(true);
        if (_data.botonAbajo != null) _data.botonAbajo.gameObject.SetActive(true);
    }

    /// <summary>Verifica que el panel y los dos botones esten asignados en el Inspector.</summary>
    /// <returns>True si todas las referencias son validas.</returns>
    private bool ValidarReferencias()
    {
        if (_data.panelSaludo == null) { Debug.LogError("[MinijuegoSaludoController] panelSaludo no asignado.");
        re ...
        if (_data.botonArriba == null || _data.botonAbajo == null) { Debug.LogError("[MinijuegoSaludoController]
        Bo ...
        return true;
    }
}
#endregion
}

```

MinijuegoSenalController.cs

273 líneas

Scripts/Modelos/Minijuego/Mano/MinijuegoSenalController.cs

Descripcion

Controlador del minijuego de senal de la cruz. En modo simple, activa los 4 botones en orden aleatorio y verifica la secuencia correcta. En modo complejo usa el set extendido de botones. La barra de tiempo decrece continuamente; si se agota sin completar, reporta fallo. Dependencias: MinijuegoSenalData.

```

using UnityEngine;
using UnityEngine.UI;
using System.Collections;

/// <summary>
/// Controlador del minijuego de senal.
/// Modo simple: 4 botones en orden fijo.
/// Modo complejo: mas botones en orden fijo, distintos GameObjects.
/// Barra de tiempo en ambos modos.

```

```

/// Exito → valor 2. Fallo → valor 0.
/// </summary>
[RequireComponent(typeof(MinijuegoSenalData))]
public class MinijuegoSenalController : MonoBehaviour
{
    private MinijuegoSenalData _data;
    private bool _esperandoRespuesta = false;
    private float _tiempoRestante;
    // Botones activos segun el modo
    private Button[] _botonesActivos;
    #region Unity Callbacks
    private void Awake()
    {
        _data = GetComponent<MinijuegoSenalData>();
    }
    private void Update()
    {
        if (!_esperandoRespuesta) return;
        _tiempoRestante -= Time.deltaTime;
        ActualizarBarra();
        if (_tiempoRestante <= 0f)
        {
            _esperandoRespuesta = false;
            StartCoroutine(Terminar(false));
        }
    }
    #endregion
    #region API Publica
    /// <summary>
    /// Inicializa y abre el minijuego de senal de la cruz.
    /// Configura el modo simple o complejo, genera el orden y activa el panel.
    /// Llamado desde MinijuegoManager cuando el jugador elige Senal en el panel de mano.
    /// </summary>
    public void Iniciar()
    {
        if (!ValidarReferencias()) return;
        LimpiarEstado();
        ConfigurarModo();
        ConfigurarBotones();
        MostrarOrdenVisual();
        ResetearBarra();
        _tiempoRestante = _data.tiempoLimite;
        _esperandoRespuesta = true;
        _data.panelSenal.SetActive(true);
    }
    #endregion
    #region Modo
    /// <summary>
    /// Activa los objetos del modo simple o complejo segun la configuracion y
    /// establece el array de botones activos y el orden a seguir.
    /// </summary>
    private void ConfigurarModo()
    {
        if (_data.esCompleja)
        {
            // Activar complejos, desactivar simples
            if (_data.objetosSimple != null)
            foreach (GameObject obj in _data.objetosSimple)
            if (obj != null) obj.SetActive(false);
            if (_data.objetosComplejos != null)
            foreach (GameObject obj in _data.objetosComplejos)
            if (obj != null) obj.SetActive(true);
            _botonesActivos = _data.botonesComplejos;
            _data.ordenAleatorio = GenerarOrden(_botonesActivos.Length);
        }
        else
        {

```

```

    // Activar simples, desactivar complejos
    if (_data.objetosComplejos != null)
    foreach (GameObject obj in _data.objetosComplejos)
    if (obj != null) obj.SetActive(false);
    if (_data.objetosSimple != null)
    foreach (GameObject obj in _data.objetosSimple)
    if (obj != null) obj.SetActive(true);
    _botonesActivos = _data.botones;
    _data.ordenAleatorio = GenerarOrden(_botonesActivos.Length);
}
}

/// <summary>
/// Genera un array de indices en orden secuencial (0, 1, 2, ...) para definir el orden de la senal.
/// </summary>
/// <param name="cantidad">Numero de botones del modo activo.</param>
/// <returns>Array de indices en orden natural.</returns>
private int[] GenerarOrden(int cantidad)
{
    int[] indices = new int[cantidad];
    for (int i = 0; i < cantidad; i++) indices[i] = i;
    return indices;
}
#endregion
#region Barra
    /// <summary>Actualiza el ancho de la barra de tiempo proporcional al tiempo restante.</summary>
    private void ActualizarBarra()
    {
        if (_data.barraTiempo == null) return;
        float t = Mathf.Clamp01(_tiempoRestante / _data.tiempoLimite);
        _data.barraTiempo.sizeDelta = new Vector2(_data.anchoBarraInicial * t, _data.barraTiempo.sizeDelta.y);
    }

    /// <summary>Restaura la barra de tiempo a su ancho inicial al comenzar el minijuego.</summary>
    private void ResetearBarra()
    {
        if (_data.barraTiempo == null) return;
        _data.barraTiempo.sizeDelta = new Vector2(_data.anchoBarraInicial, _data.barraTiempo.sizeDelta.y);
    }
#endregion
#region Logica
    /// <summary>
    /// Escribe en el texto de cada boton el numero de posicion que le corresponde en el orden de la senal.
    /// </summary>
    private void MostrarOrdenVisual()
    {
        for (int i = 0; i < _botonesActivos.Length; i++)
        {
            if (_botonesActivos[i] == null) continue;
            int posicion = 0;
            for (int j = 0; j < _data.ordenAleatorio.Length; j++)
            {
                if (_data.ordenAleatorio[j] == i)
                {
                    posicion = j + 1;
                    break;
                }
            }
        }

        var texto = _botonesActivos[i].GetComponentInChildren<TMPPro.TextMeshProUGUI>();
        if (texto != null) texto.text = posicion.ToString();
    }

    /// <summary>Asigna los listeners de click a cada boton activo del modo configurado.</summary>
    private void ConfigurarBotones()
    {
        LimpiarBotones();
        for (int i = 0; i < _botonesActivos.Length; i++)
        {
            if (_botonesActivos[i] == null) continue;

```

```

_botonesActivos[i].gameObject.SetActive(true);
_botonesActivos[i].interactable = true;
int indiceCapturado = i;
_botonesActivos[i].onClick.AddListener(() => AlPresionar(indiceCapturado));
}
}

/// <summary>
/// Verifica si el boton presionado corresponde al siguiente en el orden correcto.
/// Si es incorrecto termina con fallo; si se completan todos, con exito.
/// </summary>
/// <param name="indiceBoton">Indice del boton que presiono el jugador.</param>
private void AlPresionar(int indiceBoton)
{
    if (!_esperandoRespuesta) return;
    int botonEsperado = _data.ordenAleatorio[_data.indiceActual];
    if (indiceBoton != botonEsperado)
    {
        _esperandoRespuesta = false;
        StartCoroutine(Terminar(false));
        return;
    }
    _botonesActivos[indiceBoton].interactable = false;
    _data.indiceActual++;
    if (_data.indiceActual >= _botonesActivos.Length)
    {
        _esperandoRespuesta = false;
        StartCoroutine(Terminar(true));
    }
}

/// <summary>
/// Limpia los botones, cierra el panel y reporta el resultado a MisionesGlobal con TipoAccion.Mano.
/// </summary>
/// <param name="acerto">True si el jugador completo el orden correcto; false si fallo o se acabo el
tiempo.</p ...
private IEnumerator Terminar(bool acerto)
{
    LimpiarBotones();
    yield return new WaitForSeconds(0.3f);
    _data.panelSenal.SetActive(false);
    int valor = acerto ? 2 : 0;
    Debug.Log($"[MinijuegoSenal] {(acerto ? "Exito" : "Fallo")} → TipoAccion.Mano valor={valor}");
    AccionResultado resultado = new AccionResultado(TipoAccion.Mano, valor);
    if (MisionesGlobal.Instancia != null)
        MisionesGlobal.Instancia.ReportarResultado(resultado);
    else
        Debug.LogWarning("[MinijuegoSenalController] MisionesGlobal no encontrado.");
}

#endregion
#region Helpers

/// <summary>Resetea el indice actual y la bandera de espera antes de iniciar.</summary>
private void LimpiarEstado()
{
    _data.indiceActual = 0;
    _esperandoRespuesta = false;
}

/// <summary>Elimina todos los listeners de los botones activos del modo actual.</summary>
private void LimpiarBotones()
{
    if (_botonesActivos == null) return;
    foreach (Button b in _botonesActivos)
    {
        if (b != null) b.onClick.RemoveAllListeners();
    }
}

/// <summary>Verifica que el panel y los botones del modo configurado esten asignados en el
Inspector.</summary ...
/// <returns>True si todas las referencias son validas.</returns>
private bool ValidarReferencias()
{

```

```

if (_data.panelSenal == null)
{
    Debug.LogError("[MinijuegoSenalController] panelSenal no asignado.");
    return false;
}
if (!_data.esCompleja && (_data.botones == null || _data.botones.Length == 0))
{
    Debug.LogError("[MinijuegoSenalController] botones simples no asignados.");
    return false;
}
if (_data.esCompleja && (_data.botonesComplejos == null || _data.botonesComplejos.Length == 0))
{
    Debug.LogError("[MinijuegoSenalController] botonesComplejos no asignados.");
    return false;
}
return true;
}
#endregion
}

```

PanelManoController.cs

100 líneas

Scripts/Modelos/Minijuego/Mano/PanelManoController.cs

Descripción	Controlador del panel de selección de la mano. Al abrir el panel registra listeners en botonSaludo y botonSenal. Al pulsar dispara el evento estático correspondiente (OnSaludoSeleccionado / OnSenalSeleccionada) que MinijuegoManager escucha. Dependencias: PanelManoData, MinijuegoManager.
--------------------	---

```

using UnityEngine;

/// <summary>
/// Controlador del panel intermedio de la mano.
/// Muestra dos opciones al jugador: Saludo o Senal.
/// </summary>
[RequireComponent(typeof(PanelManoData))]
public class PanelManoController : MonoBehaviour
{
    private PanelManoData _data;

    /// <summary>Se dispara cuando el jugador elige la opción Saludo en el panel de mano.</summary>
    public static event System.Action OnSaludoSeleccionado;
    /// <summary>Se dispara cuando el jugador elige la opción Senal en el panel de mano.</summary>
    public static event System.Action OnSenalSeleccionada;

    #region Unity Callbacks
    private void Awake()
    {
        _data = GetComponent<PanelManoData>();
    }

    private void Start()
    {
        CerrarPanel();
    }

    #endregion
    #region API Pública
    /// <summary>
    /// Abre el panel de mano y configura los botones de Saludo y Senal.
    /// Llamado por MinijuegoManager cuando el jugador selecciona la parte ManoDerecha.
    /// </summary>
    public void AbrirPanel()
    {
        if (_data.panelMano == null)
        {
            Debug.LogError("[PanelManoController] panelMano no asignado.");
            return;
        }
        ConfigurarBotones();
        _data.panelMano.SetActive(true);
    }
}

```

```

    /// <summary>
    /// Cierra el panel de mano y limpia los listeners de los botones.
    /// </summary>
    public void CerrarPanel()
    {
        if (_data.panelMano == null) return;
        _data.panelMano.SetActive(false);
        LimpiarBotones();
    }
#endregion
#region Botones
    /// <summary>Asigna los listeners a los botones de Saludo y Senal.</summary>
    private void ConfigurarBotones()
    {
        LimpiarBotones();
        if (_data.botonSaludo != null)
            _data.botonSaludo.onClick.AddListener(AlSeleccionarSaludo);
        if (_data.botonSenal != null)
            _data.botonSenal.onClick.AddListener(AlSeleccionarSenal);
    }

    /// <summary>Elimina todos los listeners de los botones de Saludo y Senal.</summary>
    private void LimpiarBotones()
    {
        if (_data.botonSaludo != null)
            _data.botonSaludo.onClick.RemoveAllListeners();
        if (_data.botonSenal != null)
            _data.botonSenal.onClick.RemoveAllListeners();
    }

    /// <summary>Cierra el panel y dispara el evento OnSaludoSeleccionado.</summary>
    private void AlSeleccionarSaludo()
    {
        CerrarPanel();
        OnSaludoSeleccionado?.Invoke();
    }

    /// <summary>Cierra el panel y dispara el evento OnSenalSeleccionada.</summary>
    private void AlSeleccionarSenal()
    {
        CerrarPanel();
        OnSenalSeleccionada?.Invoke();
    }
#endregion
}

```

MinijuegoPechoController.cs

188 líneas

Scripts/Modelos/Minijuego/Pecho/MinijuegoPechoController.cs

Descripción	Controlador del minijuego de ritmo del pecho. Cada ronda inicia el círculo externo en tamañoInicial y lo reduce cada frame a velocidadActual. Al pulsar el botón del corazón compara tamañoActual con tamañoCorazón ± margenAcierto. Acierto: siguiente ronda. Fallo: penalización. Al terminar todas las rondas reporta el resultado. Dependencias: MinijuegoPechoData.
--------------------	--

```

using UnityEngine;
using System.Collections;

/// <summary>
/// Controlador del minijuego de ritmo del pecho.
/// Un círculo externo se achica hacia el corazón.
/// El jugador debe hacer click cuando coincidan.
/// Exito N veces → valor 1. Fallo → valor 0.
/// </summary>
[RequireComponent(typeof(MinijuegoPechoData))]
public class MinijuegoPechoController : MonoBehaviour
{
    private MinijuegoPechoData _data;
#region Unity Callbacks
    private void Awake()
    {

```



```

_data = GetComponent<MinijuegoPechoData>();
}
private void Update()
{
    if (!_data.esperandoClick) return;
    // Achicar el circulo
    _data.tamanoActual -= _data.velocidadActual * Time.deltaTime;
    // Aplicar tamano al circulo externo
    _data.circuloExterno.sizeDelta = new Vector2(_data.tamanoActual, _data.tamanoActual);
    // Si el circulo paso el tamano minimo sin click → fallo
    if (_data.tamanoActual <= _data.tamanoCorazon - _data.margenAcierto)
    {
        _data.esperandoClick = false;
        Debug.Log("[MinijuegoPecho] Fallo - click tardio o no presionado");
        StartCoroutine(Terminar(false));
    }
}
#endregion
#region API Publica
    /// <summary>
    /// Inicializa y abre el minijuego de ritmo del pecho.
    /// Resetea el estado, configura el boton del corazon e inicia la primera ronda.
    /// Llamado desde MinijuegoManager cuando el jugador selecciona la parte Pecho.
    /// </summary>
    public void Iniciar()
    {
        if (!ValidarReferencias()) return;
        LimpiarEstado();
        _data.panelPecho.SetActive(true);
        _data.botonCorazon.onClick.RemoveAllListeners();
        _data.botonCorazon.onClick.AddListener(AlClickearCorazon);
        IniciarRonda();
    }
#endregion
#region Logica de Ronda
    /// <summary>
    /// Configura el circulo externo al tamano inicial con velocidad aleatoria y habilita el click.
    /// </summary>
    private void IniciarRonda()
    {
        _data.tamanoActual = _data.tamanoInicial;
        _data.velocidadActual = Random.Range(_data.velocidadMin, _data.velocidadMax);
        _data.esperandoClick = true;
        _data.circuloExterno.sizeDelta = new Vector2(_data.tamanoActual, _data.tamanoActual);
        Debug.Log($"[MinijuegoPecho] Ronda {_data.rondaActual + 1}/{_data.rondasTotales} " +
            $"- velocidad={_data.velocidadActual:F1}");
    }
    /// <summary>
    /// Evalua si el click se realizo dentro del margen de tolerancia alrededor del tamano del corazon.
    /// Avanza a la siguiente ronda o termina segun el resultado.
    /// </summary>
    private void AlClickearCorazon()
    {
        if (!_data.esperandoClick) return;
        float diferencia = Mathf.Abs(_data.tamanoActual - _data.tamanoCorazon);
        if (diferencia <= _data.margenAcierto)
        {
            // Acierto
            _data.esperandoClick = false;
            _data.rondaActual++;
            Debug.Log($"[MinijuegoPecho] Acierto ronda {_data.rondaActual}/{_data.rondasTotales} " +
                $"- diferencia={diferencia:F1}");
            if (_data.rondaActual >= _data.rondasTotales)
            {
                StartCoroutine(Terminar(true));
            }
        }
    }
}

```

```

else
{
    // Pequeña pausa antes de la siguiente ronda
    StartCoroutine(PausaEntreRondas());
}
}
else
{
    // Click fuera del margen
    _data.esperandoClick = false;
    Debug.Log($"[MinijuegoPecho] Fallo - diferencia={diferencia:F1} fuera del margen={_data.margenAcierto}" ...
    StartCoroutine(Terminar(false));
}
}

/// <summary>Espera 0.3 segundos entre rondas antes de iniciar la siguiente.</summary>
private IEnumerator PausaEntreRondas()
{
    yield return new WaitForSeconds(0.3f);
    IniciarRonda();
}

/// <summary>
/// Cierra el panel y reporta el resultado a MisionesGlobal con TipoAccion.Pecho.
/// </summary>
/// <param name="acerto">True si el jugador completo todas las rondas correctamente.</param>
private IEnumerator Terminar(bool acerto)
{
    yield return new WaitForSeconds(0.5f);
    _data.panelPecho.SetActive(false);
    _data.botonCorazon.onClick.RemoveAllListeners();
    int valor = acerto ? 1 : 0;
    string descripcion = acerto ? "Exito - todas las rondas completadas" : "Fallo";
    Debug.Log($"[MinijuegoPecho] {descripcion} → TipoAccion.Pecho valor={valor}");
    AccionResultado resultado = new AccionResultado(TipoAccion.Pecho, valor);
    if (MisionesGlobal.Instancia != null)
        MisionesGlobal.Instancia.ReportarResultado(resultado);
    else
        Debug.LogWarning("[MinijuegoPechoController] MisionesGlobal no encontrado.");
}

#endregion
#region Helpers

/// <summary>Resetea la ronda actual, el tamaño del círculo y la bandera de espera de click.</summary>
private void LimpiarEstado()
{
    _data.rondaActual = 0;
    _data.esperandoClick = false;
    _data.tamanoActual = _data.tamanoInicial;
    if (_data.circuloExterno != null)
        _data.circuloExterno.sizeDelta = new Vector2(_data.tamanoInicial, _data.tamanoInicial);
}

/// <summary>Verifica que el panel, el círculo externo y el botón del corazón estén asignados.</summary>
/// <returns>True si todas las referencias son válidas.</returns>
private bool ValidarReferencias()
{
    {
        if (_data.panelPecho == null)
        {
            Debug.LogError("[MinijuegoPechoController] panelPecho no asignado.");
            return false;
        }
    }
    if (_data.circuloExterno == null)
    {
        Debug.LogError("[MinijuegoPechoController] circuloExterno no asignado.");
        return false;
    }
    if (_data.botonCorazon == null)
    {
        Debug.LogError("[MinijuegoPechoController] botonCorazon no asignado.");
        return false;
    }
}

```

```

}
return true;
}
#endregion
}

```

MinijuegoPiernaController.cs

394 líneas

Scripts/Modelos/Minijuego/Pierna/MinijuegoPiernaController.cs

Descripción	Controlador del minijuego de ensamblaje de pierna. Genera un orden aleatorio de Cadera→Rodilla→Pie al iniciar y muestra el objetivo en el panel lateral. El jugador hace click en celdas para colocar las partes en el orden correcto. Al colocar la parte equivocada limpia la celda. Al completar el orden correcto reporta victoria. Dependencias: MinijuegoPiernaData, CeldaCuadrículaData.
--------------------	---

```

using System.Collections;
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// Controlador del minijuego de pierna.
/// Genera el orden aleatorio, gestiona los clicks en la cuadrícula,
/// actualiza las conexiones visuales y reporta el resultado a MisionesGlobal.
/// </summary>
[RequireComponent(typeof(MinijuegoPiernaData))]
public class MinijuegoPiernaController : MonoBehaviour
{
    private MinijuegoPiernaData _data;
    private RectTransform _posicionCadera;
    private RectTransform _posicionRodilla;
    private RectTransform _posicionPie;
    #region Unity Callbacks
    private void Awake()
    {
        _data = GetComponent<MinijuegoPiernaData>();
    }
    #endregion
    #region API Publica
    /// <summary>
    /// Inicializa y abre el minijuego.
    /// Llamado desde CuerpoUIController cuando el jugador selecciona Piernas.
    /// </summary>
    public void Iniciar()
    {
        if (!ValidarReferencias())
            return;
        LimpiarEstado();
        GenerarOrdenAleatorio();
        ConfigurarImagenesOrden();
        ConfigurarBotonesCeldas();
        OcultarConexiones();
        _data.panelMinijuego.SetActive(true);
    }
    #endregion
    #region Inicializacion
    /// <summary>Verifica que el panel y el array de 9 celdas estén asignados en el Inspector.</summary>
    /// <returns>True si las referencias son válidas; false si falta alguna.</returns>
    private bool ValidarReferencias()
    {
        if (_data.panelMinijuego == null)
        {
            Debug.LogError("[MinijuegoPiernaController] panelMinijuego no asignado.");
            return false;
        }
        if (_data.celdas == null || _data.celdas.Length != 9)
        {
            Debug.LogError("[MinijuegoPiernaController] Se necesitan exactamente 9 celdas.");
            return false;
        }
    }
    #endregion

```

```

}
return true;
}

/// <summary>
/// Limpia el estado de celdas y variables del turno anterior.
/// </summary>
private void LimpiarEstado()
{
    _data.turnoActual = 0;
    _posicionCadera = null;
    _posicionRodilla = null;
    _posicionPie = null;
    foreach (CeldaCuadrículaData celda in _data.celdas)
    {
        if (celda != null)
        {
            celda.Limpiar();
            // Agregar estas dos líneas:
            if (celda.boton != null)
            {
                celda.boton.gameObject.SetActive(true);
                celda.boton.interactable = true;
            }
        }
    }
}

/// <summary>
/// Mezcla las 3 partes en orden aleatorio usando Fisher-Yates.
/// </summary>
private void GenerarOrdenAleatorio()
{
    PartePierna[] partes = { PartePierna.Cadera, PartePierna.Rodilla, PartePierna.Pie };
    for (int i = partes.Length - 1; i > 0; i--)
    {
        int j = Random.Range(0, i + 1);
        PartePierna temp = partes[i];
        partes[i] = partes[j];
        partes[j] = temp;
    }
    _data.ordenAleatorio = partes;
}

/// <summary>
/// Asigna los sprites al panel de orden y los hace visibles.
/// </summary>
private void ConfigurarImagenesOrden()
{
    for (int i = 0; i < 3; i++)
    {
        UnityEngine.UI.Image img = _data.ObtenerImagenOrden(i);
        if (img == null) continue;
        Sprite sprite = _data.ObtenerSprite(_data.ordenAleatorio[i]);
        img.sprite = sprite;
        img.gameObject.SetActive(true);
        // Transparencia completa = visible
        Color c = img.color;
        c.a = 1f;
        img.color = c;
    }
}

/// <summary>
/// Asigna listeners a los botones de cada celda.
/// </summary>
private void ConfigurarBotonesCeldas()
{
    foreach (CeldaCuadrículaData celda in _data.celdas)
    {
        if (celda == null || celda.boton == null) continue;
    }
}

```

```

celda.boton.onClick.RemoveAllListeners();
// Captura local para el lambda
CeldaCuadrículaData celdaCapturada = celda;
celda.boton.onClick.AddListener(() => AlClickearCelda(celdaCapturada));
}
}

/// <summary>Desactiva las imágenes de línea entre partes al iniciar para que no se muestren de
antemano.</summary>
private void OcultarConexiones()
{
    if (_data.lineaCaderaRodilla != null)
        _data.lineaCaderaRodilla.gameObject.SetActive(false);
    if (_data.lineaRodillaPie != null)
        _data.lineaRodillaPie.gameObject.SetActive(false);
}
#endregion
#region Lógica de Clicks
/// <summary>
/// Gestiona el click en una celda: coloca la parte en turno, desvanece su imagen de orden,
/// guarda su posición para conexiones visuales y avanza el turno.
/// </summary>
/// <param name="celda">Celda que el jugador acaba de seleccionar.</param>
private void AlClickearCelda(CeldaCuadrículaData celda)
{
    if (celda == null) return;
    if (celda.EstaOcupada) return;
    if (_data.turnoActual >= 3) return;
    PartePierna parteActual = _data.ordenAleatorio[_data.turnoActual];
    // Colocar parte en la celda
    ColocarParteEnCelda(celda, parteActual);
    // Desvanecer imagen del panel de orden
    DesvanecerImagenOrden(_data.turnoActual);
    // Guardar posición para conexiones
    GuardarPosicion(parteActual, celda.GetComponent<RectTransform>());
    // Actualizar conexiones visuales
    ActualizarConexiones();
    _data.turnoActual++;
    // Si ya se colocaron las 3 partes, esperar y reportar
    if (_data.turnoActual >= 3)
        StartCoroutine(EsperarYReportar());
}

/// <summary>
/// Asigna la parte a la celda y muestra el sprite correspondiente en la imagen de la celda.
/// </summary>
/// <param name="celda">Celda donde se coloca la parte.</param>
/// <param name="parte">Parte de la pierna a colocar.</param>
private void ColocarParteEnCelda(CeldaCuadrículaData celda, PartePierna parte)
{
    celda.parteOcupante = parte;
    if (celda.imagenParte != null)
    {
        celda.imagenParte.sprite = _data.ObtenerSprite(parte);
        celda.imagenParte.gameObject.SetActive(true);
    }
}

/// <summary>
/// Reduce el alpha de la imagen del panel de orden al 25% para indicar que esa parte ya fue colocada.
/// </summary>
/// <param name="indice">Índice del slot de orden (0-2) a desvanecer.</param>
private void DesvanecerImagenOrden(int indice)
{
    UnityEngine.UI.Image img = _data.ObtenerImagenOrden(indice);
    if (img == null) return;
    Color c = img.color;
    c.a = 0.25f;
    img.color = c;
}

```

```

    /// <summary>
    /// Guarda el RectTransform de la celda donde se coloco cada parte
    /// para poder calcular las conexiones visuales entre ellas.
    /// </summary>
    /// <param name="parte">Parte de la pierna que se acaba de colocar.</param>
    /// <param name="rect">RectTransform de la celda donde fue colocada.</param>
    private void GuardarPosicion(PartePierna parte, RectTransform rect)
    {
    switch (parte)
    {
    case PartePierna.Cadera: _posicionCadera = rect; break;
    case PartePierna.Rodilla: _posicionRodilla = rect; break;
    case PartePierna.Pie: _posicionPie = rect; break;
    }
    }
    #endregion
    #region Conexiones Visuales
    /// <summary>
    /// Actualiza posicion y visibilidad de las lineas de conexion
    /// segun que partes ya han sido colocadas.
    /// Usa el punto medio entre dos RectTransform como posicion de la linea
    /// y rota la imagen para apuntar de uno al otro.
    /// </summary>
    private void ActualizarConexiones()
    {
    if (_posicionCadera != null && _posicionRodilla != null)
    ActualizarLinea(_data.lineaCaderaRodilla, _posicionCadera, _posicionRodilla);
    if (_posicionRodilla != null && _posicionPie != null)
    ActualizarLinea(_data.lineaRodillaPie, _posicionRodilla, _posicionPie);
    }

    /// <summary>
    /// Posiciona, rota y redimensiona la imagen de linea para que conecte visualmente dos RectTransforms.
    /// </summary>
    /// <param name="linea">Imagen de linea a actualizar.</param>
    /// <param name="desde">RectTransform de la parte origen.</param>
    /// <param name="hasta">RectTransform de la parte destino.</param>
    private void ActualizarLinea(UnityEngine.UI.Image linea, RectTransform desde, RectTransform hasta)
    {
    if (linea == null || desde == null || hasta == null) return;
    linea.gameObject.SetActive(true);
    RectTransform lineaRect = linea.GetComponent<RectTransform>();
    RectTransform parentRect = lineaRect.parent as RectTransform;
    // Convertir posiciones al espacio local del parent directamente
    Vector2 posDesde = parentRect.InverseTransformPoint(desde.position);
    Vector2 posHasta = parentRect.InverseTransformPoint(hasta.position);
    lineaRect.pivot = new Vector2(0.5f, 0.5f);
    lineaRect.anchoredPosition = (posDesde + posHasta) / 2f;
    Vector2 direccion = posHasta - posDesde;
    float angulo = Mathf.Atan2(direccion.y, direccion.x) * Mathf.Rad2Deg;
    lineaRect.localRotation = Quaternion.Euler(0f, 0f, angulo);
    float distancia = direccion.magnitude;
    lineaRect.localScale = Vector3.one;
    lineaRect.sizeDelta = new Vector2(distancia, 60f);
    }
    #endregion
    #region Reporte
    /// <summary>
    /// Espera el tiempo configurado, cierra el panel y reporta el resultado de postura a MisionesGlobal.
    /// </summary>
    private IEnumerator EsperarYReportar()
    {
    DesactivarBotones();
    yield return new WaitForSeconds(_data.tiempoEspera);
    _data.panelMinijuego.SetActive(false);
    int valorPostura = EvaluarPostura();
    AccionResultado resultado = new AccionResultado(TipoAccion.Postura, valorPostura);

```

```

if (MisionesGlobal.Instancia != null)
MisionesGlobal.Instancia.ReportarResultado(resultado);
else
Debug.LogWarning("[MinijuegoPiernaController] MisionesGlobal no encontrado al reportar.");
}

/// <summary>
/// Desactiva la interaccion de todos los botones de celda sin opacar las imagenes ya colocadas.
/// </summary>
private void DesactivarBotones()
{
foreach (CeldaCuadrriculaData celda in _data.celdas)
{
if (celda != null && celda.boton != null)
{
celda.boton.onClick.RemoveAllListeners();
// Mantener color normal para no opacar las imagenes colocadas
ColorBlock colores = celda.boton.colors;
colores.disabledColor = Color.white;
celda.boton.colors = colores;
celda.boton.interactable = false;
}
}
}

#endregion
#region Reporte
/// <summary>
/// Evalua la disposicion de las partes de la pierna en la cuadrricula
/// y devuelve el codigo de postura: 1=parado, 2=sentado, 3=arrodillado, 0=no reconocida.
/// </summary>
/// <returns>Codigo de postura reconocida o 0 si no coincide con ninguna valida.</returns>
private int EvaluarPostura()
{
// Obtener fila y columna de cada parte
int cadFila = -1, cadCol = -1;
int rodFila = -1, rodCol = -1;
int pieFila = -1, pieCol = -1;
foreach (CeldaCuadrriculaData celda in _data.celdas)
{
if (!celda.EstaOcupada) continue;
switch (celda.parteOcupante.Value)
{
case PartePierna.Cadera: cadFila = celda.fila; cadCol = celda.columna; break;
case PartePierna.Rodilla: rodFila = celda.fila; rodCol = celda.columna; break;
case PartePierna.Pie: pieFila = celda.fila; pieCol = celda.columna; break;
}
}

// 1 - Parado: misma columna, cadera < rodilla < pie en fila
if (cadCol == rodCol && rodCol == pieCol &&
cadFila < rodFila && rodFila < pieFila)
{
Debug.Log("[Postura] Parado (1)");
return 1;
}

// 2 - Sentado: pie y rodilla misma columna, rodilla arriba del pie,
// cadera misma fila que rodilla distinta columna
if (pieCol == rodCol && rodFila < pieFila &&
cadFila == rodFila && cadCol != rodCol)
{
Debug.Log("[Postura] Sentado (2)");
return 2;
}

// 3 - Arrodillado: cadera y rodilla misma columna, cadera arriba,
// pie misma fila que rodilla distinta columna
if (cadCol == rodCol && cadFila < rodFila &&
pieFila == rodFila && pieCol != rodCol)
{
Debug.Log("[Postura] Arrodillado (3)");
}
}

```

```

return 3;
}
Debug.Log("[Postura] No reconocida (0)");
return 0;
}
#endregion
}

```

MinijuegoResponderController.cs

220 líneas

Scripts/Modelos/Minijuego/boca/MinijuegoResponderController.cs

Descripción	Controlador del minijuego de preguntas. Carga los textos de los botones desde MinijuegoResponderData, inicia la barra de tiempo y registra listeners en los tres botones. Presionar espacio reduce el alpha de imagenAyuda gradualmente. Al responder evalúa índiceRespuestaCorrecta y reporta el resultado. Dependencias: MinijuegoResponderData.
--------------------	--

```

using UnityEngine;
using System.Collections;

/// <summary>
/// Controlador del minijuego de responder.
/// 3 botones con respuesta correcta definida por int.
/// Espacio transparente ayuda pero acelera el tiempo.
/// Exito → valor 1. Fallo → valor 0.
/// </summary>
[RequireComponent(typeof(MinijuegoResponderData))]
public class MinijuegoResponderController : MonoBehaviour
{
    private MinijuegoResponderData _data;
    #region Unity Callbacks
    private void Awake()
    {
        _data = GetComponent<MinijuegoResponderData>();
    }
    private void Update()
    {
        if (!_data.esperandoRespuesta) return;
        ActualizarBarra();
        DetectarAyuda();
    }
    #endregion
    #region API Publica
    /// <summary>
    /// Inicializa y abre el minijuego de responder preguntas sobre la misa.
    /// Carga el contenido de la pregunta, configura botones y activa el panel.
    /// Llamado desde MinijuegoManager cuando el jugador elige Responder en el panel de boca.
    /// </summary>
    public void Iniciar()
    {
        if (!ValidarReferencias()) return;
        LimpiarEstado();
        ConfigurarContenido();
        ConfigurarBotones();
        ResetearAyuda();
        _data.panelResponder.SetActive(true);
        _data.esperandoRespuesta = true;
    }
    #endregion
    #region Update
    /// <summary>
    /// Reduce el tiempo restante aplicando el multiplicador de velocidad y actualiza el ancho de la barra.
    /// Si el tiempo se agota, termina con fallo.
    /// </summary>
    private void ActualizarBarra()
    {
        _data.tiempoRestante -= Time.deltaTime * _data.multiplicadorVelocidad;
    }

```



```

float t = Mathf.Clamp01(_data.tiempoRestante / _data.tiempoLimite);
float nuevoAncho = _data.anchoBarraInicial * t;
_data.barraTiempo.sizeDelta = new Vector2(nuevoAncho, _data.barraTiempo.sizeDelta.y);
if (_data.tiempoRestante <= 0f)
{
    _data.esperandoRespuesta = false;
    Debug.Log("[MinijuegoResponder] Fallo - tiempo agotado");
    StartCoroutine(Terminar(false));
}
}

/// <summary>
/// Detecta si el jugador presiona espacio para revelar parcialmente la ayuda
/// y acelera la barra como penalización.
/// </summary>
private void DetectarAyuda()
{
    if (Input.GetKeyDown(KeyCode.Space))
    {
        // Transparentar imagen de ayuda
        if (_data.imagenAyuda != null)
        {
            Color c = _data.imagenAyuda.color;
            c.a = Mathf.Max(0f, c.a - _data.alphaAyudaPorEspacio);
            _data.imagenAyuda.color = c;
        }

        // Acelerar barra
        _data.multiplicadorVelocidad *= _data.aceleracionBarraPorEspacio;
        Debug.Log($"[MinijuegoResponder] Ayuda usada - " +
            $"alphaAyuda={_data.imagenAyuda?.color.a:F2} " +
            $"multiplicador={_data.multiplicadorVelocidad:F2}");
    }
}

#endregion
#region Configuración

/// <summary>Aplica el texto de los botones y el texto de ayuda al estado actual del Data.</summary>
private void ConfigurarContenido()
{
    if (_data.textoBoton0 != null) _data.textoBoton0.text = _data.textoBotones[0];
    if (_data.textoBoton1 != null) _data.textoBoton1.text = _data.textoBotones[1];
    if (_data.textoBoton2 != null) _data.textoBoton2.text = _data.textoBotones[2];
    if (_data.textoAyuda != null)
        _data.textoAyuda.text = _data.textoAyudaContenido;
}

/// <summary>Asigna los listeners de click a los tres botones de respuesta.</summary>
private void ConfigurarBotones()
{
    LimpiarBotones();
    if (_data.boton0 != null) _data.boton0.onClick.AddListener(() => AlResponder(0));
    if (_data.boton1 != null) _data.boton1.onClick.AddListener(() => AlResponder(1));
    if (_data.boton2 != null) _data.boton2.onClick.AddListener(() => AlResponder(2));
}

/// <summary>Restaura el alpha de la imagen de ayuda a 1 (completamente opaca) al iniciar.</summary>
private void ResetearAyuda()
{
    if (_data.imagenAyuda != null)
    {
        Color c = _data.imagenAyuda.color;
        c.a = 1f;
        _data.imagenAyuda.color = c;
    }
}

#endregion
#region Lógica

/// <summary>
/// Compara el botón presionado con el índice de respuesta correcta y termina el minijuego.
/// </summary>
/// <param name="indice">Índice del botón presionado (0, 1 o 2).</param>

```

```

private void AlResponder(int indice)
{
    if (!_data.esperandoRespuesta) return;
    _data.esperandoRespuesta = false;
    bool correcto = indice == _data.indiceRespuestaCorrecta
    && _data.indiceRespuestaCorrecta >= 0
    && _data.indiceRespuestaCorrecta <= 2;
    Debug.Log($"[MinijuegoResponder] Boton {indice} presionado - " +
    $"correcto={correcto}");
    StartCoroutine(Terminar(correcto));
}

/// <summary>
/// Cierra el panel y reporta el resultado a MisionesGlobal con TipoAccion.Boca.
/// </summary>
/// <param name="acerto">True si el jugador eligio la respuesta correcta.</param>
private IEnumerator Terminar(bool acerto)
{
    LimpiarBotones();
    yield return new WaitForSeconds(0.4f);
    _data.panelResponder.SetActive(false);
    int valor = acerto ? 1 : 0;
    Debug.Log($"[MinijuegoResponder] Resultado → TipoAccion.Boca valor={valor}");
    AccionResultado resultado = new AccionResultado(TipoAccion.Boca, valor);
    if (MisionesGlobal.Instancia != null)
        MisionesGlobal.Instancia.ReportarResultado(resultado);
    else
        Debug.LogWarning("[MinijuegoResponderController] MisionesGlobal no encontrado.");
}

#endregion
#region Helpers

/// <summary>Resetea el tiempo restante, el multiplicador de velocidad y la barra al ancho
inicial.</summary>
private void LimpiarEstado()
{
    _data.tiempoRestante = _data.tiempoLimite;
    _data.multiplicadorVelocidad = 1f;
    _data.esperandoRespuesta = false;
    if (_data.barraTiempo != null)
        _data.barraTiempo.sizeDelta = new Vector2(_data.anchoBarraInicial, _data.barraTiempo.sizeDelta.y);
}

/// <summary>Elimina todos los listeners de los tres botones de respuesta.</summary>
private void LimpiarBotones()
{
    if (_data.boton0 != null) _data.boton0.onClick.RemoveAllListeners();
    if (_data.boton1 != null) _data.boton1.onClick.RemoveAllListeners();
    if (_data.boton2 != null) _data.boton2.onClick.RemoveAllListeners();
}

/// <summary>Verifica que el panel y la barra de tiempo esten asignados en el Inspector.</summary>
/// <returns>True si todas las referencias son validas.</returns>
private bool ValidarReferencias()
{
    if (_data.panelResponder == null)
    {
        Debug.LogError("[MinijuegoResponderController] panelResponder no asignado.");
        return false;
    }
    if (_data.barraTiempo == null)
    {
        Debug.LogError("[MinijuegoResponderController] barraTiempo no asignada.");
        return false;
    }
    return true;
}

#endregion
}

```

PanelBocaController.cs

60 líneas

Scripts/Modelos/Minijuego/boca/PanelBocaController.cs

Descripción	Controlador del panel de selección de la boca. Muestra dos opciones al jugador: Responder o Cantar. Al pulsar, dispara el evento estático correspondiente (OnResponderSeleccionado / OnCantarSeleccionado) que MinijuegoManager escucha. Dependencias: PanelBocaData, MinijuegoManager.
--------------------	---

```

using UnityEngine;

/// <summary>
/// Controlador del panel intermedio de la boca.
/// </summary>
[RequireComponent(typeof(PanelBocaData))]
public class PanelBocaController : MonoBehaviour
{
    private PanelBocaData _data;

    /// <summary>Se dispara cuando el jugador elige la opción Responder en el panel de boca.</summary>
    public static event System.Action OnResponderSeleccionado;

    /// <summary>Se dispara cuando el jugador elige la opción Cantar en el panel de boca.</summary>
    public static event System.Action OnCantarSeleccionado;

    private void Awake()
    {
        _data = GetComponent<PanelBocaData>();
    }

    private void Start()
    {
        CerrarPanel();
    }

    /// <summary>
    /// Abre el panel de boca y configura los botones de Responder y Cantar.
    /// Llamado por MinijuegoManager cuando el jugador selecciona la parte Boca.
    /// </summary>
    public void AbrirPanel()
    {
        if (_data.panelBoca == null) { Debug.LogError("[PanelBocaController] panelBoca no asignado."); return; }
        LimpiarBotones();
        if (_data.botonResponder != null) _data.botonResponder.onClick.AddListener(AlSeleccionarResponder);
        if (_data.botonCantar != null) _data.botonCantar.onClick.AddListener(AlSeleccionarCantar);
        _data.panelBoca.SetActive(true);
    }

    /// <summary>
    /// Cierra el panel de boca y limpia los listeners de los botones.
    /// </summary>
    public void CerrarPanel()
    {
        if (_data.panelBoca == null) return;
        _data.panelBoca.SetActive(false);
        LimpiarBotones();
    }

    /// <summary>Elimina todos los listeners de los botones Responder y Cantar.</summary>
    private void LimpiarBotones()
    {
        if (_data.botonResponder != null) _data.botonResponder.onClick.RemoveAllListeners();
        if (_data.botonCantar != null) _data.botonCantar.onClick.RemoveAllListeners();
    }

    /// <summary>Cierra el panel y dispara el evento OnResponderSeleccionado.</summary>
    private void AlSeleccionarResponder() { CerrarPanel(); OnResponderSeleccionado?.Invoke(); }

    /// <summary>Cierra el panel y dispara el evento OnCantarSeleccionado.</summary>
    private void AlSeleccionarCantar() { CerrarPanel(); OnCantarSeleccionado?.Invoke(); }
}

```

TransicionZonaController.cs

163 líneas

Scripts/Modelos/transicion/TransicionZonaController.cs

Descripcion	Controlador de la transicion entre zonas de escena mediante un cuadro negro. Al activarse baja el cuadro negro (fade out), desactiva la zona de salida, activa la zona de entrada, teletransporta al jugador y a la camara a los puntos destino, y luego sube el cuadro (fade in). Opcionalmente cambia el audio de caminata y reactiva el follow de camara. Dependencias: TransicionZonaData, CamaraData.
--------------------	--

```

using UnityEngine;
using System.Collections;
/// <summary>
/// Trigger de transicion entre zonas.
/// Al chocar el jugador: bloquea movimiento, baja cuadro negro,
/// teletransporta, intercambia zonas, sube cuadro negro y reactiva movimiento.
/// Requiere: Collider con isTrigger = true en el mismo GameObject.
/// </summary>
[RequireComponent(typeof(TransicionZonaData))]
[RequireComponent(typeof(Collider))]
public class TransicionZonaController : MonoBehaviour
{
    private TransicionZonaData _data;
    private JugadorData _jugadorData;
    private bool _enTransicion = false;
    #region Unity Callbacks
    private void Awake()
    {
        _data = GetComponent<TransicionZonaData>();
        AsegurarTrigger();
    }
    private void OnTriggerEnter(Collider other)
    {
        if (!_enTransicion) return;
        if (!other.CompareTag("Player")) return;
        _jugadorData = other.GetComponent<JugadorData>();
        if (_jugadorData == null)
        {
            Debug.LogWarning("[TransicionZonaController] El jugador no tiene JugadorData.");
            return;
        }
        if (!ValidarReferencias()) return;
        StartCoroutine(EjecutarTransicion(other.transform));
    }
    #endregion
    #region Inicializacion
    /// <summary>
    /// Garantiza que el Collider tenga isTrigger = true, corrigiendolo automaticamente si no lo esta.
    /// </summary>
    private void AsegurarTrigger()
    {
        Collider col = GetComponent<Collider>();
        if (!col.isTrigger)
        {
            col.isTrigger = true;
            Debug.LogWarning("[TransicionZonaController] Collider no era trigger. Se corrigio automaticamente.");
        }
    }
    /// <summary>Verifica que el cuadro negro, los puntos de teletransporte y la camara esten
    asignados.</summary>
    /// <returns>True si todas las referencias son validas para ejecutar la transicion.</returns>
    private bool ValidarReferencias()
    {
        if (_data.cuadroNegro == null)
        {
            Debug.LogError("[TransicionZonaController] cuadroNegro no asignado.");
            return false;
        }
        if (_data.puntoJugador == null)
        {
            Debug.LogError("[TransicionZonaController] puntoJugador no asignado.");
        }
    }

```

```

return false;
}
if (_data.puntoCamara == null)
{
    Debug.LogError("[TransicionZonaController] puntoCamara no asignado.");
    return false;
}
if (_data.camara == null)
{
    Debug.LogError("[TransicionZonaController] camara no asignada.");
    return false;
}
return true;
}
#endregion
#region Transicion
    /// <summary>
    /// Ejecuta la secuencia de transicion: bloquea movimiento, baja el cuadro negro,
    /// teletransporta jugador y camara, intercambia zonas y sube el cuadro negro.
    /// </summary>
    /// <param name="jugador">Transform del jugador que activo el trigger.</param>
    private IEnumerator EjecutarTransicion(Transform jugador)
    {
        _enTransicion = true;
        // 1 - Bloquear movimiento
        _jugadorData.puedeMoverse = false;
        if (_data.audioTransicion != null)
            _data.audioTransicion.Play();
        // 2 - Bajar cuadro negro hasta Y = 0
        yield return StartCoroutine(MoverCuadro(0f));
        // 3 - Teletransportar jugador y camara
        jugador.position = _data.puntoJugador.position;
        _data.camara.position = _data.puntoCamara.position;
        // 4 - Intercambiar zonas
        if (_data.zonaEntrada != null)
            _data.zonaEntrada.SetActive(true);
        if (_data.zonaSalida != null)
            _data.zonaSalida.SetActive(false);
        if (_data.nuevaCaminata != null && _jugadorData.audioCaminata != null)
            _jugadorData.audioCaminata.clip = _data.nuevaCaminata;
        CamaraData camaraData = _data.camara.GetComponent<CamaraData>();
        if (camaraData != null)
            camaraData.seguirJugador = _data.SeguirJugadorSiguiente;
        // 5 - Subir cuadro negro hasta Y = 2
        yield return StartCoroutine(MoverCuadro(2f));
        // 6 - Reactivar movimiento
        _jugadorData.puedeMoverse = true;
        _enTransicion = false;
    }
    /// <summary>
    /// Mueve el cuadro negro suavemente hasta el Y destino.
    /// </summary>
    private IEnumerator MoverCuadro(float yDestino)
    {
        Vector3 posInicial = _data.cuadroNegro.localPosition;
        Vector3 posDestino = new Vector3(posInicial.x, yDestino, posInicial.z);
        float distancia = Mathf.Abs(posInicial.y - yDestino);
        if (distancia < 0.001f) yield break;
        float tiempoTotal = distancia / _data.velocidadTransicion;
        float tiempoTranscurrido = 0f;
        while (tiempoTranscurrido < tiempoTotal)
        {
            tiempoTranscurrido += Time.deltaTime;
            float t = Mathf.Clamp01(tiempoTranscurrido / tiempoTotal);
            _data.cuadroNegro.localPosition = Vector3.Lerp(posInicial, posDestino, t);
            yield return null;
        }
    }

```

```
}  
_data.cuadroNegro.localPosition = posDestino;  
}  
#endregion  
}
```